DevOps and Containerization Lab Report

Khushi Yadav

Contents

1 Experiment 1 **15**
1.0.1 Note:- As I am a mac-user I have performed the experiment on Someone else WINDOWS system. . . . 15
1.1 Comparison of VMs and containers using Ubuntu and Nginx . . . 15
1.2 Objective . . . 15
1.3 Theory . . . 15
1.3.1 Step 1: Install VirtualBox and vagrant. . . . 16
1.3.2 Step 2: Install ubuntu . . . 16
1.3.3 Step 3: start the VM . . . 17
1.3.4 Step 4: Access the VM . . . 18
1.3.5 Step 5: Verifying. . . . 18
1.3.6 Step 6: Destroying and halting VMs . . . 18
1.3.7 Step 7: Checking wsl . . . 18
1.3.8 Step 8: Configuring . . . 20
1.3.9 Step 9: Install Docker inside WSL . . . 21
1.3.10 Step 10: Verify Docker Installation . . . 21
1.3.11 Step 11: Install Nginx in Container . . . 22
1.3.12 Step 12: Compare VM and Container Performance . . . 22
1.4 Result . . . 23
1.5 Conclusion . . . 23

2 Experiment 2: Docker Installation, Configuration, and Running Images **23**
2.1 Objective . . . 23
2.2 Commands Executed . . . 24
2.2.1 1. Docker Installation & Configuration Check . . . 24
2.2.2 2. Pull Docker Image . . . 24
2.2.3 3. Run Docker Container with Port Mapping . . . 24
2.2.4 4. Container Lifecycle Management . . . 26
2.2.5 5. Remove Docker Image . . . 26
2.2.6 Result . . . 26

3 Experiment 3: Deploying NGINX Using Different Base Images **26**
3.1 Objective . . . 26

4 Prerequisites **26**

5 Part 1: Deploy NGINX Using Official Image **27**
5.1 Step 1: Pull Image . . . 27
5.2 Step 2: Run Container . . . 27
5.3 Step 3: Verify . . . 27

5.4 [redacted] 28

6 Part 2: Custom NGINX Using Ubuntu Base Image **28**

6.1 Step 1: Create Dockerfile 28

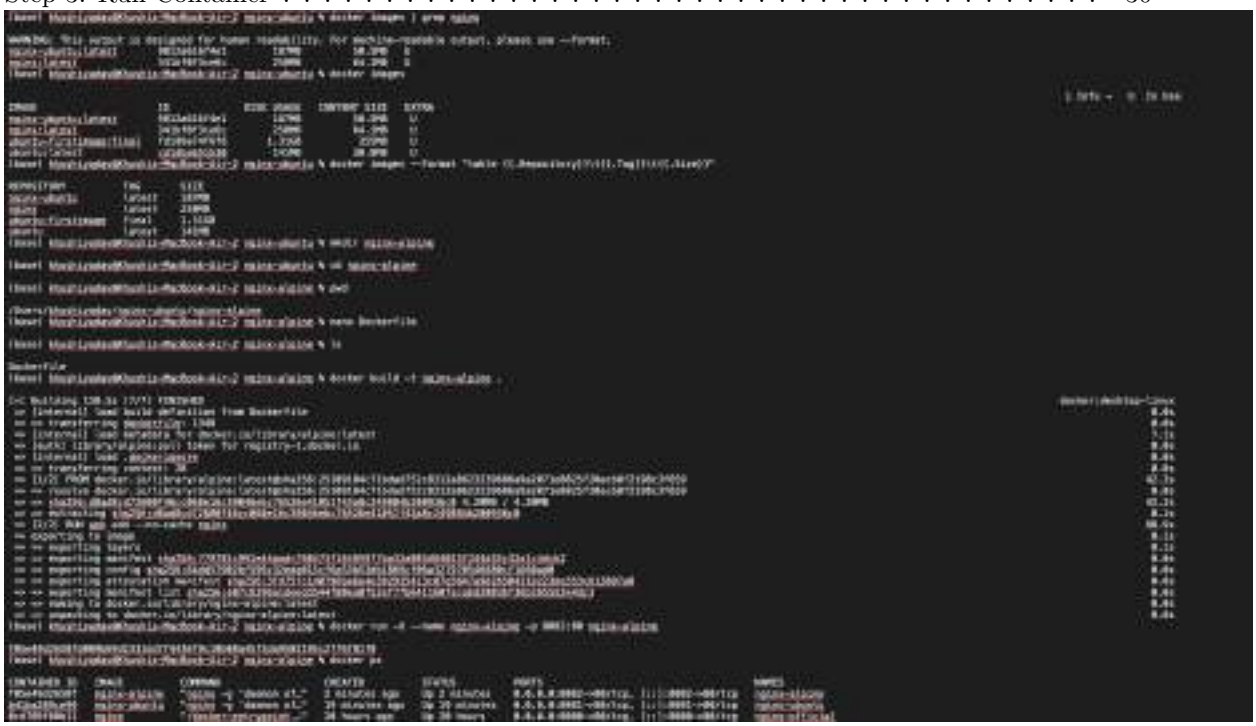
6.2 Step 2: Build Image 28

6.3 Step 3: Run Container 28

[redacted]

6.4 29

7 Part 3: Custom NGINX Using Alpine Base Image **29**
7.1 Step 1: Create Dockerfile 29
7.2 Step 2: Build Image 30
7.3 Step 3: Run Container 30



7.4 30

8 Issue Faced in Alpine **30**

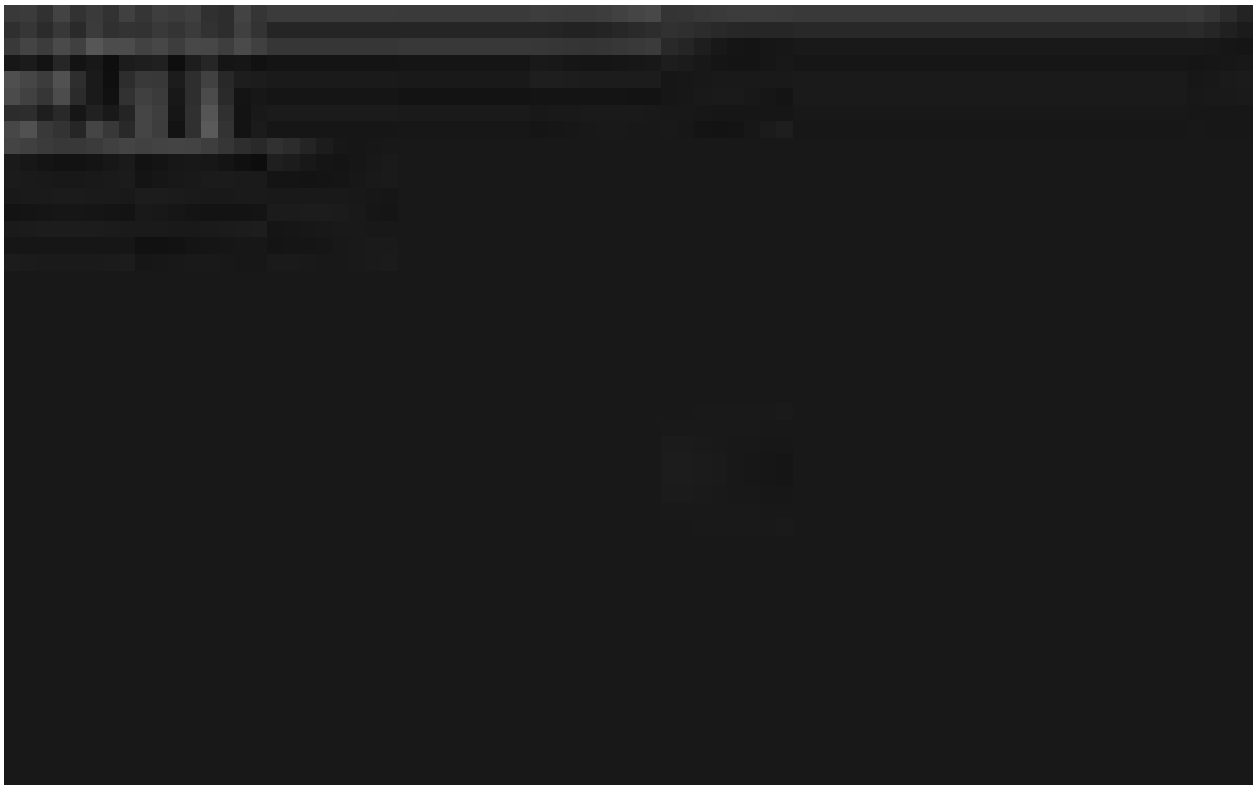
8.1 31
8.2 Debugging Process 31

8.2.1 Checked Container 31
8.2.2 Entered Container 31
8.2.3 Checked Default Config 31

9 Fix Applied **32**
9.0.1 Created Web Directory 32
9.0.2 Modified Configuration 32
9.0.3 Tested Configuration 32
9.0.4 Reloaded NGINX 32

9.1 34

10 Image Size Comparison **34**
10.1 Results: 34

10.2  35

11 Observations **35**

12 Conclusion **35**

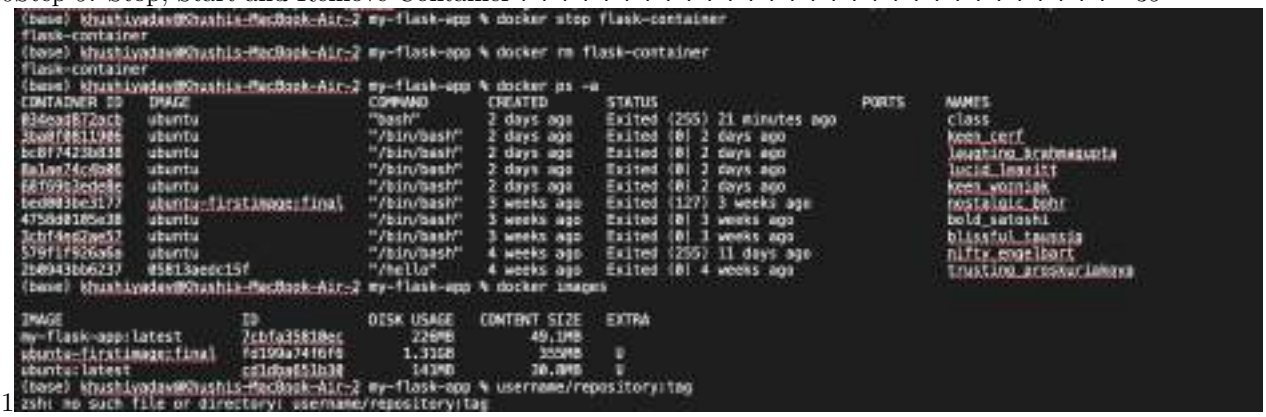
13 Cleanup Commands **36**

14 Key Learnings **36**

15 Experiment 4 – Docker Essentials **36**
15.1 Aim 36

16 Part 1 – Containerizing a Flask Application **36**
16.1 Step 1: Create Project Directory 36
16.2 Step 2: Create Flask Application (app.py) 36
16.3 Step 3: Create requirements.txt 37
16.4 Step 4: Create Dockerfile 37
16.5 Step 5: Build Docker Image 37
16.6 Step 6: Run Container 37
16.7 Step 7: Test Health Endpoint 39
16.8 Step 8: View Logs 39

16.9


39
16.10Step 9: Stop, Start and Remove Container 39
16.11


40

17 Part 2 – Docker Hub Operations **40**
17.1 Step 10: Tag Image 40
17.2 Step 11: Login to Docker Hub 40
17.3 Step 12: Push Image 41
17.4 Step 13: Remove Local Image 41
17.5 Step 14: Pull Image from Docker Hub 41
17.6 Step 15: Run Pulled Image 41

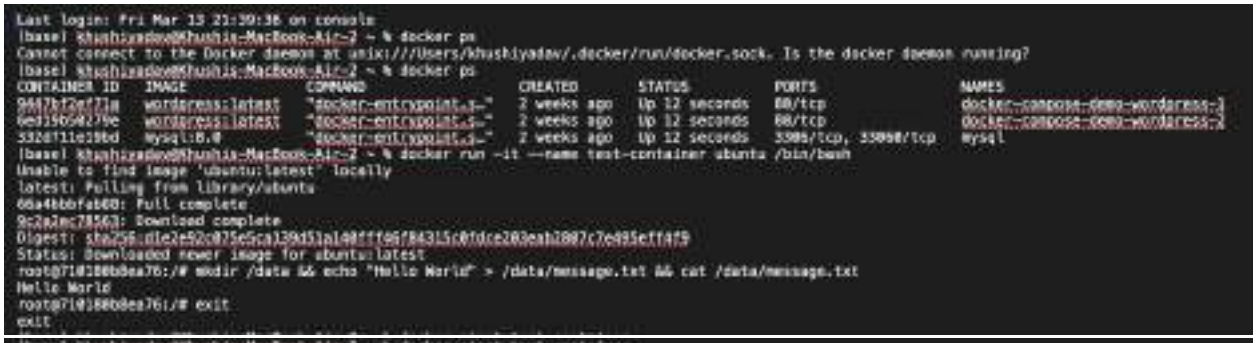
18 Errors and Issues Faced **42**
18.1 -> Docker Daemon Not RunningAt 9 AM 42
18.2 -> Port 5000 Already in Use 42
18.3 -> Container Name Already in Use 42

19 Results **43**

20 Conclusion **43**

21 Experiment 5: **44**

22 Docker — Volumes, Environment Variables, Monitoring & Networks **44**
22.1 Table of Contents 44
22.2 Overview 44
22.3 Part 1: Docker Volumes 44
22.3.1 Lab 1: Understanding Data Persistence 44

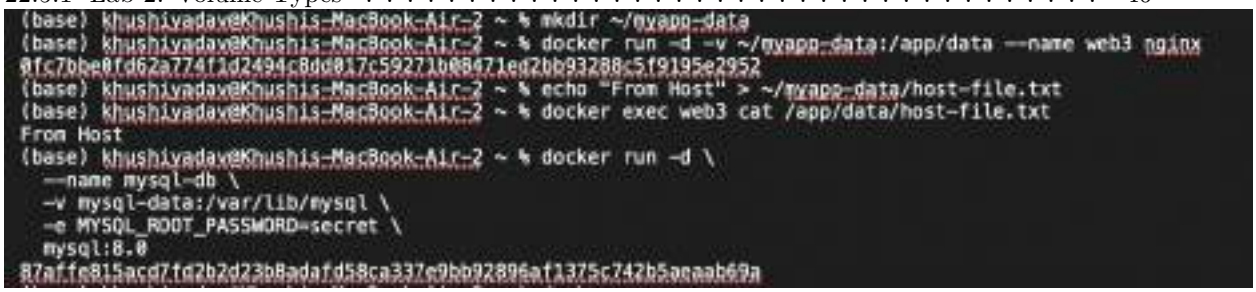
22.4


45

22.5

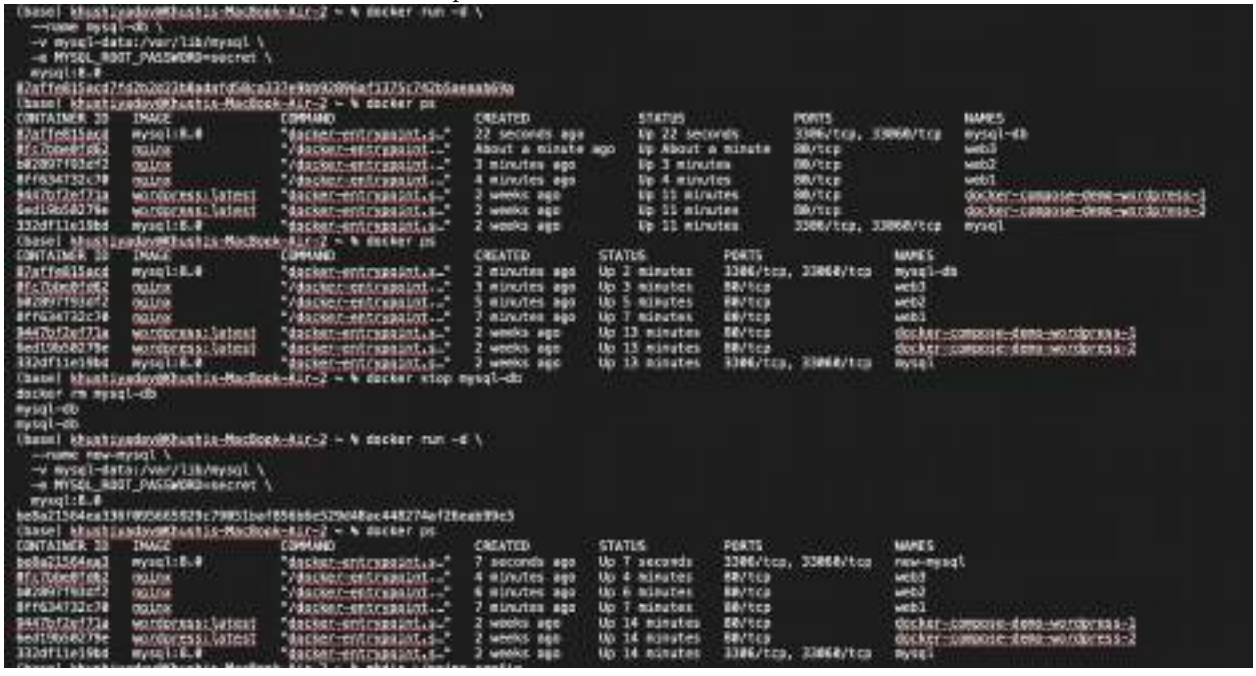

45

22.5.1 Lab 2: Volume Types 46



22.6 47

22.6.1 Lab 3: Practical Volume Examples 47

22.7

47
48

```
(base) khushiyadav@Khushis-MacBook-Air-2 ~ % mkdir ~/nginx-config
(base) khushiyadav@Khushis-MacBook-Air-2 ~ % echo 'server {
  listen 80;
  server_name localhost;
  location / {
    return 200 "Hello from mounted config!";
  }
}' > ~/nginx-config/nginx.conf
(base) khushiyadav@Khushis-MacBook-Air-2 ~ % docker run -d \
  --name nginx-custom \
  -p 8080:80 \
  -v ~/nginx-config/nginx.conf:/etc/nginx/conf.d/default.conf \
  nginx
1a8a47eb558c657686ad247e0e1652a92359d4dba7b5044dcd12f4f1d4b6a66e
(base) khushiyadav@Khushis-MacBook-Air-2 ~ % curl http://localhost:8080
Hello from mounted config!%
```

22.8 49
22.8.1 Lab 4: Volume Management Commands 49

```
khushiyadav@Khushis-MacBook-Air-2 ~ % docker volume ls
DRIVER    VOLUME NAME
local     8656cd6c89f184e4e895d0b2469679b92906d870d95d17f5af70bfac78a5f777
local     docker-compose-demo_mysql_data
local     docker-compose-demo_wp_content
local     mydata
local     mysql-data
(base) khushiyadav@Khushis-MacBook-Air-2 ~ % docker volume create app-volume
app-volume
(base) khushiyadav@Khushis-MacBook-Air-2 ~ % docker volume inspect app-volume
[
    {
        "CreatedAt": "2026-03-14T17:48:24Z",
        "Driver": "local",
        "Labels": null,
        "Mountpoint": "/var/lib/docker/volumes/app-volume/_data",
        "Name": "app-volume",
        "Options": null,
        "Scope": "local"
    }
]
(base) khushiyadav@Khushis-MacBook-Air-2 ~ % docker volume prune
WARNING! This will remove anonymous local volumes not used by at least one container.
Are you sure you want to continue? [y/N] y
Total reclaimed space: 0B
```

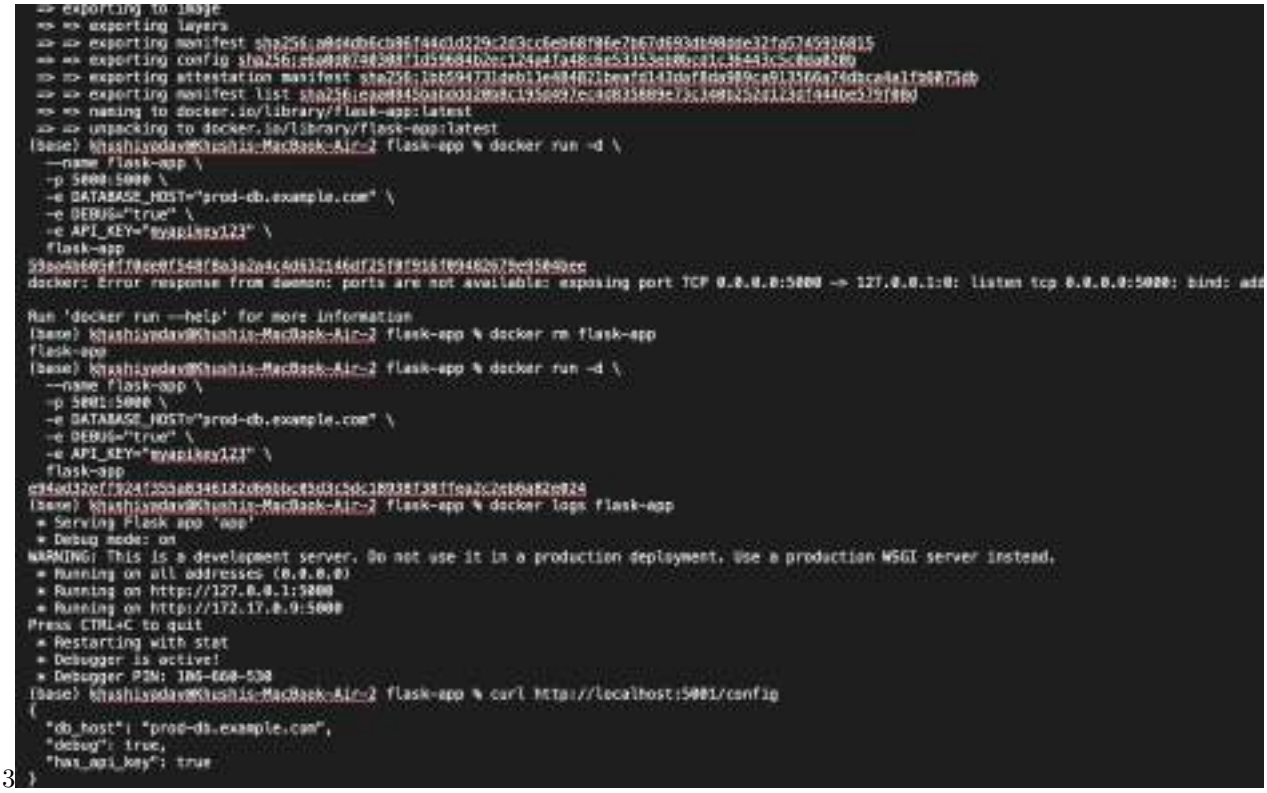
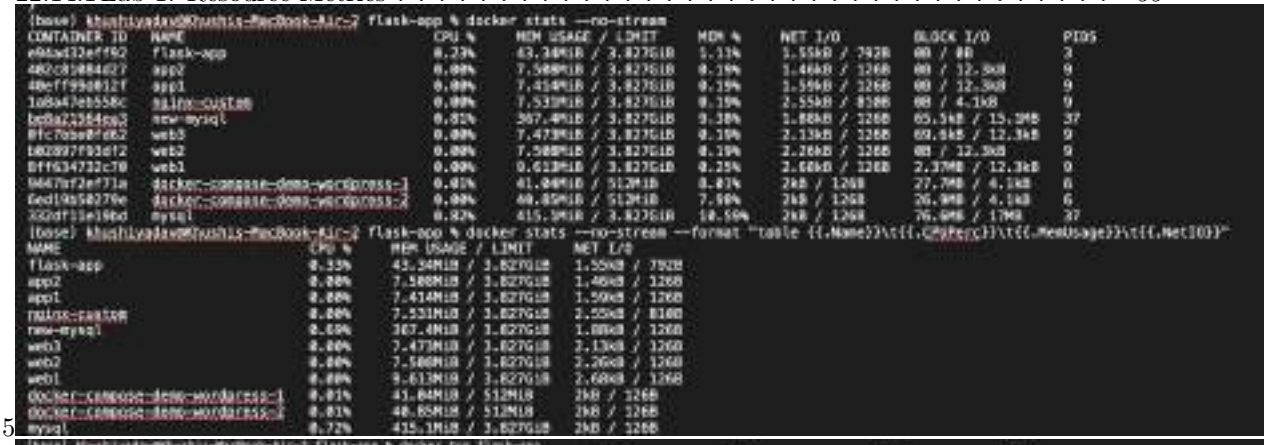
22.9 49
22.10 Part 2: Environment Variables 49
22.10.1 Lab 1: Setting Environment Variables 50

```
(base) khushiyadav@Khushis-MacBook-Air-2 ~ % docker run -d \
  --name app1 \
  -e VAR1=value1 \
  -e VAR2=value2 \
  -e VAR3=value3 \
  nginx
40eff99d012f40757151c10d9ab370f7d60c1a612c5fd99f7f015f02d6e0b7d8
(base) khushiyadav@Khushis-MacBook-Air-2 ~ % docker exec app1 env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=40eff99d012f
VAR1=value1
VAR2=value2
VAR3=value3
NGINX_VERSION=1.29.6
NJS_VERSION=0.9.6
NJS_RELEASE=1~trixie
ACME_VERSION=0.3.1
PKG_RELEASE=1~trixie
DYNPKG_RELEASE=1~trixie
HOME=/root
```

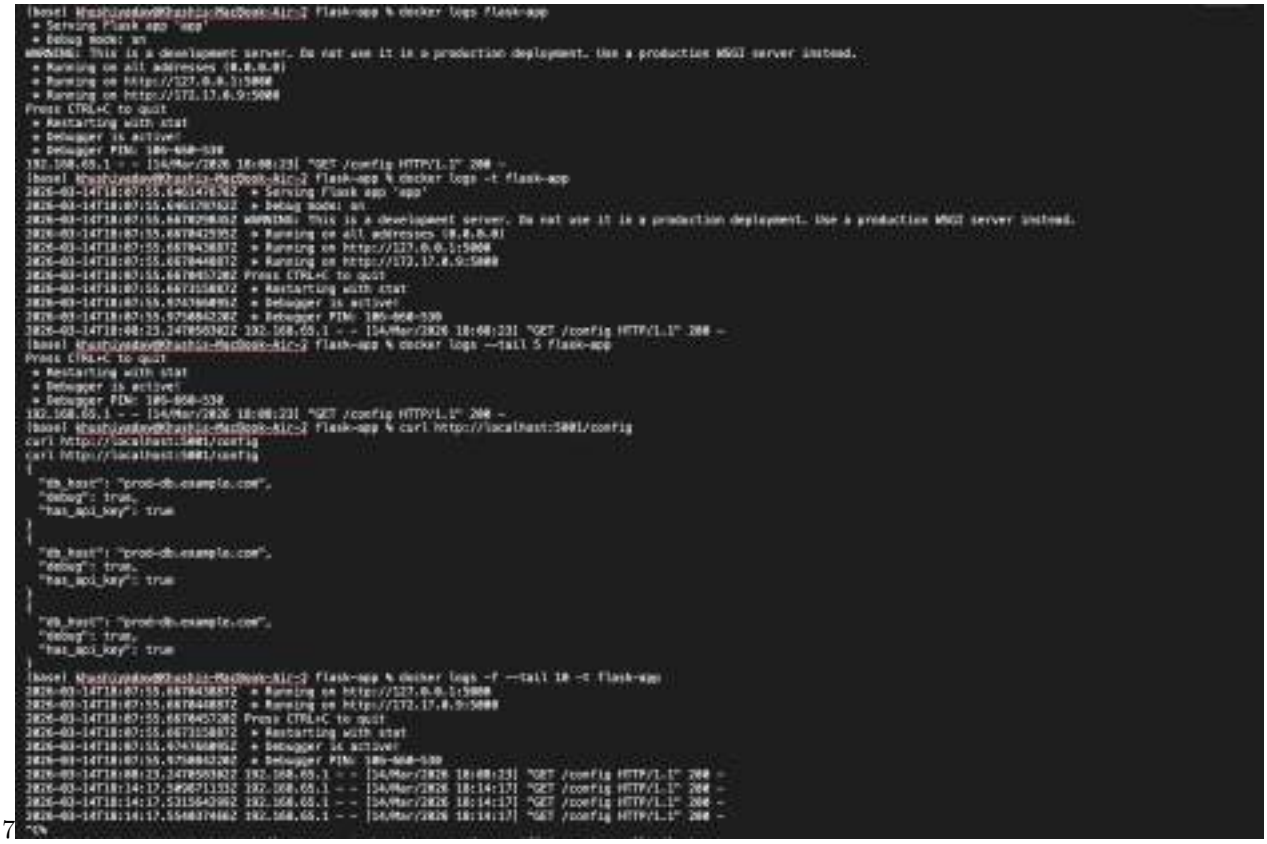
22.11 50

```
(base) khushiyadav@Khushis-MacBook-Air-2 ~ % cd ~ && echo "DATABASE_HOST=localhost" > .env
echo "DATABASE_PORT=5432" >> .env
echo "API_KEY=secret123" >> .env
(base) khushiyadav@Khushis-MacBook-Air-2 ~ % cat ~/.env
DATABASE_HOST=localhost
DATABASE_PORT=5432
API_KEY=secret123
(base) khushiyadav@Khushis-MacBook-Air-2 ~ % docker run -d \
  --name app2 \
  --env-file ~/.env \
  nginx
402c81084d27024f99a8c8fa24bf69c0783ab907726625c7f6ae8edaa3f87e60
(base) khushiyadav@Khushis-MacBook-Air-2 ~ % docker exec app2 env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=402c81084d27
DATABASE_HOST=localhost
DATABASE_PORT=5432
API_KEY=secret123
NGINX_VERSION=1.29.6
NJS_VERSION=0.9.6
NJS_RELEASE=1~trixie
ACME_VERSION=0.3.1
PKG_RELEASE=1~trixie
DYNPKG_RELEASE=1~trixie
HOME=/root
```

22.12 51
22.12.1 Lab 2: Flask App with Environment Variables 51

22.13

55
22.14Part 3: Docker Monitoring 55
22.14.1Lab 1: Resource Metrics 55
22.15

56
22.16

56
22.16.1Lab 2: Log Monitoring 56

22.17  57
22.17.1 Lab 3: Container Inspection 57
22.18  57
22.19 Part 4: Docker Networks 57
22.19.1 Lab 1: Network Types 58

```
(base) khushiyadav@Khushis-MacBook-Air-2 flask-app % docker network ls
NETWORK ID     NAME                          DRIVER    SCOPE
f484b063b6a8   bridge                        bridge    local
27941e90f3c5   docker-compose-demo_default   bridge    local
1eb6430c10a2   host                          host      local
9297a58de8a4   none                          null      local
(base) khushiyadav@Khushis-MacBook-Air-2 flask-app % docker network create my-network
45070714afc966b8c3aac31eb6c5eb8edb7eb549d4ebd29a2c3cefe411b4dedd
(base) khushiyadav@Khushis-MacBook-Air-2 flask-app % docker network inspect my-network
[
    {
        "Name": "my-network",
        "Id": "45070714afc966b8c3aac31eb6c5eb8edb7eb549d4ebd29a2c3cefe411b4dedd",
        "Created": "2026-03-14T18:16:40.143954879Z",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv4": true,
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": {},
            "Config": [
                {
                    "Subnet": "172.19.0.0/16",
                    "IPRange": "",
                    "Gateway": "172.19.0.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Options": {
            "com.docker.network.enable_ipv4": "true",
            "com.docker.network.enable_ipv6": "false"
        },
        "Labels": {},
        "Containers": {},
        "Status": {
            "IPAM": {
                "Subnets": {
                    "172.19.0.0/16": {
                        "IPsInUse": 3,
                        "DynamicIPsAvailable": 65533
                    }
                }
            }
        }
    }
]
```

22.20 58

```
(base) khushiyadav@Khushis-MacBook-Air-2 flask-app % docker run -d --name host-app --network host nginx
a08499395968fb6356d1a7a2e5d2398c9c4791576edc72c863570e87b81829bf
(base) khushiyadav@Khushis-MacBook-Air-2 flask-app % docker run -d --name isolated-app --network none alpine sleep 3600
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
d8ad8cd72600: Pull complete
37093440b0e0: Download complete
cb04f20e6ea6: Download complete
Digest: sha256:25109184c71adad752c8312a862323900a9a2871e8825f20acb8f2198c3f659
Status: Downloaded newer image for alpine:latest
054a583385291680c115f1ebae9c2698d06e94542a7434c6eb097a0887fc1212
(base) khushiyadav@Khushis-MacBook-Air-2 flask-app % docker exec isolated-app ifconfig
lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

(base) khushiyadav@Khushis-MacBook-Air-2 flask-app % docker exec net-web1 nslookup net-web2
OCI runtime exec failed: exec failed: unable to start container process: exec: "nslookup": executable file not found in $PATH
```

22.21 59

22.22 61

22.23 Part 5: Real-World Example 61

22.23.1 Architecture 61

22.23.2 Step 19 — Deploy the Stack 62

22.24 62

22.24.1 Step 20 — Connect Flask and Verify Network 62

22.24.2 Step 20b — Verify Logs 64

22.24.3 Step 21 — Monitor All Services 64

22.25 Cleanup 64

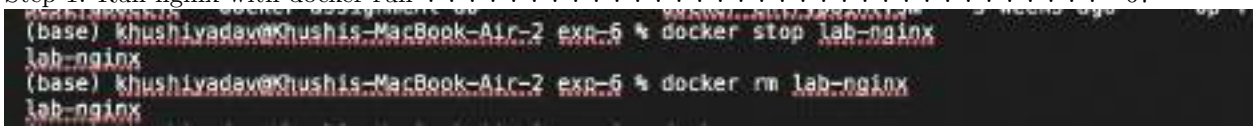
22.26 Errors Encountered & Fixes 66

22.27 Key Takeaways 67

23 Experiment 6 – Docker and Compose Exercises **67**

23.1 Aim 67

24 Part 1 – Single Container Comparison **67**

24.1 Step 1: Run nginx with docker run 67
24.2 
(base) khushiyadav@Khushis-MacBook-Air-2 exp-6 % docker stop lab-nginx
lab-nginx
(base) khushiyadav@Khushis-MacBook-Air-2 exp-6 % docker rm lab-nginx
lab-nginx
. 68
24.3 Step 2: Run nginx with docker-compose 68
24.4 Architecture 81
24.5 Process Done 81
24.5.1 Step 1: Jenkins Setup 81
24.5.2 Step 2: Installed Jenkins Plugins 82
24.5.3 Step 3: Added Docker Hub Credentials 82
24.5.4 Step 4: Created GitHub Repository 82
24.5.5 Step 5: Created Pipeline Job 82
24.5.6 Step 6: Configured GitHub Webhook 82
24.6 Jenkinsfile Pipeline 82
24.7 How It Works 83
24.8 Testing 83
24.8.1 Manual Build 83
24.8.2 Automated Trigger 83
24.9 Quick Commands 84
24.10 Key Features 84
24.11 Conclusion 84
24.12 Experiment Steps 87
24.12.1 Step 1: Build the Docker Image from Dockerfile 87
24.12.2 Step 2: Run the Docker Container 88
24.12.3 Step 3: Verify Container Status 88
24.12.4 Step 4: Test SSH Connection to Container 88
24.12.5 Step 5: Configure Ansible Inventory 88
24.12.6 Step 6: Run Ansible Playbook on Container 89
24.12.7 Step 7: Verify Automation Results 89
24.13 Cleanup Commands 89
24.14 Key Learnings 90
24.15 Troubleshooting 90
24.15.1 Issue: Permission Denied (publickey) 90
24.15.2 Issue: Connection Refused 90
24.15.3 Issue: Ansible Connection Failed 90
24.16 Conclusion 90

25 Lab 10: CI/CD Pipeline with Jenkins and SonarQube 90
25.1 Introduction 90
25.2 Prerequisites 90
25.3 Project Structure 90
25.4 Step-by-Step Instructions 91
25.5 Jenkins Pipeline Overview 91
25.6 SonarQube Integration 91
25.7 Screenshots 91
25.8 Troubleshooting 94
25.9 Conclusion 95

26 Lab 11: Deploying WordPress with Docker Swarm 95
26.1 Introduction 95
26.2 Objectives 95
26.3 Prerequisites 95
26.4 Steps 95

26.4.1 1. Initialize Docker Swarm . . . 95
26.4.2 2. Deploy the Stack . . . 95
26.4.3 3. Check Service Status . . . 95
26.4.4 4. Access WordPress . . . 96
26.4.5 5. Scaling Services (Optional) . . . 96
26.5 docker-compose.yml Overview . . . 96
26.6 Troubleshooting . . . 96
26.7 Conclusion . . . 96
26.8 Screenshots . . . 98

27 Kubernetes Lab Experiment – Deploying WordPress using kubeadm 99
27.1 Overview . . . 99
27.2 Objectives . . . 99
27.3 Prerequisites . . . 99
27.4 Cluster Setup using kubeadm . . . 100
27.4.1 Step 1: Initialize the Kubernetes Cluster . . . 100
27.4.2 Step 2: Configure kubectl for the user . . . 100
27.4.3 Step 3: Install Network Plugin (Calico) . . . 100
27.5 Application Deployment (WordPress) . . . 100
27.5.1 Step 4: Deploy WordPress . . . 100
27.5.2 Step 5: Expose WordPress Service . . . 100
27.6 Verification Commands . . . 100
27.6.1 Check Nodes . . . 100
27.6.2 Check Pods . . . 100
27.6.3 Check Services . . . 100
27.7 Scaling the Application . . . 101
27.8 Self-Healing Test . . . 101
27.9 Observations . . . 101
27.10Issues Faced During Experiment . . . 101
27.11Conclusion . . . 101

1 Experiment 1

1.0.1 Note:- As I am a mac-user I have performed the experiment on Someone else WINDOWS system.

1.1 Comparison of VMs and containers using Ubuntu and Nginx

1.2 Objective

1. To understand the conceptual and practical differences between Virtual Machine and Container.
2. To install and configure a Virtual Machine using VirtualBox and Vagrant on windows.
3. To install and configure Containers using Docker inside WSL.
4. To deploy on Ubuntu-based Nginx web server in both environments.
5. To compare resource utilization, performance, and operational characteristics of VMs and Containers.

1.3 Theory

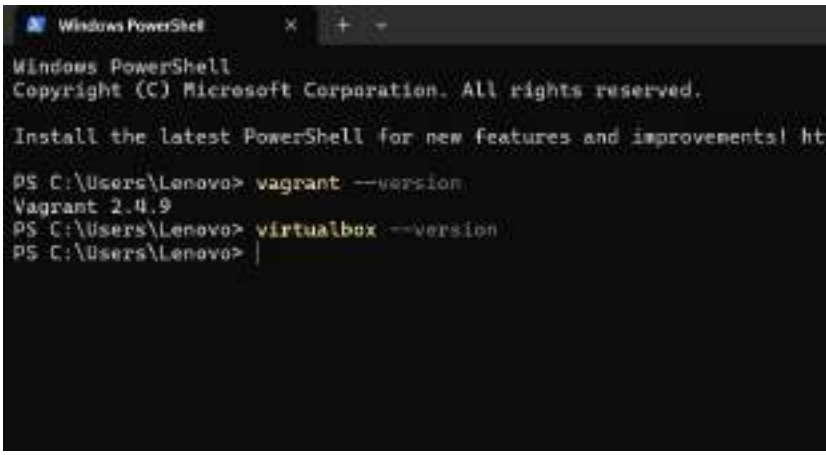
Virtualization and containerization are modern technologies used to run applications in isolated environments. A Virtual Machine (VM) emulates a complete physical system and runs its own operating system using a hypervisor. In this experiment, Ubuntu is installed as a VM using VirtualBox and Vagrant. VMs provide strong isolation but consume more system resources and have slower startup times due to the presence of a full guest OS.

Containers, on the other hand, are lightweight and share the host operating system's kernel. Using Docker

inside WSL, an Ubuntu-based Nginx container is deployed. Containers start faster, use fewer resources, and are highly portable, making them suitable for modern cloud and microservices-based applications.

Ubuntu is used as the base operating system in both environments for consistency, while Nginx is deployed as the web server due to its high performance and low resource usage. By deploying Nginx on both a VM and a container, this experiment compares resource utilization, performance, and operational characteristics of virtual machines and containers.

1.3.1 Step 1: Install VirtualBox and vagrant.



Download and install Oracle VirtualBox and vagrant.

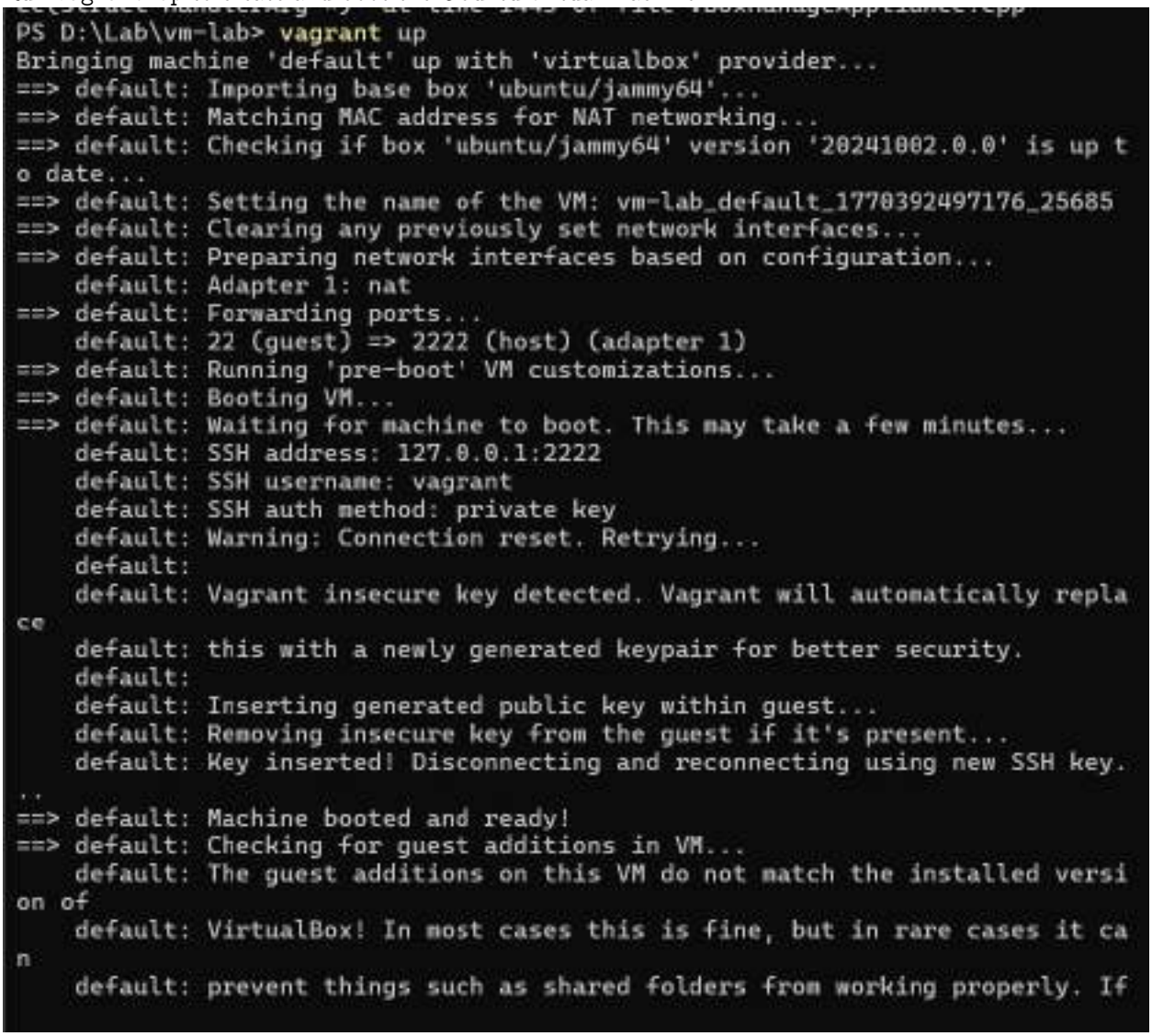
1.3.2 Step 2: Install ubuntu

Install ubuntu

```
PS D:\Lab\vm-lab> vagrant plugin update
Updating installed plugins...
Fetching vagrant-hostmanager-1.8.10.gem
All plugins are up to date.
PS D:\Lab\vm-lab> vagrant init ubuntu/jammy64
A `Vagrantfile` has been placed in this directory. You are now
ready to `vagrant up` your first virtual environment! Please read
the comments in the Vagrantfile as well as documentation on
`vagrantup.com` for more information on using Vagrant.
```

1.3.3 Step 3: start the VM

Run `vagrant up` to create and boot the Ubuntu virtual machine.



1.3.4 Step 4: Access the VM

```
PS D:\Lab\vm-lab> vagrant ssh
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-142

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Fri Feb  6 15:43:49 UTC 2

  System load:             0.2
  Usage of /:              3.9% of 38.70GB
  Memory usage:            22%
  Swap usage:              0%
  Processes:               106
  Users logged in:         0
  IPv4 address for enp0s3: 10.0.2.15
  IPv6 address for enp0s3: fd00::cf:77ff:fe5b:6d0


Expanded Security Maintenance for Applications is n

0 updates can be applied immediately.

Enable ESM Apps to receive additional future securi
See https://ubuntu.com/esm or run: sudo pro status


The list of available updates is more than a week o
To check for new updates run: sudo apt update
New release '24.04.3 LTS' available.
Run 'do-release-upgrade' to upgrade to it.
```

Use `vagrant ssh` to log into the Ubuntu VM.

1.3.5 Step 5: Verifying.

1.3.6 Step 6: Destroying and halting VMs

Use `vagrant halt`and `vagrant destroy` to stop and destroy the VM.

```
PS D:\Lab\vm-lab> vagrant halt
==> default: Attempting graceful shutdown of VM...
PS D:\Lab\vm-lab> vagrant destroy
    default: Are you sure you want to destroy the 'default' VM? [y/N] y
==> default: Destroying VM and associated drives...
PS D:\Lab\vm-lab>
```

1.3.7 Step 7: Checking wsl

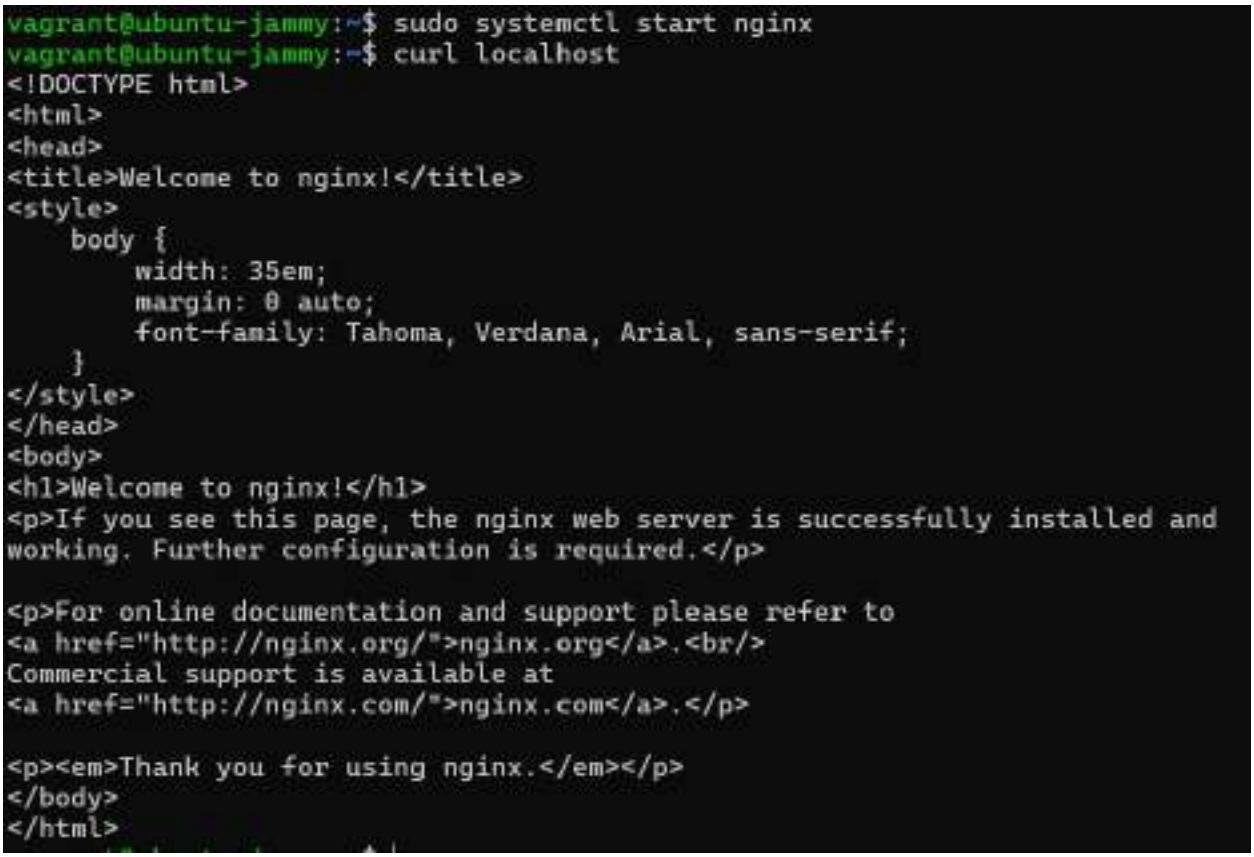


Figure 1: Step 5

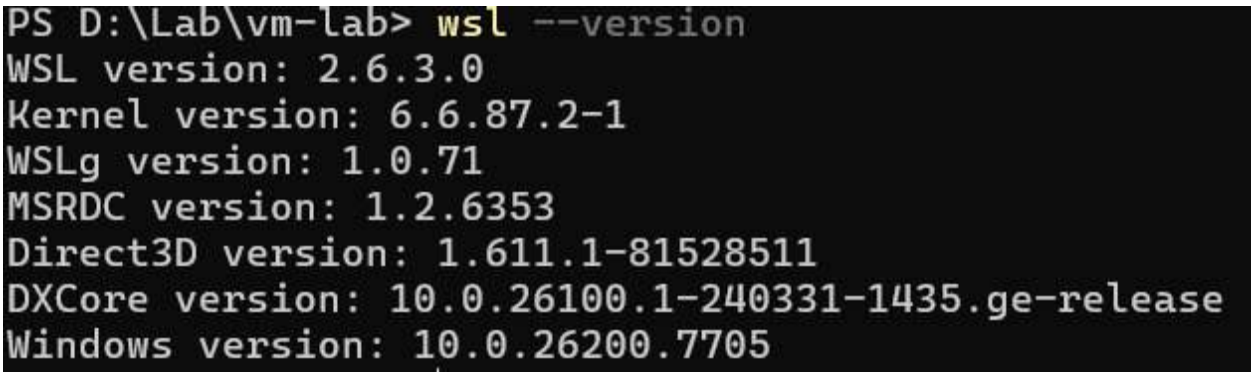
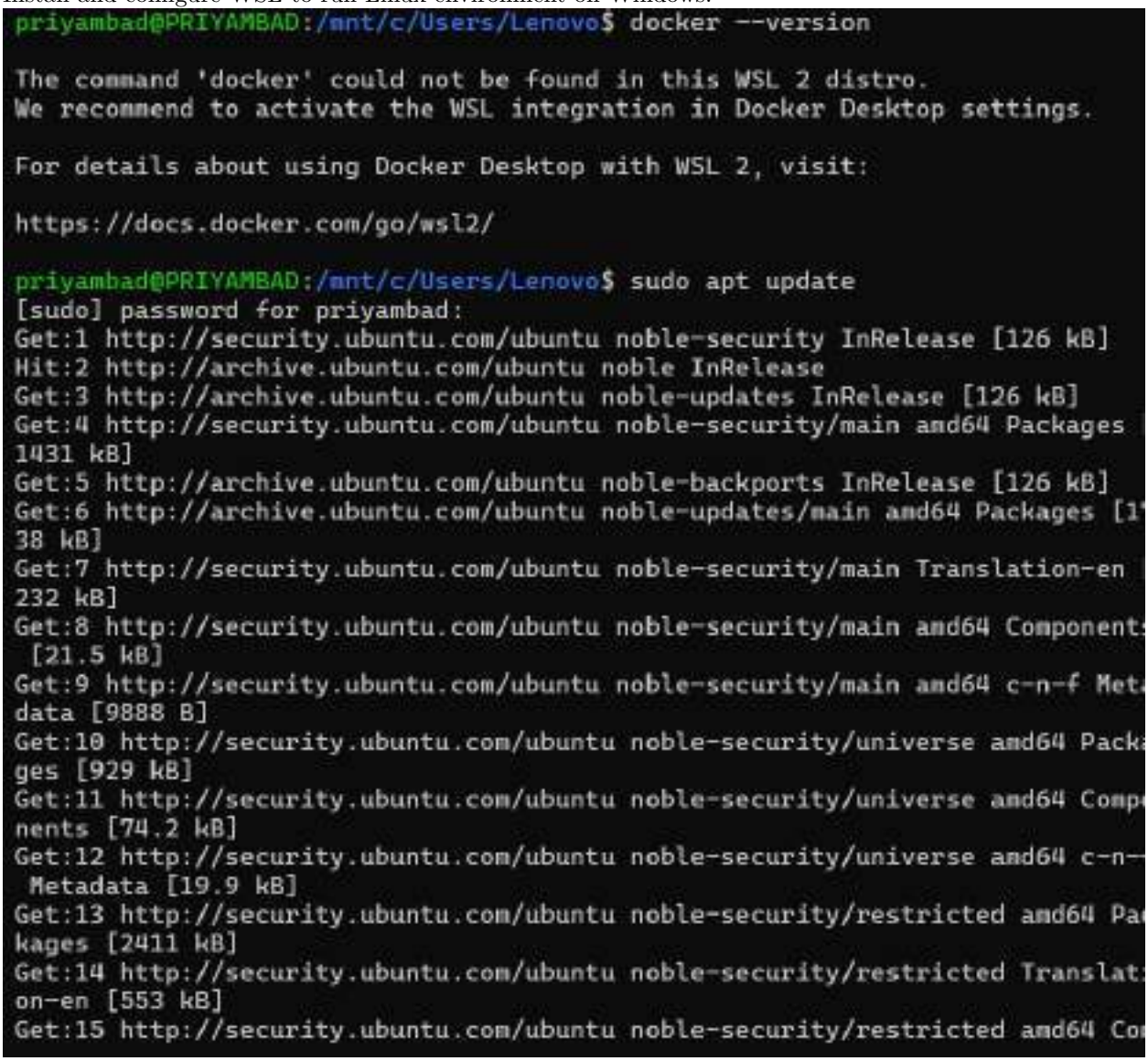


Figure 2: Step 7

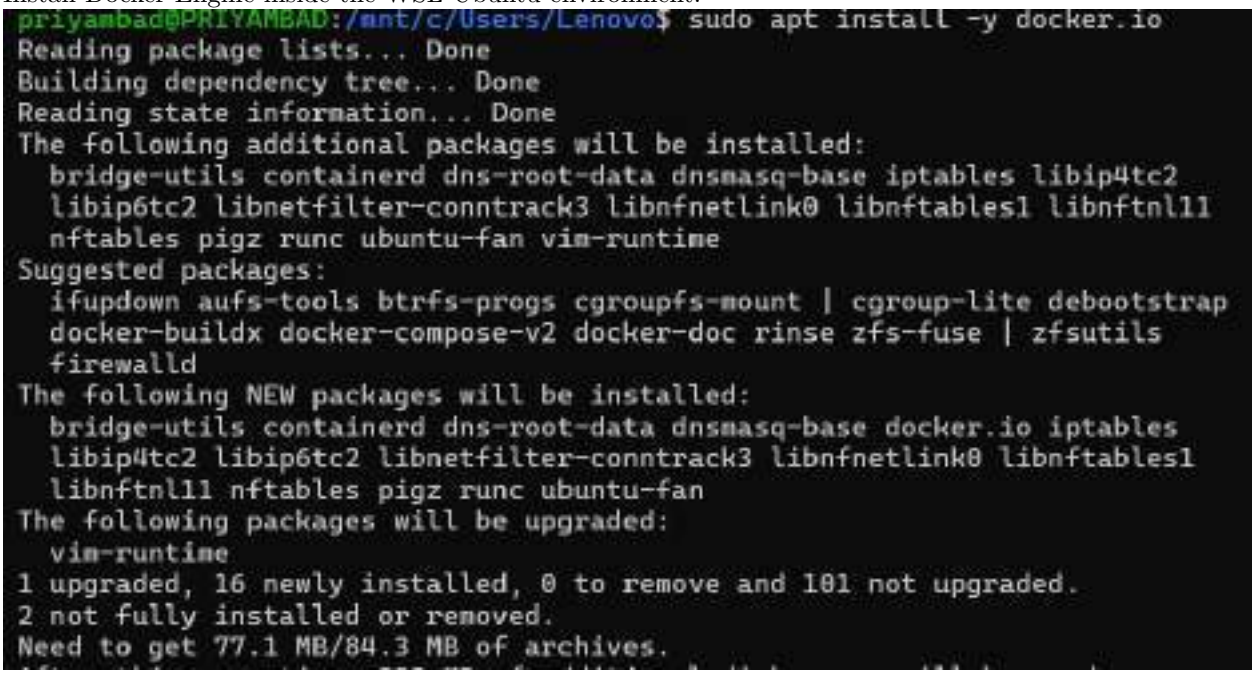
1.3.8 Step 8: Configuring

Install and configure WSL to run Linux environment on Windows.



1.3.9 Step 9: Install Docker inside WSL

Install Docker Engine inside the WSL Ubuntu environment.

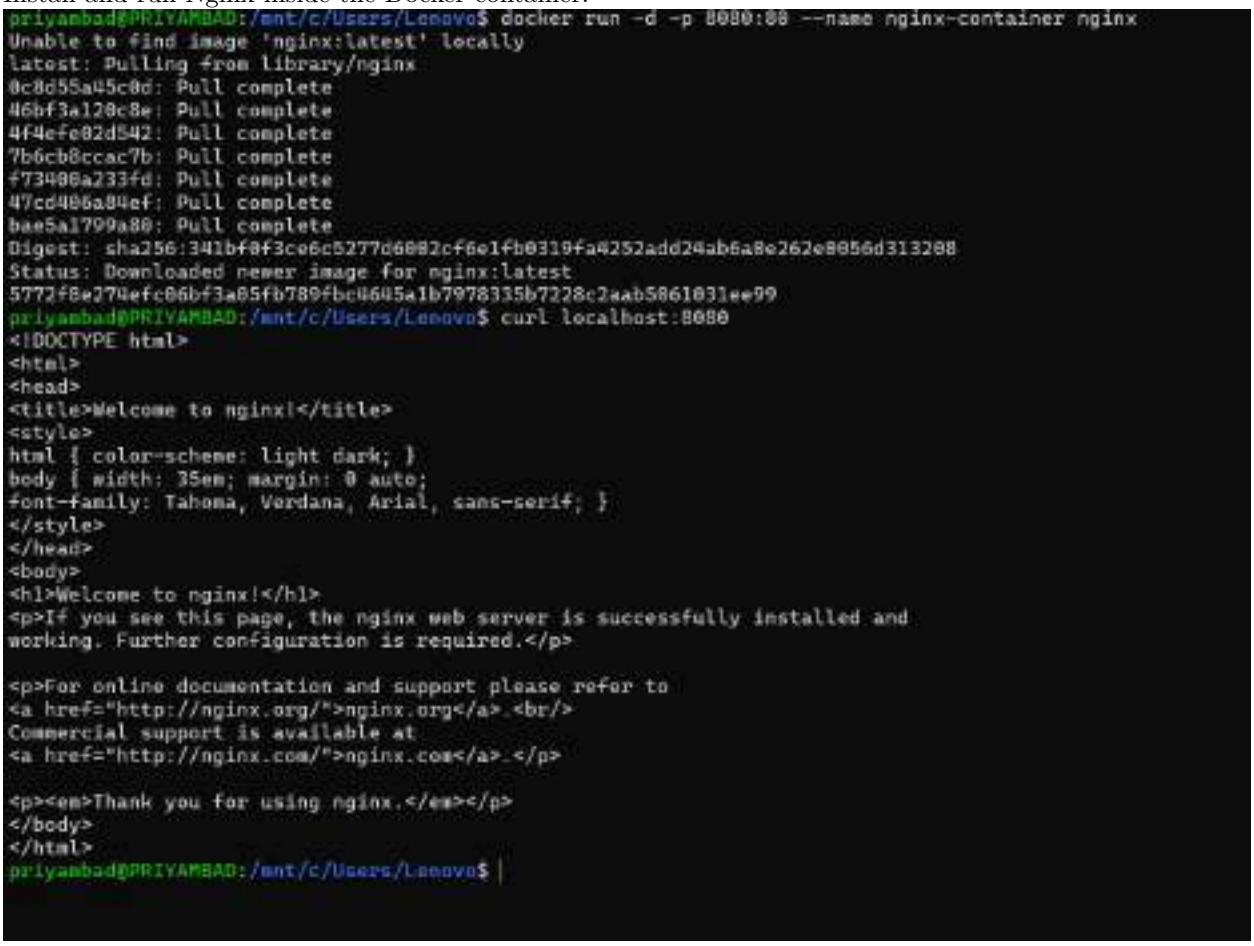


1.3.10 Step 10: Verify Docker Installation

Check Docker version and status to confirm successful installation.

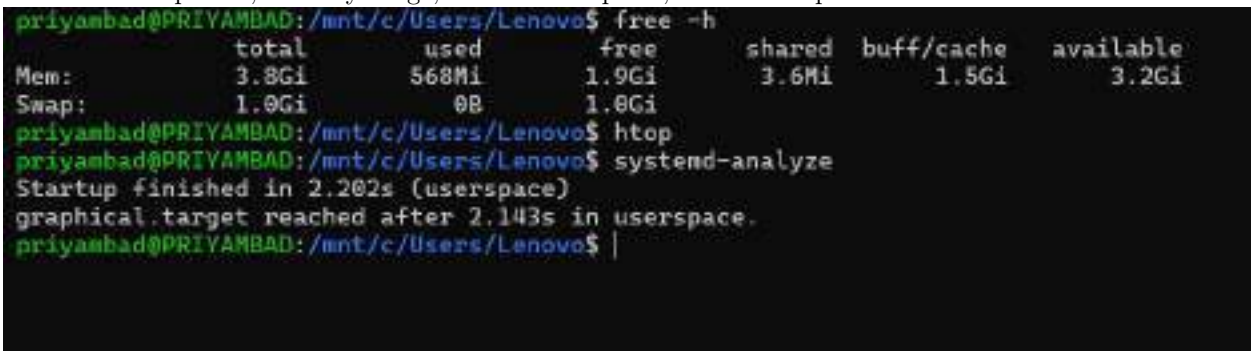
1.3.11 Step 11: Install Nginx in Container

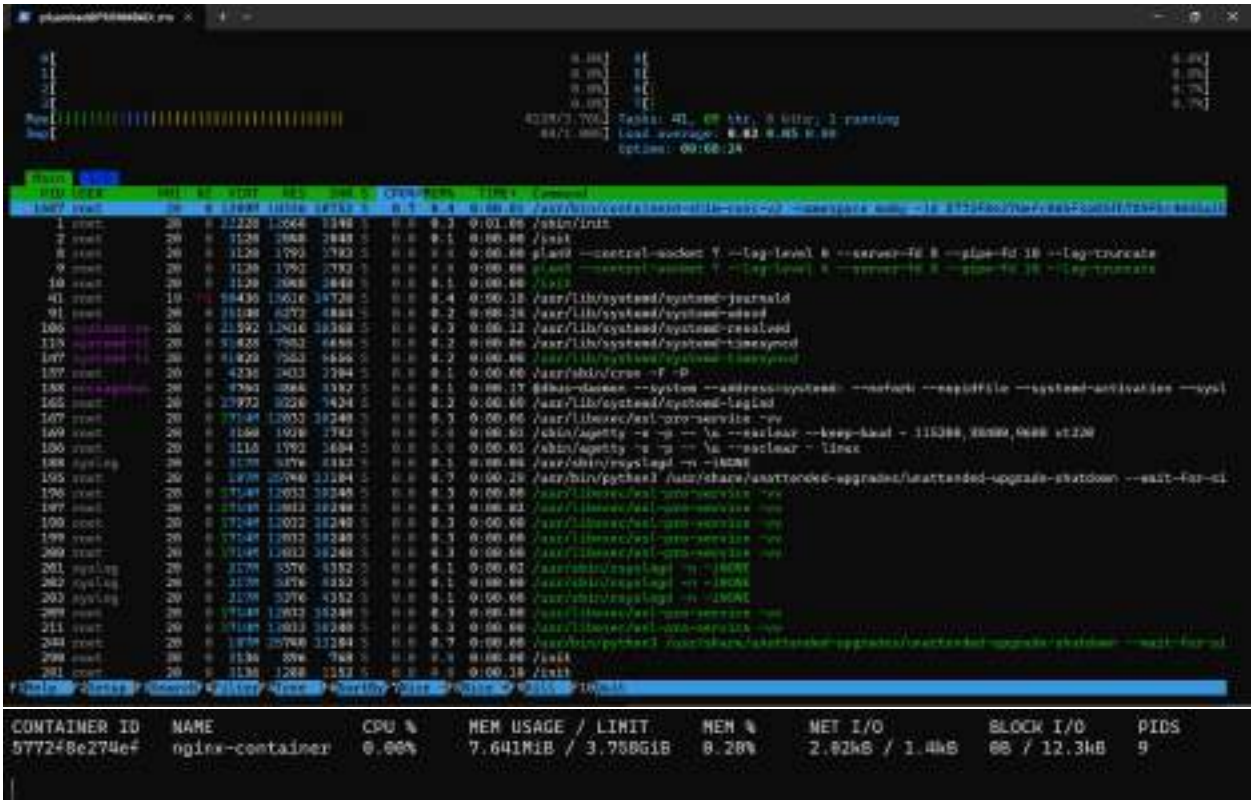
Install and run Nginx inside the Docker container.



1.3.12 Step 12: Compare VM and Container Performance

Observe startup time, memory usage, CPU consumption, and overall performance.





1.4 Result

Ubuntu-based Nginx web server was successfully deployed on both Virtual Machine and Docker Container. The experiment demonstrated that containers are more lightweight and faster, while virtual machines provide stronger isolation with higher resource usage.

1.5 Conclusion

This experiment highlights the key differences between Virtual Machines and Containers in terms of architecture, performance, and resource utilization, helping understand their appropriate use cases in modern computing environments.

2 Experiment 2: Docker Installation, Configuration, and Running Images

2.1 Objective

The objective of this experiment is to: - Verify Docker installation and configuration on macOS - Pull a Docker image from Docker Hub - Run a Docker container with port mapping - Access the running container using localhost - Manage the container lifecycle (stop, remove) - Remove Docker images after use

---

2.2 Commands Executed

2.2.1 1. Docker Installation & Configuration Check

The following commands were used to verify Docker installation and ensure that the Docker Engine was running:

```
docker --version
docker info
```

Figure 3: screenshot 1

2.2.2 2. Pull Docker Image

The nginx image was pulled from Docker Hub using the following command:

```
docker pull nginx
```

To verify the downloaded image:

```
docker images
```

2.2.3 3. Run Docker Container with Port Mapping

The nginx container was started in detached mode with port mapping:

```
docker run -d -p 8080:80 nginx
```

To verify the running container:

```
docker ps
```

Figure 4: screenshot 2

Figure 5: screenshot 3

2.2.4 4. Container Lifecycle Management

The running container was stopped and removed using the following commands:

```
docker stop <container_id>
docker rm <container_id>
```

To verify stopped containers:

```
docker ps -a
```

2.2.5 5. Remove Docker Image

After removing the container, the nginx image was deleted:

```
docker rmi nginx
```

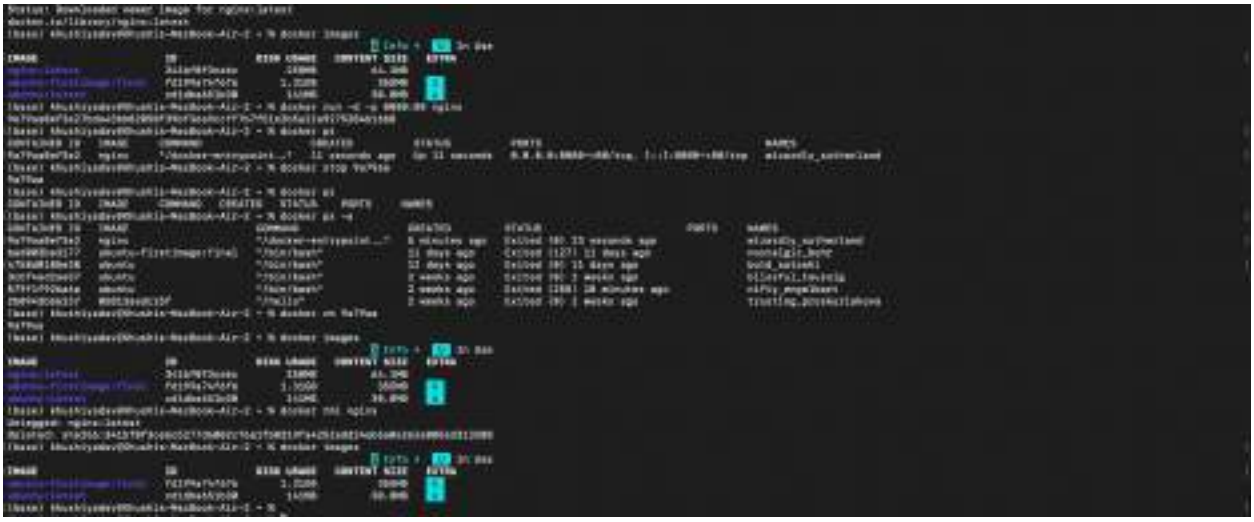


Figure 6: screenshot 4

2.2.6 Result

Docker images were successfully pulled, containers were executed with port mapping, and container lifecycle commands were performed successfully. ### Conclusion This experiment demonstrated the use of Docker for containerization. It showed how Docker images are pulled from Docker Hub, containers are created and accessed using port mapping, and how Docker efficiently manages container lifecycle operations.

3 Experiment 3: Deploying NGINX Using Different Base Images

3.1 Objective

To deploy NGINX using: - Official NGINX Image - Ubuntu-based Custom Image - Alpine-based Custom Image

And compare: - Image size - Configuration behavior - Docker layers - Real-world use cases

---

4 Prerequisites

- Docker installed and running

- Basic Linux command knowledge
- Understanding of Dockerfile and port mapping

---

5 Part 1: Deploy NGINX Using Official Image

5.1 Step 1: Pull Image

```
docker pull nginx:latest
```

5.2 Step 2: Run Container

```
docker run -d --name nginx-official -p 8080:80 nginx
```

Figure 7: screenshot 1

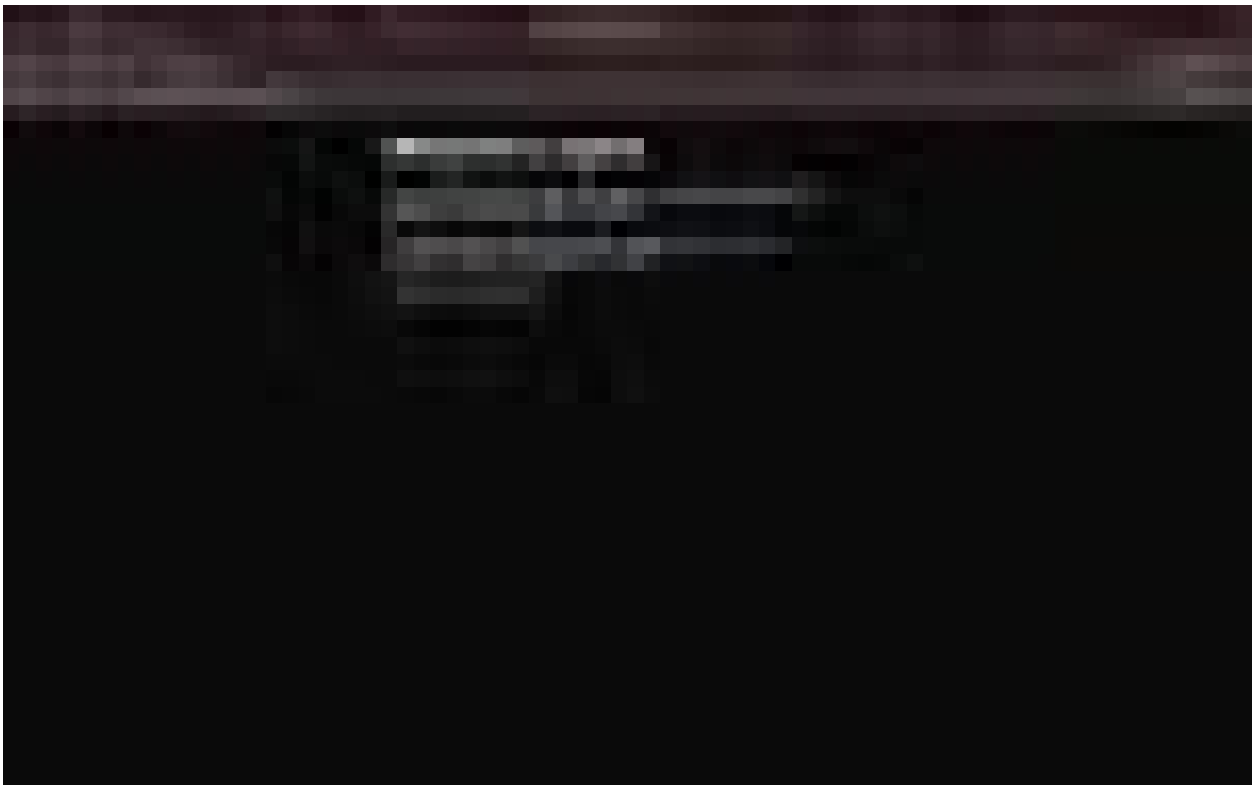
5.3 Step 3: Verify

Open in browser:

```
http://localhost:8080
```

Output:

```
Welcome to nginx!
```

5.4 

6 Part 2: Custom NGINX Using Ubuntu Base Image

6.1 Step 1: Create Dockerfile

```
FROM ubuntu:22.04

RUN apt-get update && \
    apt-get install -y nginx && \
    apt-get clean && \
    rm -rf /var/lib/apt/lists/*

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

6.2 Step 2: Build Image

```
docker build -t nginx-ubuntu .
```

6.3 Step 3: Run Container

```
docker run -d --name nginx-ubuntu -p 8081:80 nginx-ubuntu
```

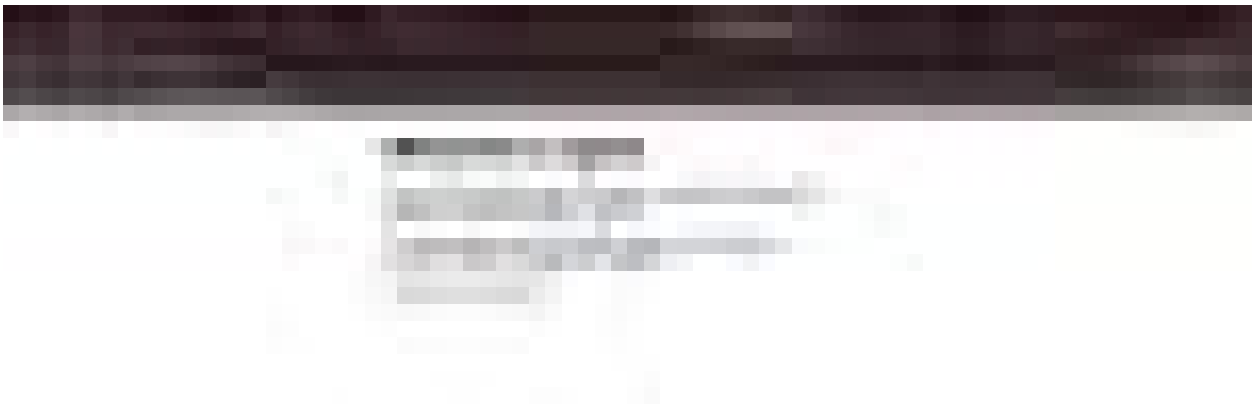
Verify in browser:

```
http://localhost:8081
```

Output:

```
Welcome to nginx!
```

Figure 8: screenshot 3



6.4

7 Part 3: Custom NGINX Using Alpine Base Image

7.1 Step 1: Create Dockerfile

```
FROM alpine:latest

RUN apk add --no-cache nginx

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

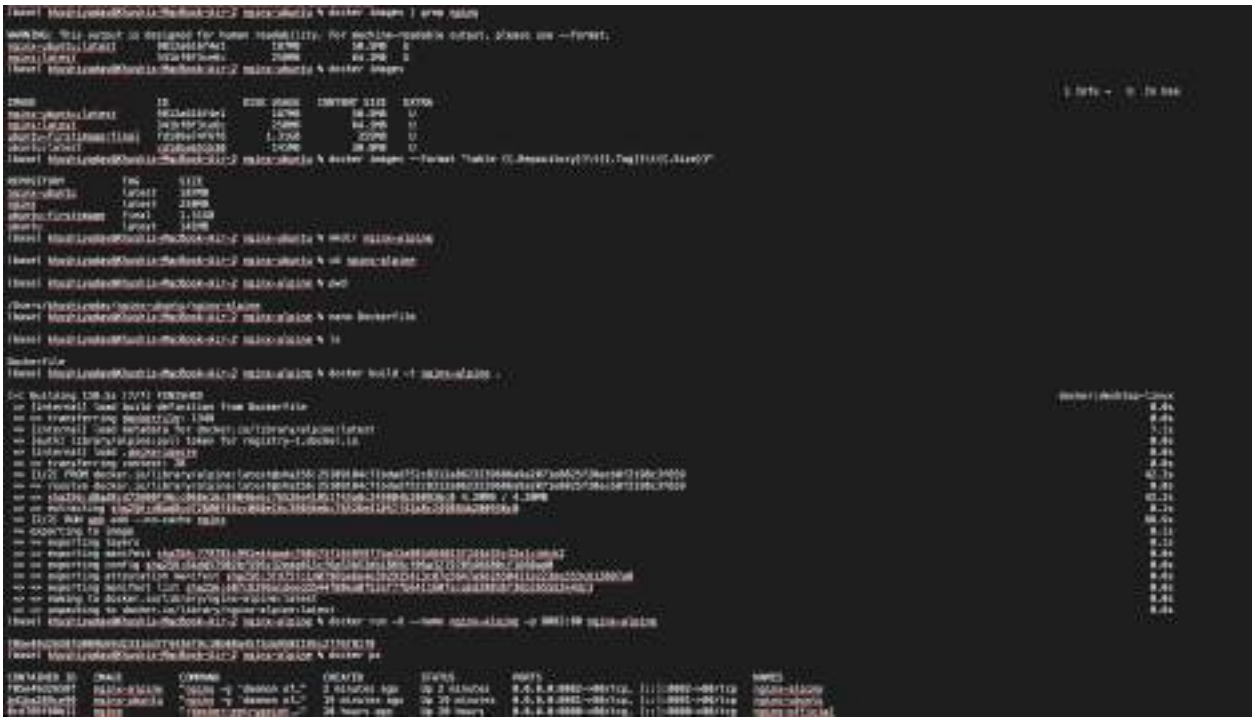
7.2 Step 2: Build Image

```
docker build -t nginx-alpine .
```

7.3 Step 3: Run Container

```
docker run -d --name nginx-alpine -p 8082:80 nginx-alpine
```

7.4



8 Issue Faced in Alpine

After running:

```
http://localhost:8082
```

It showed:

```
404 Not Found
```

8.1

8.2 Debugging Process

8.2.1 Checked Container

```
docker ps
```

Container was running.

8.2.2 Entered Container

```
docker exec -it nginx-alpine sh
```

8.2.3 Checked Default Config

```
cat /etc/nginx/http.d/default.conf
```

Found:

```
server {
    listen 80 default_server;

    location / {
        return 404;
    }
}
```

Alpine default configuration intentionally returns 404.

---

9 Fix Applied

9.0.1 Created Web Directory

```
mkdir -p /var/www/html
echo "<h1>Hello from Alpine NGINX</h1>" > /var/www/html/index.html
```

9.0.2 Modified Configuration

```
server {
    listen 80 default_server;

    root /var/www/html;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }
}
```

9.0.3 Tested Configuration

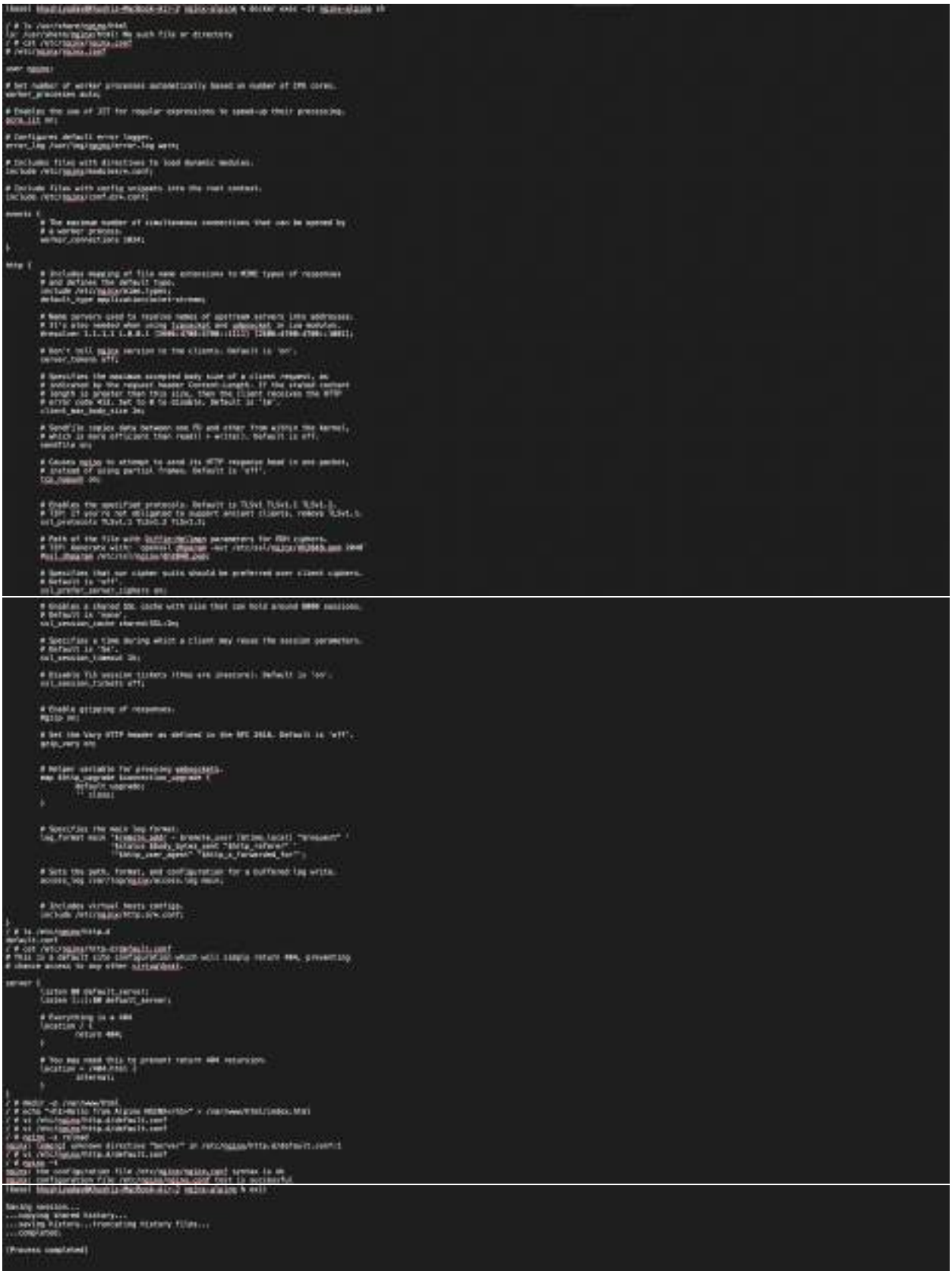
```
nginx -t
```

Output:

```
syntax is ok
test is successful
```

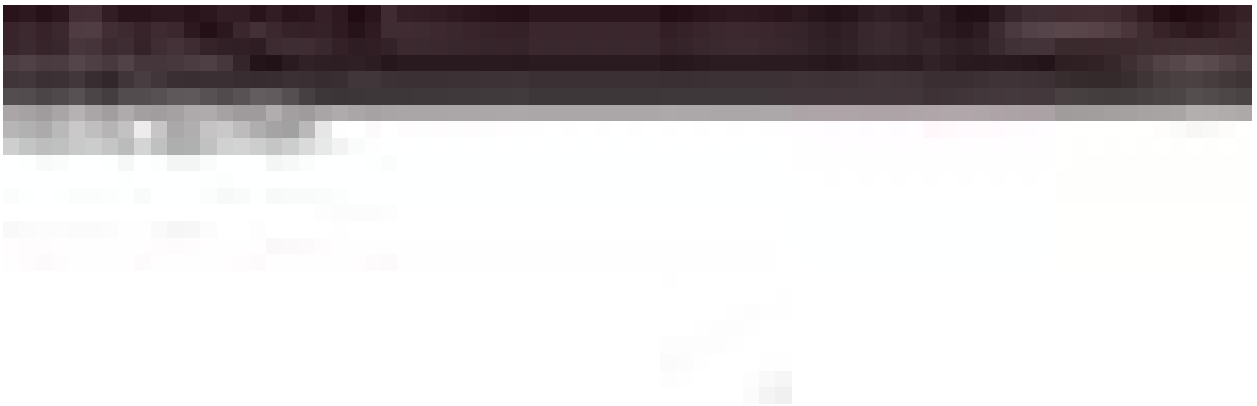
9.0.4 Reloaded NGINX

```
nginx -s reload
```



Now browser showed:

```
Hello from Alpine NGINX
```



9.1

10 Image Size Comparison

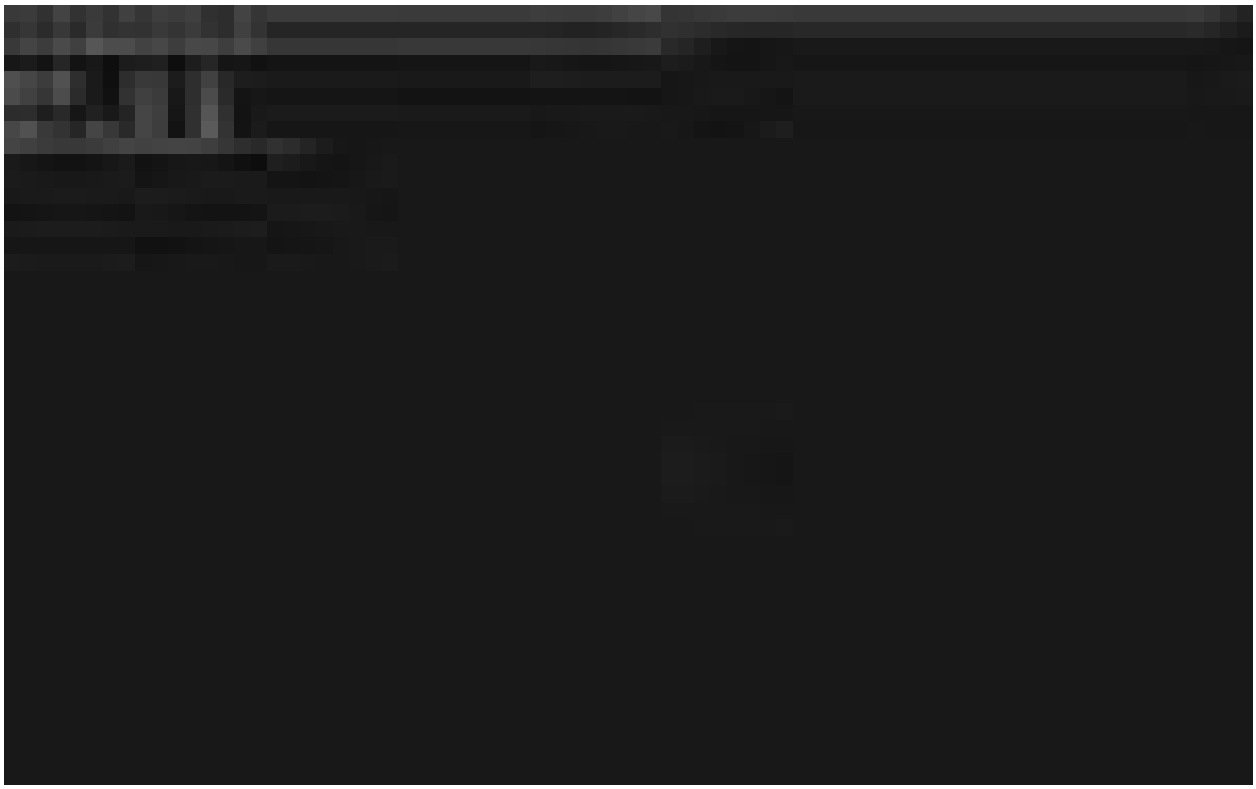
Command used:

```
docker images --format "table {{.Repository}}\t{{.Tag}}\t{{.Size}}"
```

10.1 Results:

Image Type	Size
nginx	258MB
nginx-ubuntu	187MB
nginx-alpine	16.7MB

10.2



11 Observations

- Alpine image is the smallest (16.7MB)
- Ubuntu image is larger due to full OS dependencies
- Official image is optimized but heavier than Alpine
- Alpine default configuration returns 404 intentionally
- Base image selection impacts:
  - Security
  - Performance
  - Image size
  - Startup time

---

12 Conclusion

In this experiment, NGINX was deployed using three different base images. The experiment demonstrated that base image selection significantly impacts container size and configuration behavior. Alpine-based images are lightweight and ideal for microservices and cloud environments, while Ubuntu-based images provide more debugging tools but increase image size. Official images are production-ready but may include additional modules.

This experiment highlights the importance of optimizing container images for performance and security in real-world deployments.

---

13 Cleanup Commands

```
docker stop nginx-official nginx-ubuntu nginx-alpine
docker rm nginx-official nginx-ubuntu nginx-alpine
docker rmi nginx nginx-ubuntu nginx-alpine
docker system prune -a
```

---

14 Key Learnings

- Docker image layering
- Base image impact on size
- NGINX configuration basics
- Debugging containerized applications
- Production vs minimal container strategies

15 Experiment 4 – Docker Essentials

15.1 Aim

To containerize a Flask application using Docker, manage containers, and push the image to Docker Hub.

---

16 Part 1 – Containerizing a Flask Application

16.1 Step 1: Create Project Directory

```
mkdir my-flask-app
cd my-flask-app
```

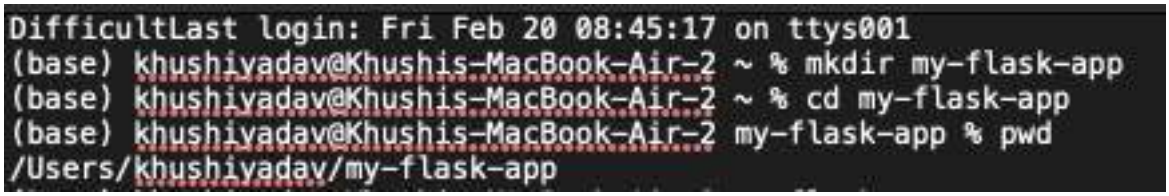


Figure 9: screenshot 1

16.2 Step 2: Create Flask Application (app.py)

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello():
    return "Hello from Docker!"

@app.route('/health')
def health():
```

```
    return "OK"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

16.3 Step 3: Create requirements.txt

```
Flask==2.3.3
```

16.4 Step 4: Create Dockerfile

```
FROM python:3.9-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 5000

CMD ["python", "app.py"]
```

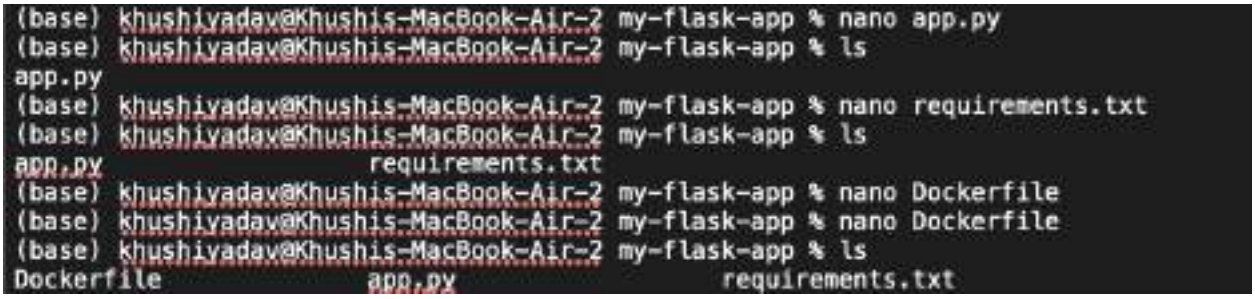


Figure 10: screenshot 1

16.5 Step 5: Build Docker Image

```
docker build -t my-flask-app .
```

16.6 Step 6: Run Container

```
docker run -d -p 5001:5000 --name flask-container my-flask-app
```

Access application:

```
http://localhost:5001
```

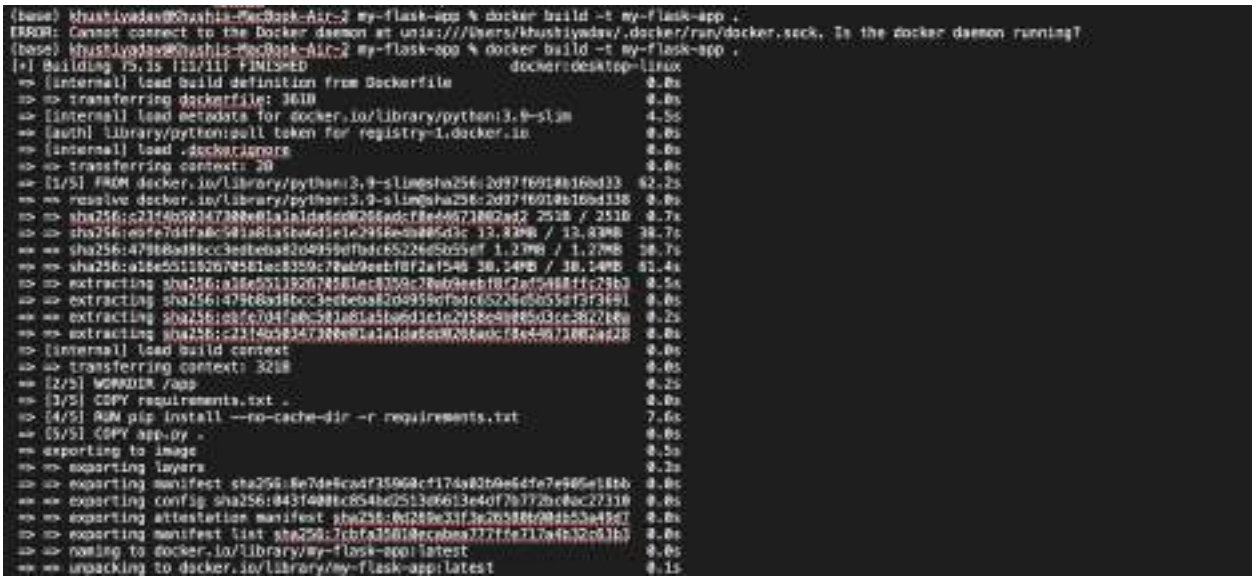


Figure 11: screenshot 1

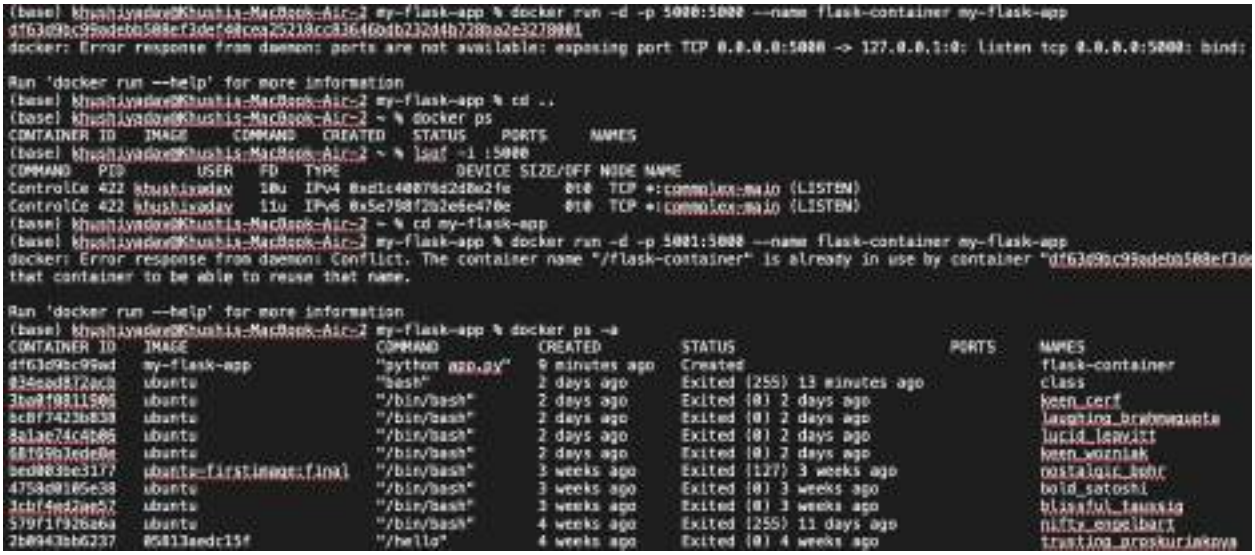


Figure 12: screenshot 1

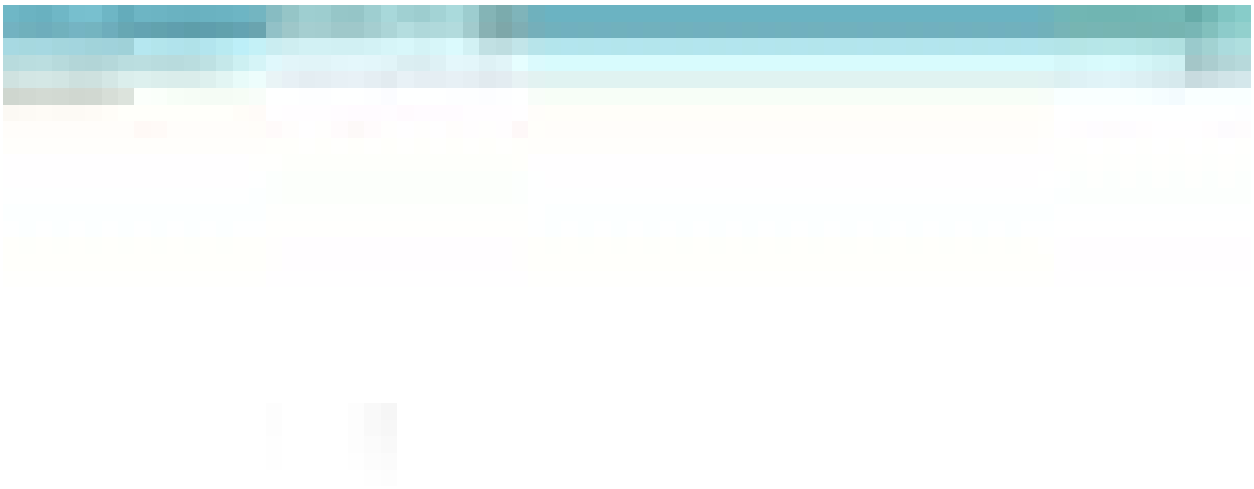


Figure 13: screenshot 1

16.7 Step 7: Test Health Endpoint

`http://localhost:5001/health`

16.8 Step 8: View Logs

`docker logs flask-container`

16.9

16.10 Step 9: Stop, Start and Remove Container

```
docker stop flask-container
docker start flask-container
docker rm -f flask-container
```

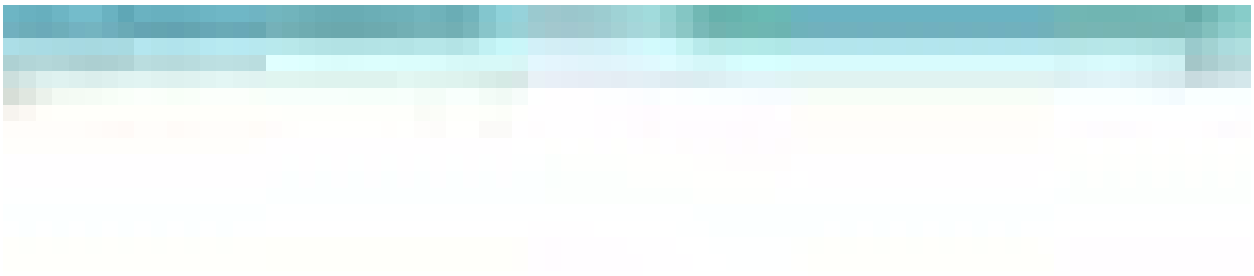
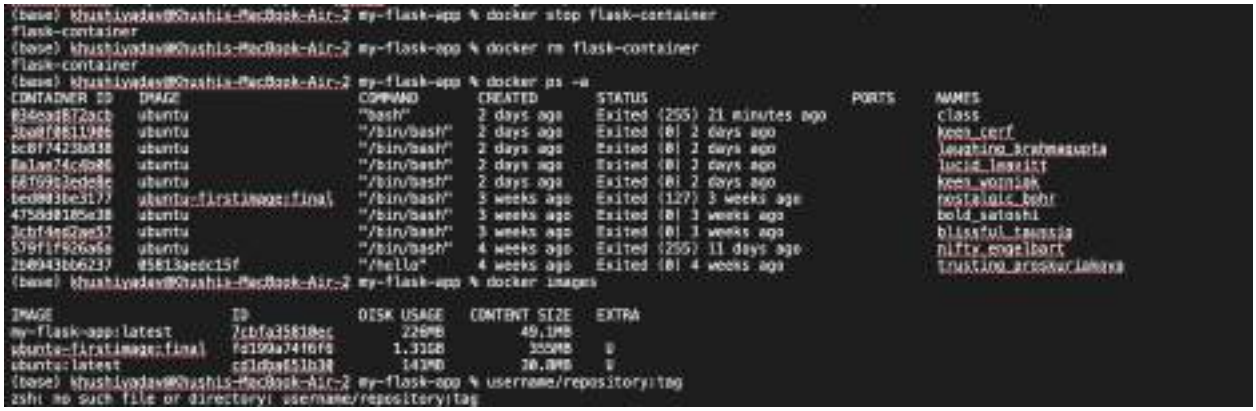


Figure 14: screenshot 1

16.11



17 Part 2 – Docker Hub Operations

17.1 Step 10: Tag Image

```
docker tag my-flask-app ky270405/my-flask-app:v1
```

17.2 Step 11: Login to Docker Hub

```
docker login
```

17.3 Step 12: Push Image

```
docker push ky270405/my-flask-app:v1
```

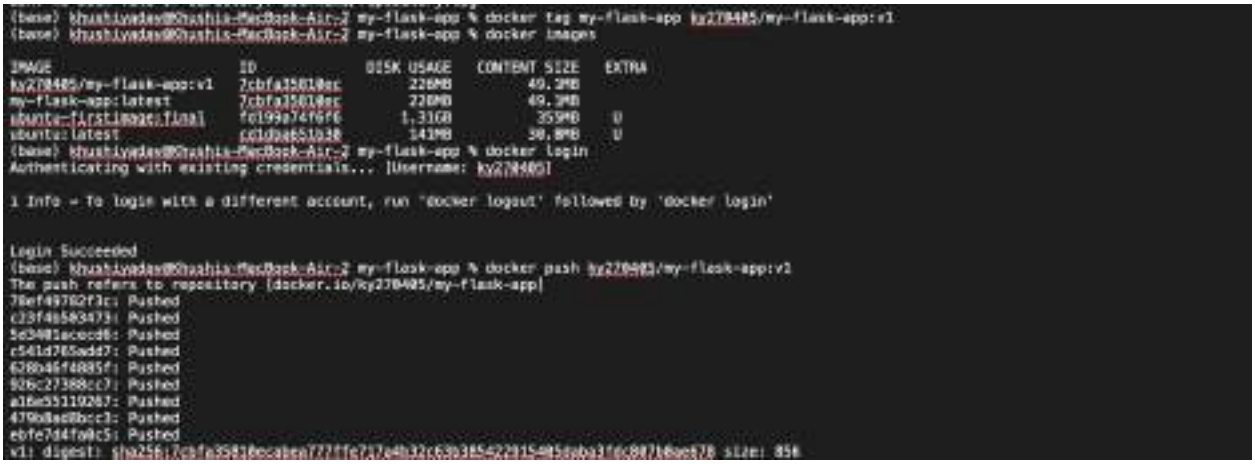


Figure 15: screenshot 1

17.4 Step 13: Remove Local Image

```
docker rmi ky270405/my-flask-app:v1
docker rmi my-flask-app
```

17.5 Step 14: Pull Image from Docker Hub

```
docker pull ky270405/my-flask-app:v1
```

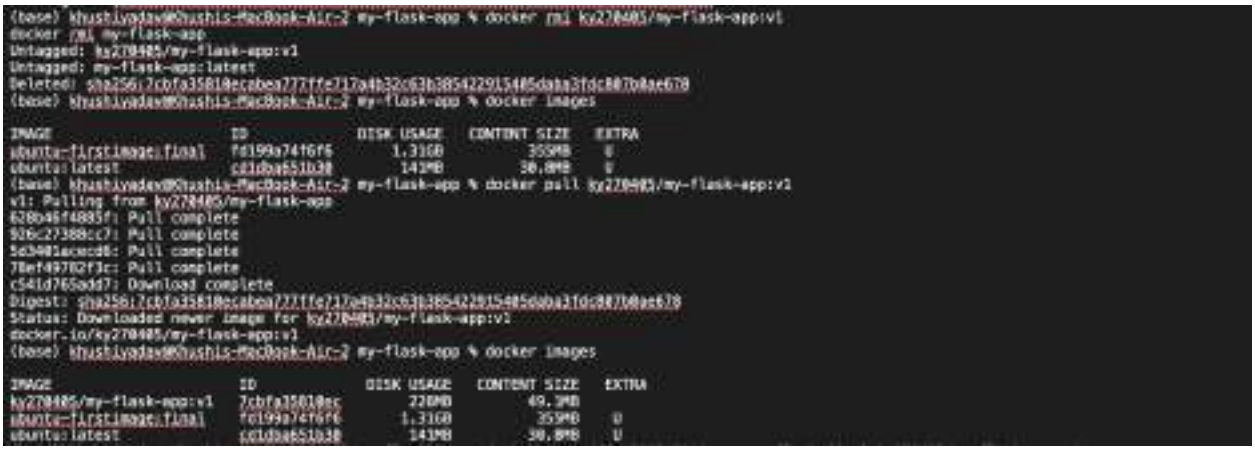


Figure 16: screenshot 1

17.6 Step 15: Run Pulled Image

```
docker run -d -p 5002:5000 --name flask-final ky270405/my-flask-app:v1
```

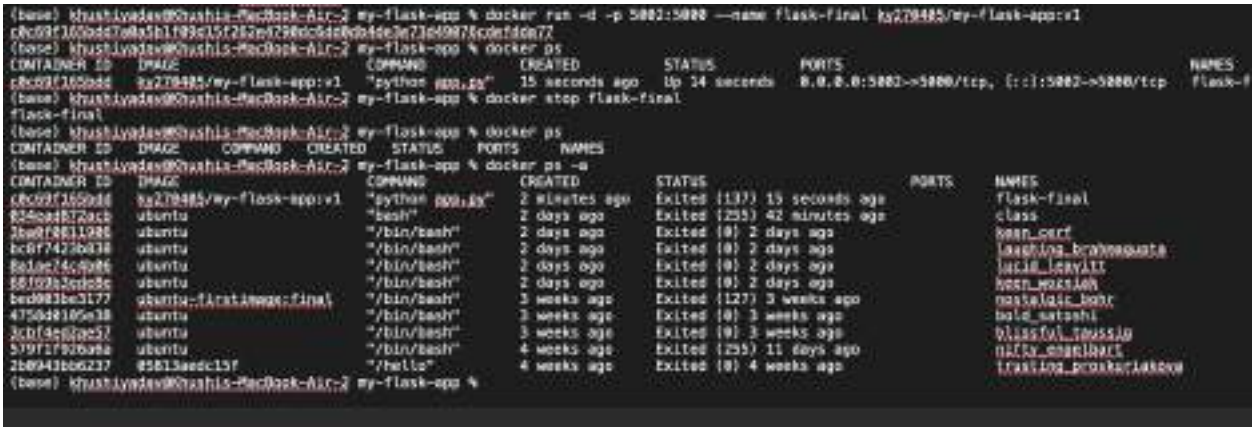


Figure 17: screenshot 1

Access:

`http://localhost:5002`

---

18 Errors and Issues Faced

18.1 -> Docker Daemon Not RunningAt 9 AM

Error:

```
Cannot connect to the Docker daemon...
```

Reason: Docker Desktop was not started.

Solution: Started Docker Desktop and verified using:

```
docker version
```

18.2 -> Port 5000 Already in Use

Error:

```
bind: address already in use
```

Reason: Port 5000 was already used by a system process.

Solution: Used a different port:

```
docker run -d -p 5001:5000 ...
```

18.3 -> Container Name Already in Use

Error:

```
container name is already in use
```

Reason: A container with the same name already existed.

Solution:

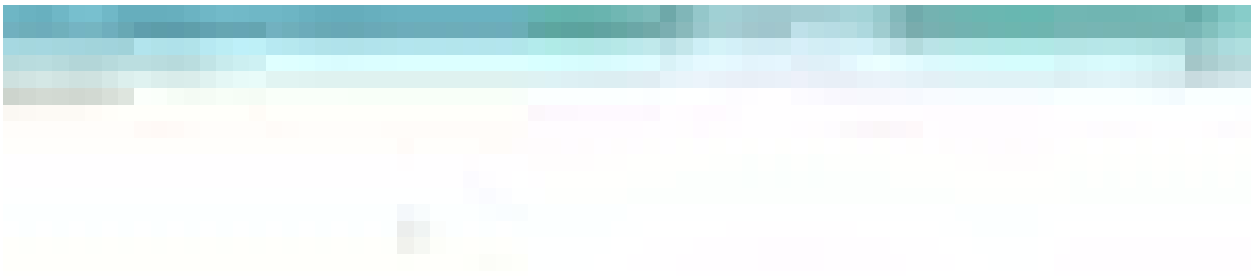


Figure 18: screenshot 6

```
docker rm -f flask-container
```

---

19 Results

- Successfully containerized a Flask application.
- Built Docker image.
- Ran container with port mapping.
- Managed container lifecycle.
- Tagged and pushed image to Docker Hub.
- Pulled and ran image successfully.

---

20 Conclusion

The experiment successfully demonstrated Docker containerization, image management, port mapping, container lifecycle management, and integration with Docker Hub.

21 Experiment 5:

22 Docker — Volumes, Environment Variables, Monitoring & Networks

Platform: MacBook Air M1 | **Date:** March 14–15, 2026
Purpose: Learn essential Docker features for building, configuring, and managing production-ready containerized applications.

22.1 Table of Contents

- Overview
- Part 1: Docker Volumes
- Part 2: Environment Variables
- Part 3: Docker Monitoring
- Part 4: Docker Networks
- Part 5: Real-World Example
- Cleanup
- Errors Encountered & Fixes
- Key Takeaways

22.2 Overview

Part	Topic	What Was Done
Part 1	Docker Volumes	Anonymous, Named & Bind Mounts with persistence demo
Part 2	Environment Variables	-e flag, –env-file, Flask app configuration
Part 3	Docker Monitoring	stats, logs, top, inspect, system df
Part 4	Docker Networks	Bridge, Host, None & Custom networks
Part 5	Real-World App	Flask + PostgreSQL + Redis multi-container setup

---

22.3 Part 1: Docker Volumes

Docker containers are ephemeral — all data written inside a container is lost when it is removed. Docker Volumes solve this by persisting data outside the container lifecycle.

22.3.1 Lab 1: Understanding Data Persistence

22.3.1.1 Step 1 — Verify Docker Installation

```
docker --version
docker ps
```

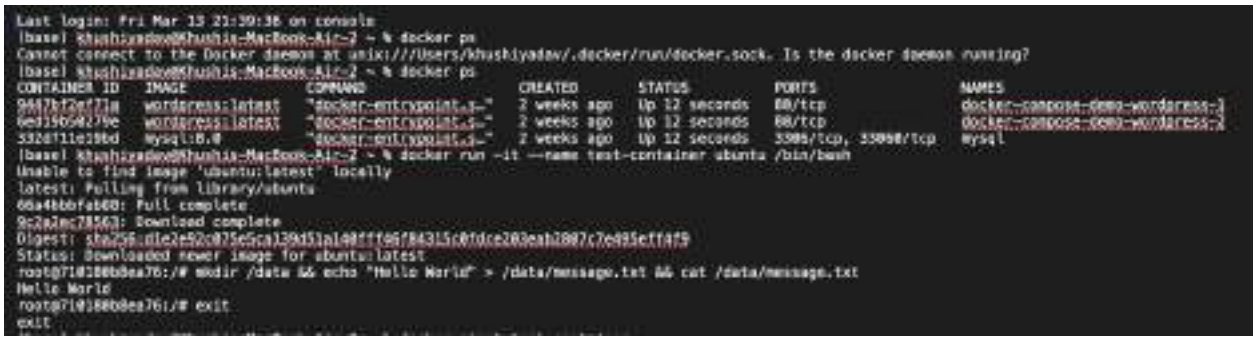
Docker Desktop must be running before executing any docker commands.

22.3.1.2 Step 2 — Create Container and Write Data

```
docker run -it --name test-container ubuntu /bin/bash

# Inside container:
mkdir /data && echo "Hello World" > /data/message.txt
cat /data/message.txt
# Output: Hello World
exit
```

22.4



22.4.0.1 Step 3 — Prove Data is Ephemeral

```
# Restart and check - data still exists (container just stopped)
docker start test-container
docker exec test-container cat /data/message.txt

# Now remove and recreate
docker stop test-container
docker rm test-container

docker run -it --name test-container ubuntu /bin/bash
cat /data/message.txt
# ERROR: No such file or directory ← data is gone!
exit
```

This proves container data does NOT survive removal.

22.5

22.5.0.1 Error: Cannot remove running container

```
Error response from daemon: cannot remove container "test-container":
container is running: stop the container before removing or force remove
```

22.5.0.2 Fix:

```
docker stop test-container
docker rm test-container
```

22.5.1 Lab 2: Volume Types

22.5.1.1 Step 4 — Anonymous Volumes

```
docker run -d -v /app/data --name web1 nginx
docker volume ls
docker inspect web1 | grep -A 5 Mounts
```

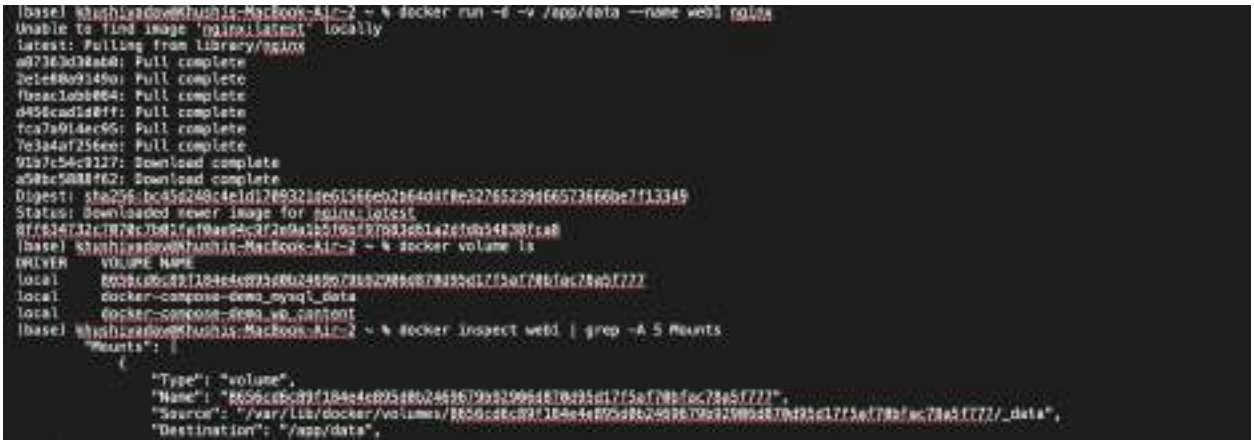
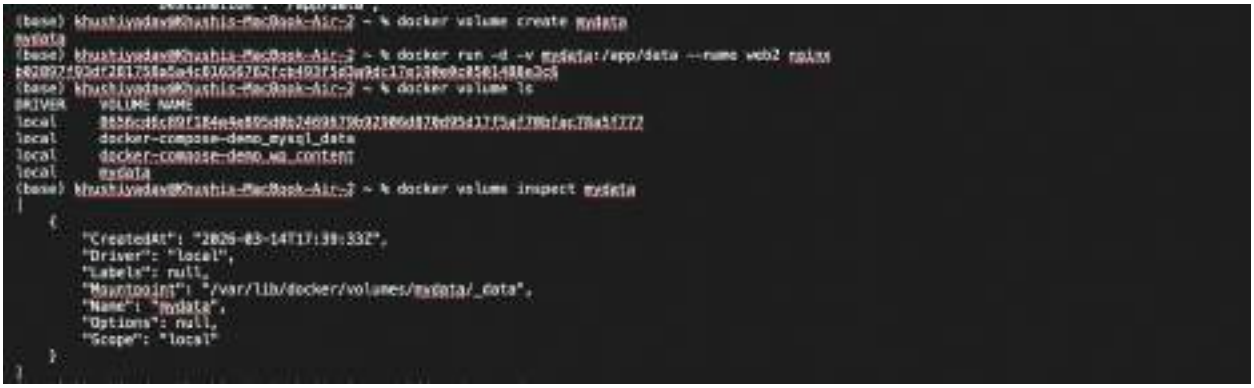


Figure 19: screenshot 4

Output: A random hash name is auto-generated for the anonymous volume, mounted at `/app/data`.

22.5.1.2 Step 5 — Named Volumes

```
docker volume create mydata
docker run -d -v mydata:/app/data --name web2 nginx
docker volume ls
docker volume inspect mydata
```



Output:

```
{
  "Name": "mydata",
  "Driver": "local",
  "Mountpoint": "/var/lib/docker/volumes/mydata/_data",
```

```
  "Scope": "local"
}
```

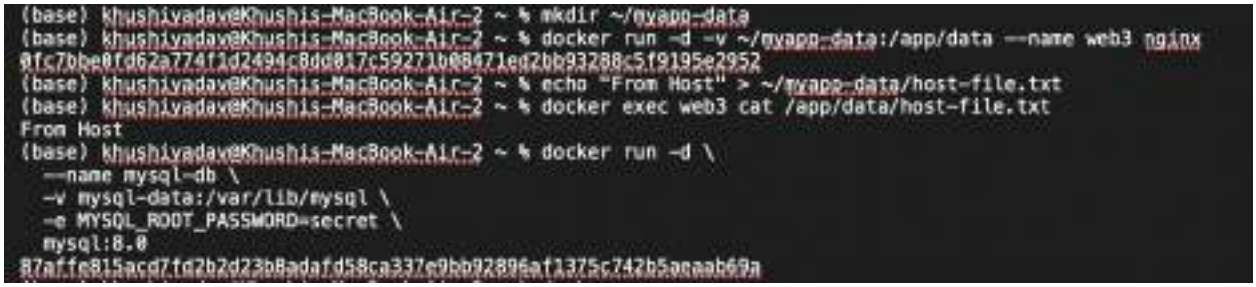
22.5.1.3 Step 6 — Bind Mounts (Host Directory)

```
mkdir ~/myapp-data
docker run -d -v ~/myapp-data:/app/data --name web3 nginx

# Add file on host
echo "From Host" > ~/myapp-data/host-file.txt

# Verify inside container
docker exec web3 cat /app/data/host-file.txt
# Output: From Host
```

Bind mounts enable live sync between Mac host and container — any file added on the host is immediately visible inside the container.



22.6

22.6.1 Lab 3: Practical Volume Examples

22.6.1.1 Step 7 — MySQL with Persistent Volume

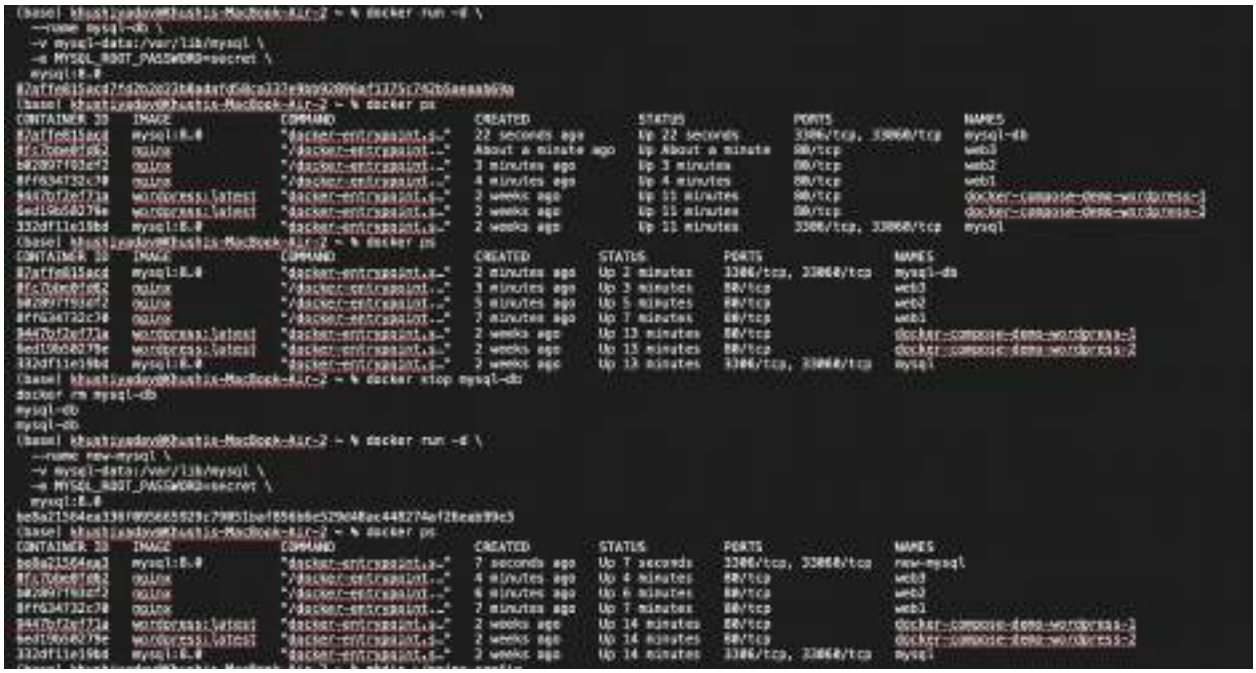
```
docker run -d \
  --name mysql-db \
  -v mysql-data:/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=secret \
  mysql:8.0

# Remove the container
docker stop mysql-db && docker rm mysql-db

# Recreate with same volume - data is preserved!
docker run -d \
  --name new-mysql \
  -v mysql-data:/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=secret \
  mysql:8.0

docker ps
```

22.7



22.7.0.1 Step 8 — Nginx with Config Bind Mount

```
mkdir ~/nginx-config

echo 'server {
  listen 80;
  server_name localhost;
  location / {
    return 200 "Hello from mounted config!";
  }
}' > ~/nginx-config/nginx.conf

docker run -d \
  --name nginx-custom \
  -p 8080:80 \
  -v ~/nginx-config/nginx.conf:/etc/nginx/conf.d/default.conf \
  nginx

curl http://localhost:8080
# Output: Hello from mounted config!
```

```
(base) khushiyadav@Khushis-MacBook-Air-2 ~ % mkdir ~/nginx-config
(base) khushiyadav@Khushis-MacBook-Air-2 ~ % echo 'server {
  listen 80;
  server_name localhost;
  location / {
    return 200 "Hello from mounted config!";
  }
}' > ~/nginx-config/nginx.conf
(base) khushiyadav@Khushis-MacBook-Air-2 ~ % docker run -d \
  --name nginx-custom \
  -p 8080:80 \
  -v ~/nginx-config/nginx.conf:/etc/nginx/conf.d/default.conf \
  nginx
1a8a47eb558c657686ad247e0e1652a92359d4dba7b5044dcd12f4f1d4b6a66e
(base) khushiyadav@Khushis-MacBook-Air-2 ~ % curl http://localhost:8080
Hello from mounted config!%
```

22.8

22.8.1 Lab 4: Volume Management Commands

22.8.1.1 Step 9 — Volume Management

```
docker volume ls                    # list all volumes
docker volume create app-volume     # create a volume
docker volume inspect app-volume    # inspect details
docker volume prune                 # remove unused anonymous volumes
docker volume rm volume-name        # remove specific volume
```

```
khushiyadav@Khushis-MacBook-Air-2 ~ % docker volume ls
DRIVER    VOLUME NAME
local     8656cd6c89f184e4e895d0b2469679b92906d870d95d17f5af70bfac78a5f777
local     docker-compose-demo_mysql_data
local     docker-compose-demo_wp_content
local     mydata
local     mysql-data
(base) khushiyadav@Khushis-MacBook-Air-2 ~ % docker volume create app-volume
app-volume
(base) khushiyadav@Khushis-MacBook-Air-2 ~ % docker volume inspect app-volume
[
    {
        "CreatedAt": "2026-03-14T17:48:24Z",
        "Driver": "local",
        "Labels": null,
        "Mountpoint": "/var/lib/docker/volumes/app-volume/_data",
        "Name": "app-volume",
        "Options": null,
        "Scope": "local"
    }
]
(base) khushiyadav@Khushis-MacBook-Air-2 ~ % docker volume prune
WARNING! This will remove anonymous local volumes not used by at least one container.
Are you sure you want to continue? [y/N] y
Total reclaimed space: 0B
```

22.9

22.10 Part 2: Environment Variables

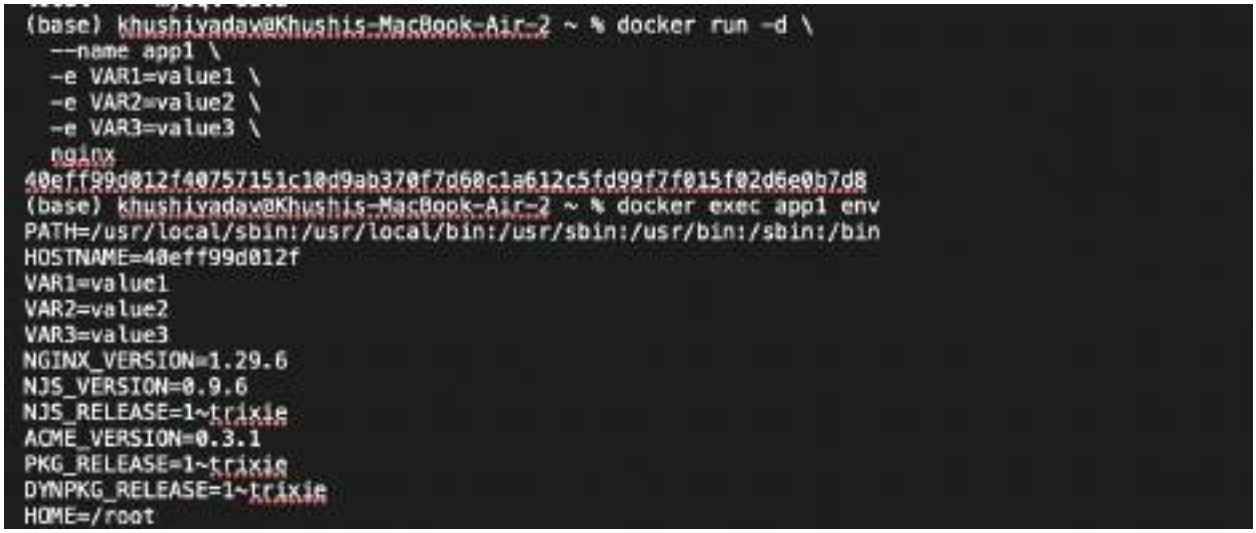
Environment variables allow dynamic configuration of containers without hardcoding values into the image.

22.10.1 Lab 1: Setting Environment Variables

22.10.1.1 Step 10 — Using the -e Flag

```
docker run -d \
  --name app1 \
  -e VAR1=value1 \
  -e VAR2=value2 \
  -e VAR3=value3 \
  nginx

docker exec app1 env
# Output includes: VAR1=value1, VAR2=value2, VAR3=value3
```



22.11

22.11.0.1 Step 11 — Using –env-file

```
echo "DATABASE_HOST=localhost" > ~/.env
echo "DATABASE_PORT=5432" >> ~/.env
echo "API_KEY=secret123" >> ~/.env

docker run -d \
  --name app2 \
  --env-file ~/.env \
  nginx

docker exec app2 env
# Output includes: DATABASE_HOST, DATABASE_PORT, API_KEY
```

```
(base) khushiyadav@Khushis-MacBook-Air-2 ~ % cd ~ && echo "DATABASE_HOST=localhost" > .env
echo "DATABASE_PORT=5432" >> .env
echo "API_KEY=secret123" >> .env
(base) khushiyadav@Khushis-MacBook-Air-2 ~ % cat ~/.env
DATABASE_HOST=localhost
DATABASE_PORT=5432
API_KEY=secret123
(base) khushiyadav@Khushis-MacBook-Air-2 ~ % docker run -d \
  --name app2 \
  --env-file ~/.env \
  nginx
402c81084d27024f99a8c8fa24bf69c0783ab907726625c7f6aeBedaa3f87e60
(base) khushiyadav@Khushis-MacBook-Air-2 ~ % docker exec app2 env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=402c81084d27
DATABASE_HOST=localhost
DATABASE_PORT=5432
API_KEY=secret123
NGINX_VERSION=1.29.6
NJS_VERSION=0.9.6
NJS_RELEASE=1~trixie
ACME_VERSION=0.3.1
PKG_RELEASE=1~trixie
DYNPKG_RELEASE=1~trixie
HOME=/root
```

22.12

22.12.1 Lab 2: Flask App with Environment Variables

22.12.1.1 Step 12 — Create Flask Application app.py:

```python
import os
from flask import Flask
app = Flask(__name__)

db_host = os.environ.get('DATABASE_HOST', 'localhost')
debug_mode = os.environ.get('DEBUG', 'false').lower() == 'true'
api_key = os.environ.get('API_KEY')

@app.route('/config')
def config():
    return {
        'db_host': db_host,
        'debug': debug_mode,
        'has_api_key': bool(api_key)
    }

if __name__ == '__main__':
    port = int(os.environ.get('PORT', 5000))
    app.run(host='0.0.0.0', port=port, debug=debug_mode)
```

requirements.txt:

```
flask
```

Dockerfile:

```
FROM python:3.9-slim
ENV PYTHONUNBUFFERED=1
ENV PYTHONDONTWRITEBYTECODE=1
WORKDIR /app
COPY requirements.txt .
```

```
(base) khushiyadav@Khushis-MacBook-Air-2 ~ % mkdir ~/flask-app && cd ~/flask-app
(base) khushiyadav@Khushis-MacBook-Air-2 flask-app % cat > app.py << 'EOF'
import os
from flask import Flask
app = Flask(__name__)

db_host = os.environ.get('DATABASE_HOST', 'localhost')
debug_mode = os.environ.get('DEBUG', 'false').lower() == 'true'
api_key = os.environ.get('API_KEY')

@app.route('/config')
def config():
    return {
        'db_host': db_host,
        'debug': debug_mode,
        'has_api_key': bool(api_key)
    }

if __name__ == '__main__':
    port = int(os.environ.get('PORT', 5000))
    app.run(host='0.0.0.0', port=port, debug=debug_mode)
EOF
(base) khushiyadav@Khushis-MacBook-Air-2 flask-app % cat ~/flask-app/app.py
import os
from flask import Flask
app = Flask(__name__)

db_host = os.environ.get('DATABASE_HOST', 'localhost')
debug_mode = os.environ.get('DEBUG', 'false').lower() == 'true'
api_key = os.environ.get('API_KEY')

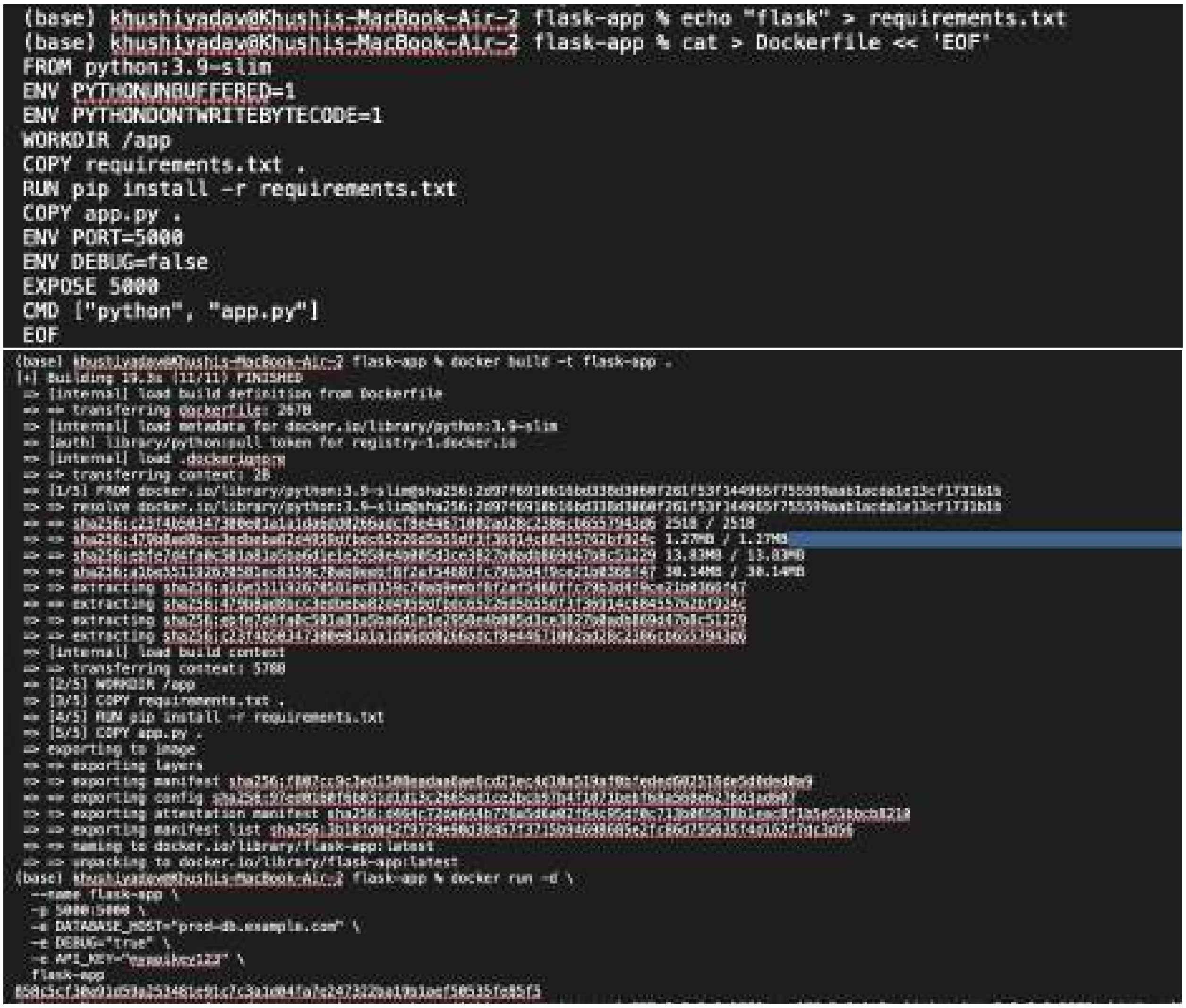
@app.route('/config')
def config():
    return {
        'db_host': db_host,
        'debug': debug_mode,
        'has_api_key': bool(api_key)
    }

if __name__ == '__main__':
    port = int(os.environ.get('PORT', 5000))
    app.run(host='0.0.0.0', port=port, debug=debug_mode)
```

Figure 20: screenshot 2

```
RUN pip install -r requirements.txt
COPY app.py .
ENV PORT=5000
ENV DEBUG=false
EXPOSE 5000
CMD ["python", "-u", "app.py"]
```

```
docker build -t flask-app .
```



—

22.12.1.2 Error: Container exited immediately in detached mode

```
docker logs flask-app → (empty output)
docker ps | grep flask-app → (no output, container not running)
```

Cause: Python output buffering — Flask was not emitting logs, so the container appeared to crash silently.

22.12.1.3 Fix: Use -u flag for unbuffered output

Changed `CMD` in Dockerfile:

```
CMD ["python", "-u", "app.py"]
```

Then rebuilt the image:

```
docker build -t flask-app .
```

22.12.1.4 Error: Port 5000 already in use

```
Error response from daemon: ports are not available: exposing port TCP 0.0.0.0:5000
→ 127.0.0.1:0: listen tcp 0.0.0.0:5000: bind: address already in use
```

Cause: macOS uses port 5000 for AirPlay Receiver by default.

22.12.1.5 Fix: Use port 5001 instead

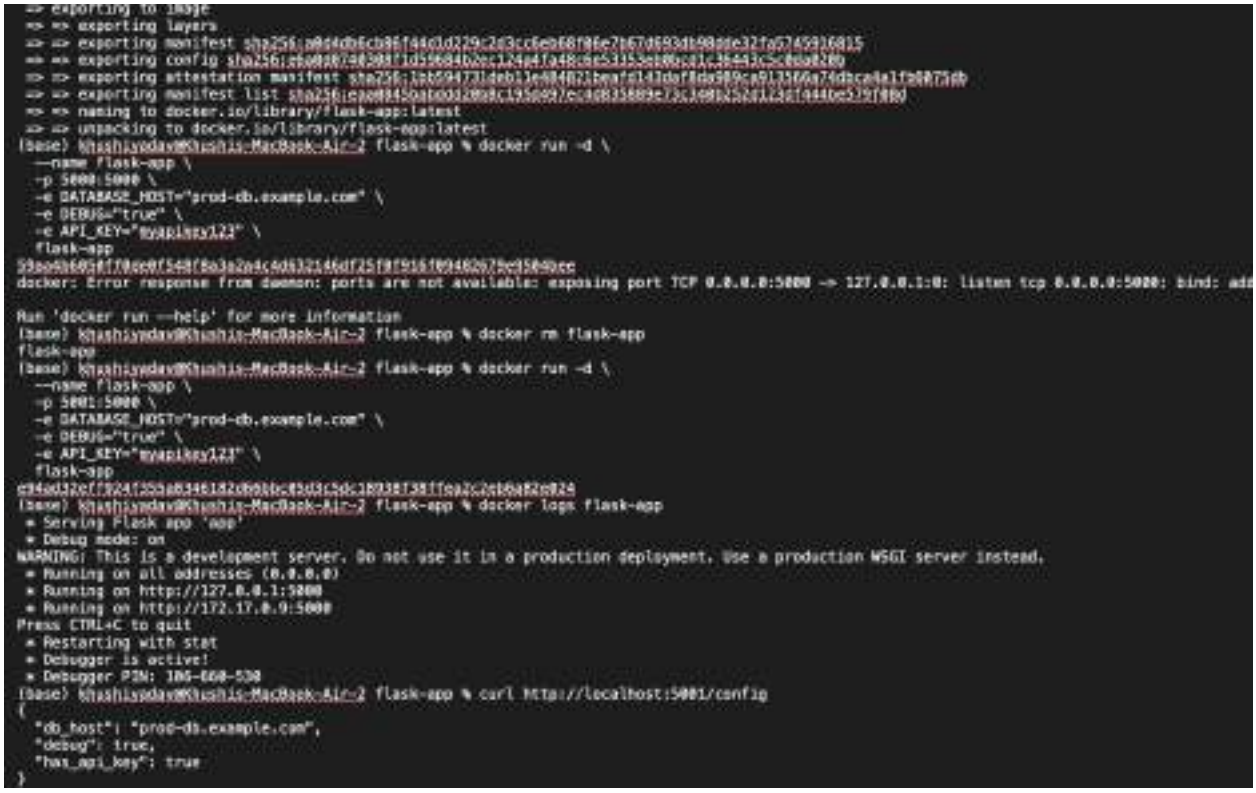
```
docker run -d \
  --name flask-app \
  -p 5001:5000 \
  -e DATABASE_HOST="prod-db.example.com" \
  -e DEBUG="true" \
  -e API_KEY="myapikey123" \
  flask-app
```

22.12.1.6 Step 13 — Test Flask App

```
curl http://localhost:5001/config
```

Response:

```
{
  "db_host": "prod-db.example.com",
  "debug": true,
  "has_api_key": true
}
```



22.13

22.14 Part 3: Docker Monitoring

Docker provides several commands to monitor container health, resource usage, processes, and logs.

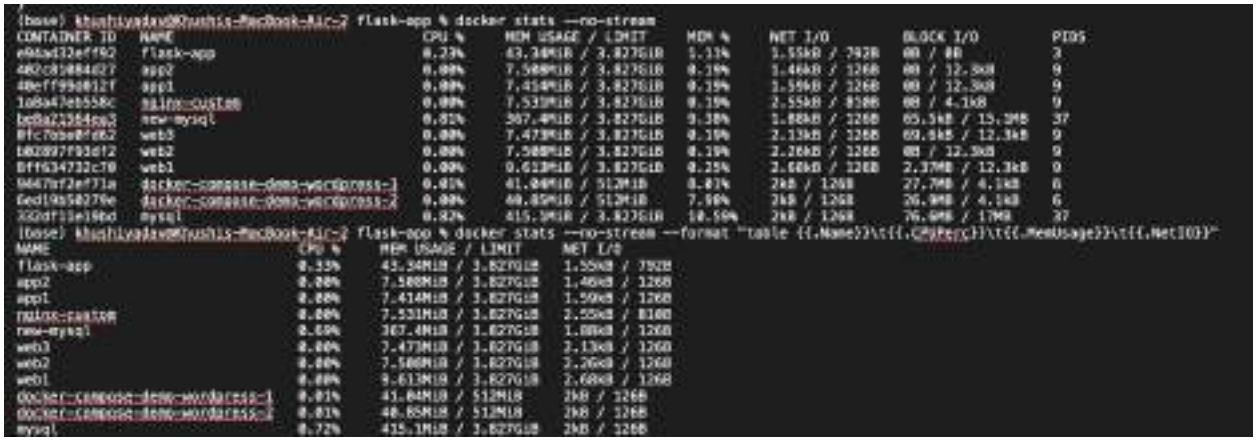
22.14.1 Lab 1: Resource Metrics

22.14.1.1 Step 14 — docker stats

```
# Snapshot of all containers
docker stats --no-stream

# Formatted output
docker stats --no-stream --format \
  "table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}\t{{.NetIO}}"
```

22.15



22.15.0.1 Step 14b — docker top

```
docker top flask-app
```

Output:

```
UID    PID   PPID  CMD
root   2448  2425  python -u app.py
root   2469  2448  /usr/local/bin/python /app/app.py
```

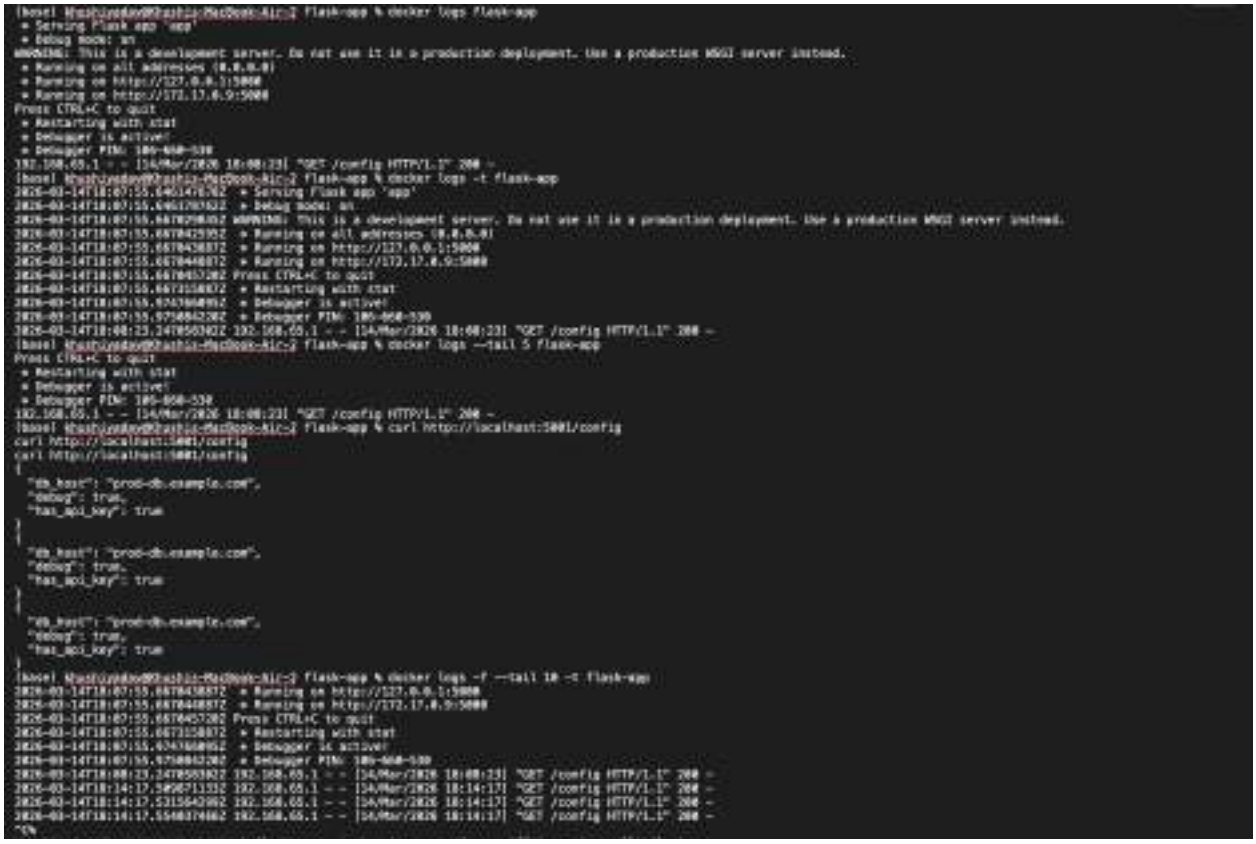
22.16

22.16.1 Lab 2: Log Monitoring

22.16.1.1 Step 15 — docker logs

```
docker logs flask-app                    # view logs
docker logs -t flask-app                 # with timestamps
docker logs --tail 5 flask-app           # last 5 lines
docker logs -f --tail 10 -t flask-app    # follow live logs
```

Live log output showed: - Flask startup on port 5000 - 3 GET /config requests with status 200



22.17

22.17.1 Lab 3: Container Inspection

22.17.1.1 Step 16 — docker inspect

```
docker inspect --format='{{.State.Status}}' flask-app
docker inspect --format='{{.NetworkSettings.IPAddress}}' flask-app
docker inspect --format='{{.Config.Env}}' flask-app
docker inspect --format='{{.HostConfig.Memory}}' flask-app
docker system df
```

docker system df output:

```
TYPE            TOTAL   SIZE      RECLAIMABLE
Images          5       2.638GB   2.638GB (100%)
Containers      12      610.3kB   12.29kB (2%)
Local Volumes   6       496.8MB   0B (0%)
Build Cache     11      217.5MB   24.58kB
```

22.18

22.19 Part 4: Docker Networks

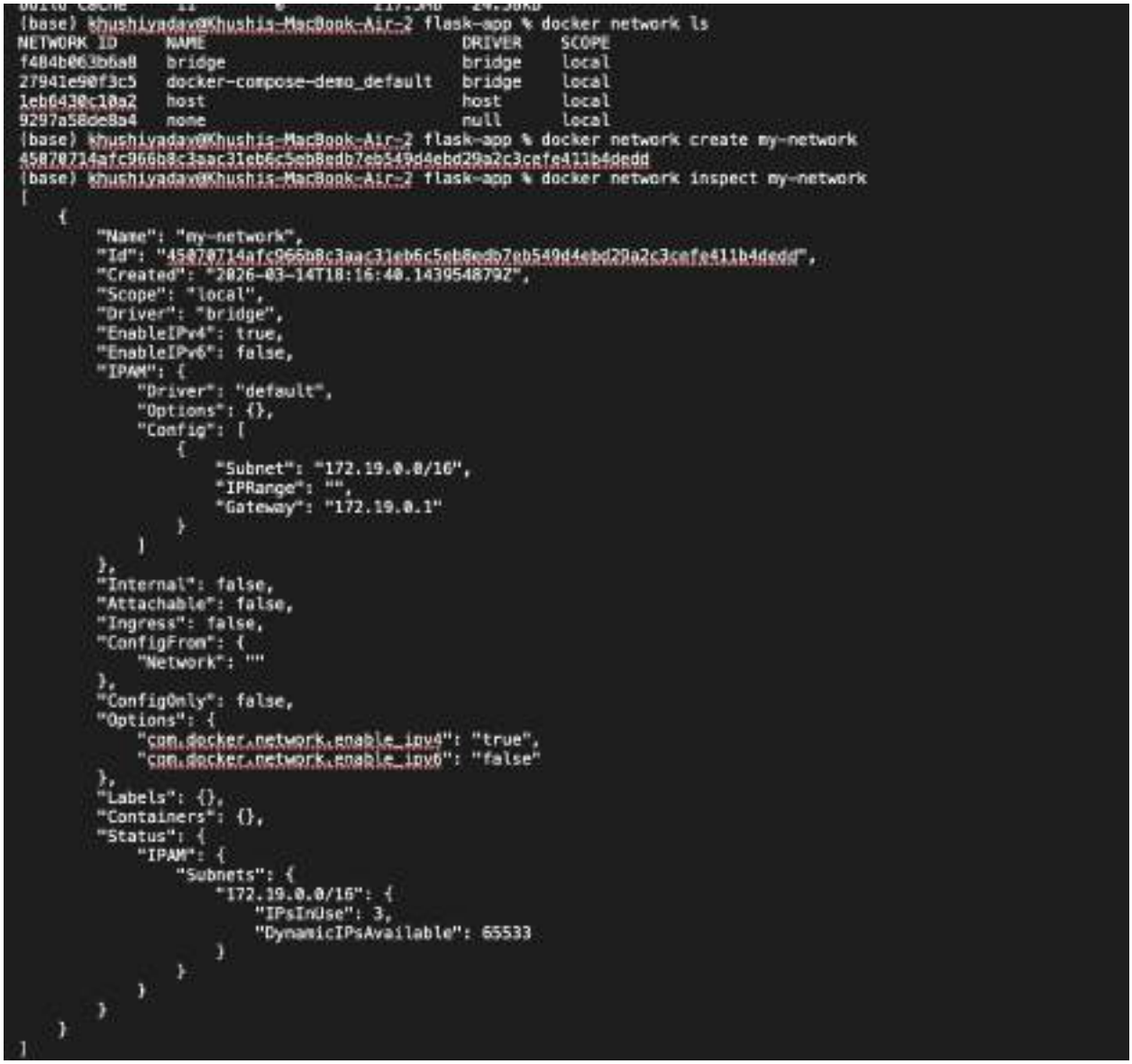
Docker networks enable secure, isolated communication between containers. Containers on the same custom network can reach each other using container names as hostnames — Docker's built-in DNS.

22.19.1 Lab 1: Network Types

22.19.1.1 Step 17 — List and Create Networks

```
docker network ls
# Output: bridge, host, none (defaults)

docker network create my-network
docker network inspect my-network
```



22.20

22.20.0.1 Step 17b — Container Communication via Custom Network

```
docker run -d --name net-web1 --network my-network nginx
docker run -d --name net-web2 --network my-network nginx

# Test communication using container name as hostname
```

```
docker exec net-web1 curl -s http://net-web2
# Output: nginx welcome HTML page
```

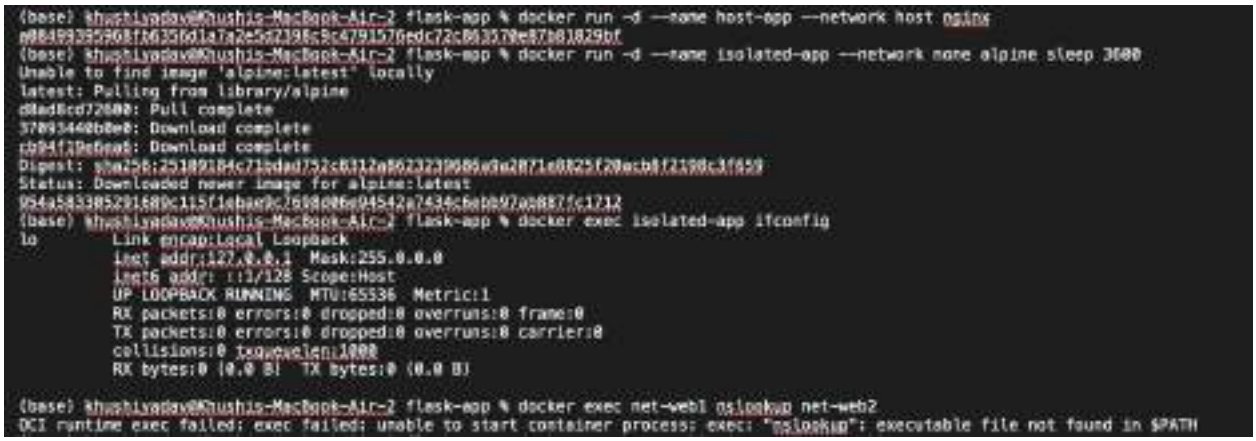

```
> Docker's built-in DNS resolved `net-web2` to its container IP automatically.

---

### Lab 2: Network Isolation

#### Step 18 - None Network (Full Isolation)

```bash
docker run -d --name isolated-app --network none alpine sleep 3600
docker exec isolated-app ifconfig
# Only loopback (lo) interface visible - no network access
```



22.21

22.21.0.1 Error: nslookup not found in nginx image

```
OCI runtime exec failed: exec: "nslookup": executable file not found in $PATH
```

22.21.0.2 Fix: Use docker network inspect instead

```
docker network inspect my-network
```

22.21.0.3 Error: ping not found in nginx image

```
OCI runtime exec failed: exec: "ping": executable file not found in $PATH
```

22.21.0.4 Fix: Connectivity already proven via curl

Container communication was already confirmed using:

```
docker exec net-web1 curl -s http://net-web2
```

22.21.0.5 Step 18b — Network Inspection

```
docker network inspect my-network
```

Output confirmed:
```

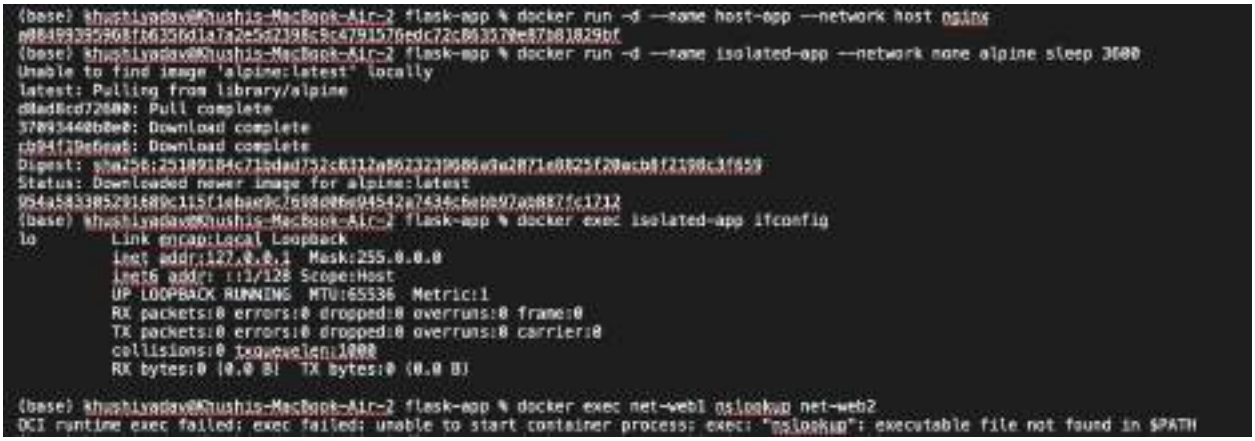
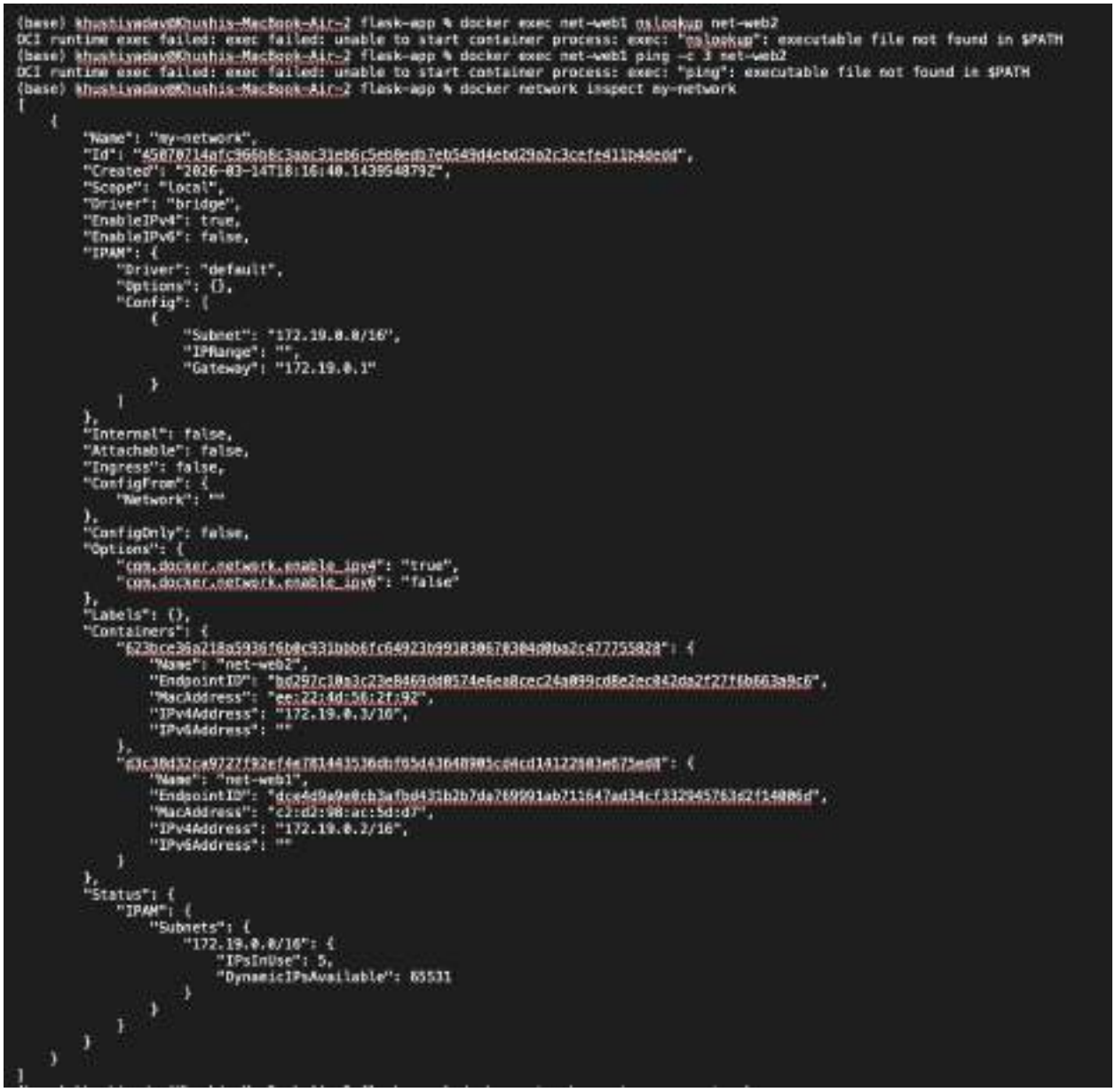


Figure 21: screenshot 2

```
net-web1 → 172.19.0.2/16
net-web2 → 172.19.0.3/16
Gateway  → 172.19.0.1
Subnet   → 172.19.0.0/16
```

22.21.0.6 Step 18c — Connect Existing Container to Network

```
docker network connect my-network flask-app
docker network inspect my-network
# flask-app now appears in Containers section
```



22.22

22.23 Part 5: Real-World Example

A full 3-tier application stack: **Flask** web app + **PostgreSQL** database + **Redis** cache — all connected via a custom Docker network with persistent volumes.

22.23.1 Architecture

```
          myapp-network


flask-app     postgres     redis
:5001→5000     :5432       :6379


          volume
```

```
postgres-data    redis-data
```

22.23.2 Step 19 — Deploy the Stack

22.23.2.1 Create Network

```
docker network create myapp-network
```

22.23.2.2 Start PostgreSQL

```
docker run -d \
  --name postgres \
  --network myapp-network \
  -e POSTGRES_PASSWORD=mysecretpassword \
  -e POSTGRES_DB=mydatabase \
  -v postgres-data:/var/lib/postgresql/data \
  postgres:15
```

22.23.2.3 Start Redis

```
docker run -d \
  --name redis \
  --network myapp-network \
  -v redis-data:/data \
  redis:7-alpine
```

22.23.2.4 Verify Services

```
docker ps | grep -E "postgres|redis"
```

Output:

```
redis:7-alpine   → Up  (port 6379)
postgres:15      → Up  (port 5432)
```

22.24 

22.24.1 Step 20 — Connect Flask and Verify Network

```
docker network connect myapp-network flask-app
docker network inspect myapp-network | grep "Name"
```

22.24.1.1 Error: curl not found in flask image

```
OCI runtime exec failed: exec: "curl": executable file not found in $PATH
```

Cause: `python:3.9-slim` is a minimal image — no curl installed.

22.24.1.2 Fix: Use docker network inspect

```
docker network inspect myapp-network
# Confirmed flask-app, postgres, and redis all listed under Containers
```

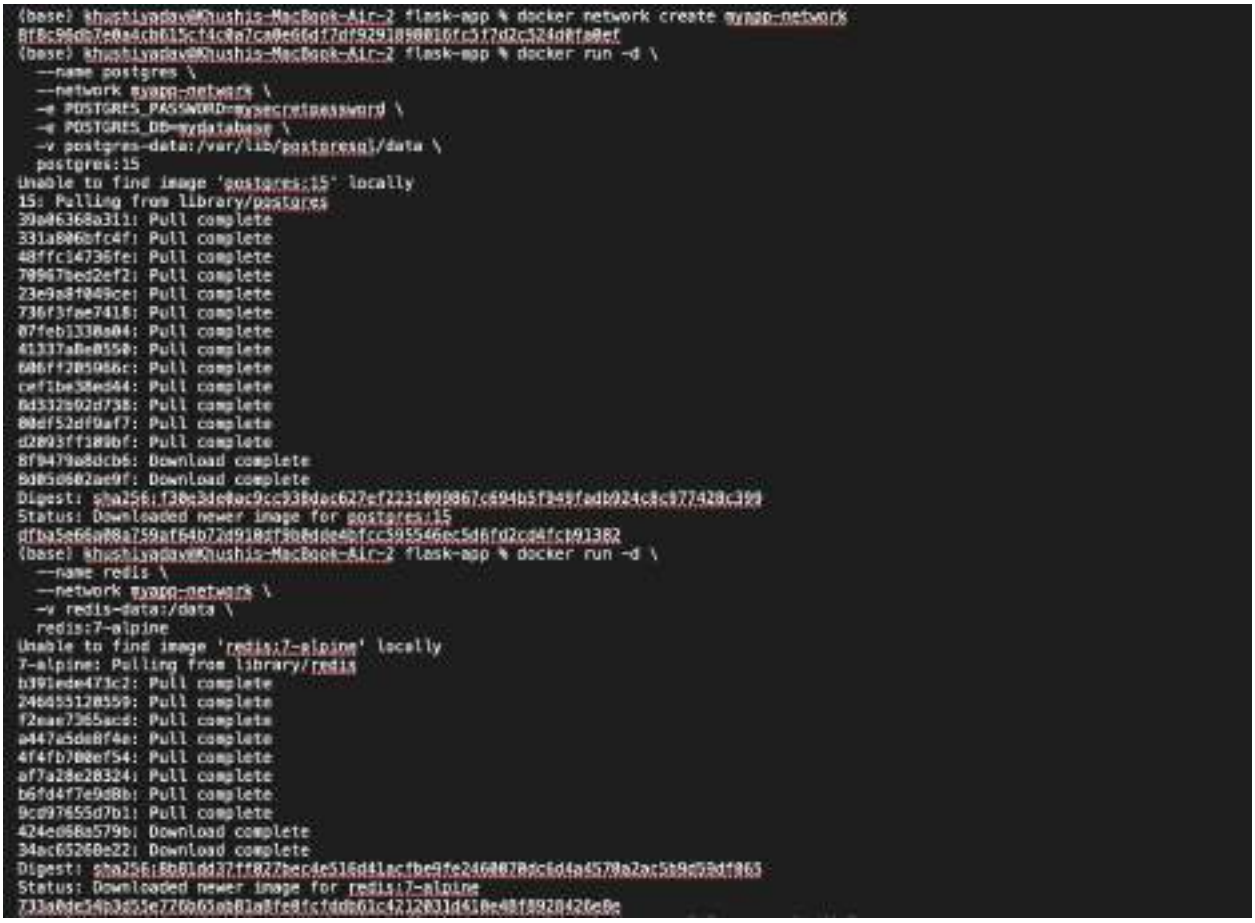


Figure 22: screenshot 2

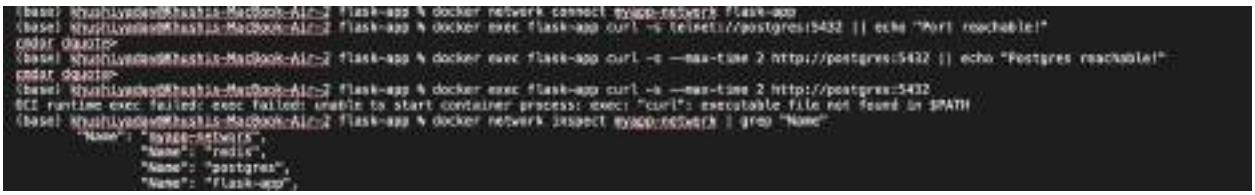


Figure 23: screenshot 2

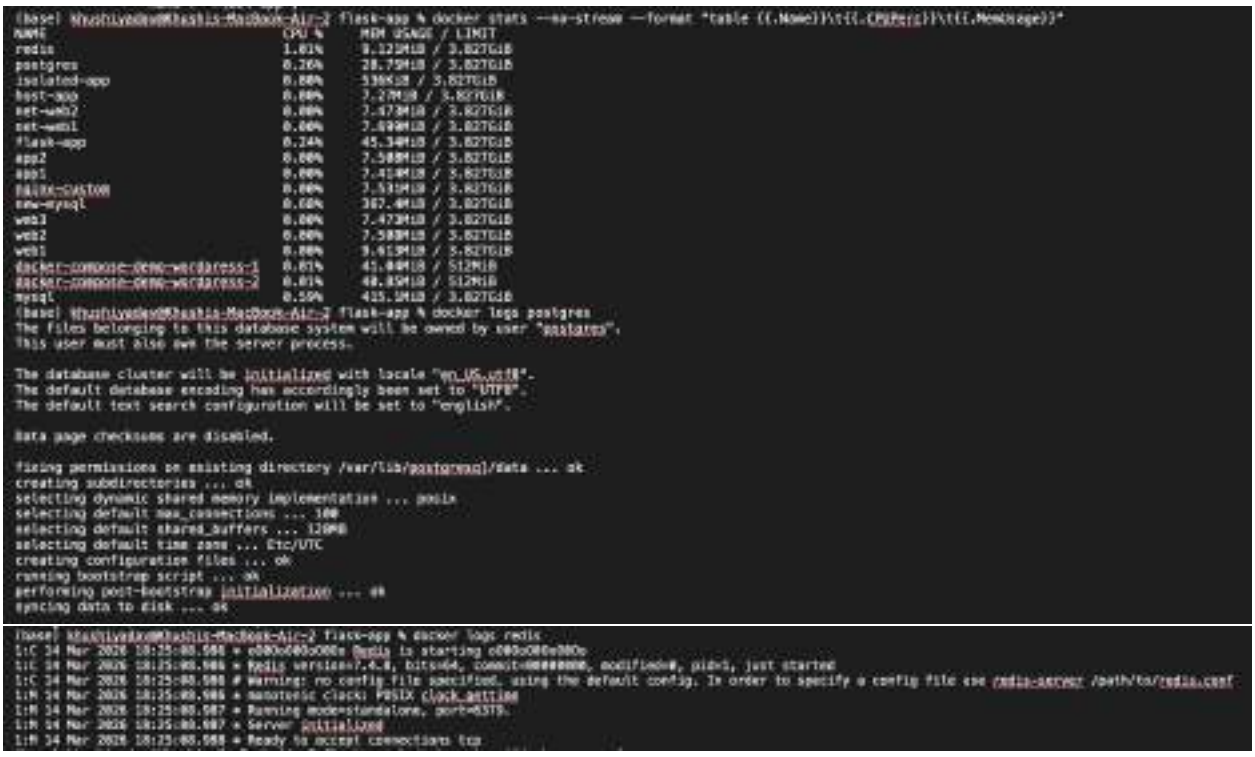
22.24.2 Step 20b — Verify Logs

```
docker logs postgres
# PostgreSQL 15 initialized, mydatabase created
# "database system is ready to accept connections"

docker logs redis
# Redis 7.4.8 started on port 6379
# "Ready to accept connections tcp"
```

22.24.3 Step 21 — Monitor All Services

```
docker stats --no-stream --format \
  "table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}"
```



22.25 Cleanup

```
# Stop and remove all containers
docker stop $(docker ps -aq)
docker rm $(docker ps -aq)

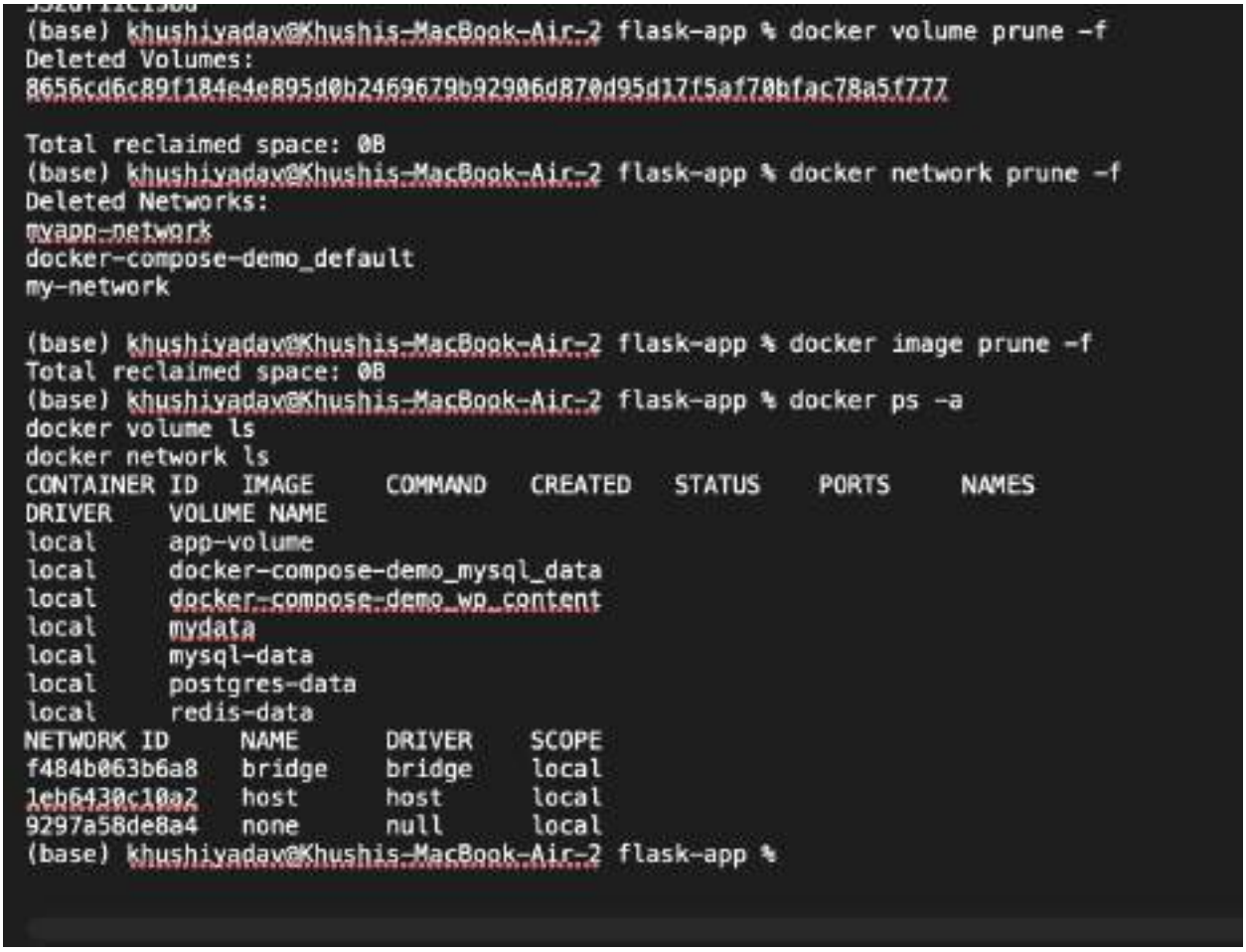
# Remove unused volumes (anonymous only - named volumes are kept)
docker volume prune -f

# Remove unused networks (except defaults)
docker network prune -f

# Remove unused images
```

```
docker image prune -f
```

```
(base) khushivadav@Khushis-MacBook-Air-2 flask-app % docker stop $(docker ps -aq)
733a0de54b3d
dfba5e66a08a
954a58330529
a08499395968
623bce36a218
d3c38d32ca97
e94ad32eff92
402c81084d27
40eff99d012f
1a8a47eb558c
be8a21564ea3
0fc7bbe0fd62
b02897f93df2
8ff634732c70
5c6eee0690d4
9447bf2ef71a
6ed19b50279e
332df11e19bd
(base) khushivadav@Khushis-MacBook-Air-2 flask-app % docker rm $(docker ps -aq)
733a0de54b3d
dfba5e66a08a
954a58330529
a08499395968
623bce36a218
d3c38d32ca97
e94ad32eff92
402c81084d27
40eff99d012f
1a8a47eb558c
be8a21564ea3
0fc7bbe0fd62
b02897f93df2
8ff634732c70
5c6eee0690d4
9447bf2ef71a
6ed19b50279e
332df11e19bd
```



After cleanup: - Containers → 0 remaining - Networks → only `bridge`, `host`, `none` - Named volumes retained (prune never removes named volumes)

22.26 Errors Encountered & Fixes

#	Error	Cause	Fix
1	Cannot remove running container	`docker rm` called while container was still running	Run `docker stop` first, then `docker rm`
2	Container exited immediately in detached mode	Python output buffering — Flask not emitting logs	Added `-u` flag: `CMD ["python", "-u", "app.py"]` and rebuilt
3	Port 5000 already in use	macOS AirPlay Receiver occupies port 5000 by default	Used `-p 5001:5000` to map host port 5001 to container 5000
4	`nslookup` not found in nginx image	Official nginx image is minimal — no DNS tools installed	Used `docker network inspect` to verify container IPs
5	`ping` not found in nginx image	Same reason — minimal nginx image	Connectivity already proven via `curl http://net-web2`

#	Error	Cause	Fix
6	`curl` not found in flask image	`python:3.9-slim` has no curl	Used `docker network inspect` to confirm network membership

---

22.27 Key Takeaways

1. **Volumes persist data** beyond container lifecycle — always use named volumes for production data
2. **Environment variables** configure containers dynamically — use `.env` files for sensitive config
3. **Custom networks** provide built-in DNS resolution — containers communicate using names
4. **Monitoring commands** (`stats`, `logs`, `top`, `inspect`) help debug and optimize containers
5. **Minimal Docker images** (nginx, python-slim) may lack common tools like curl, ping, nslookup
6. **macOS port 5000** is reserved by AirPlay — use an alternative port when running Flask locally
7. **`docker volume prune`** only removes anonymous volumes — named volumes are always preserved
8. **Multi-container apps** should always use custom networks for isolation and service discovery

---

23 Experiment 6 – Docker and Compose Exercises

23.1 Aim

Comparison of Docker Run and Docker Compose. ## Objective This lab explores container fundamentals through multiple parts:

- single container deployment with docker run
- Compose-based service definition for nginx and WordPress
- building a custom Node.js app image
- multi-container orchestration with persistent storage

---

24 Part 1 – Single Container Comparison

This section compares running nginx directly with docker run versus using docker-compose.

24.1 Step 1: Run nginx with docker run

```
docker run -d \
  --name lab-nginx \
  -p 8081:80 \
  -v "${PWD}/html:/usr/share/nginx/html" \
  nginx:alpine
```

Verify the container is running:

```
docker ps
```

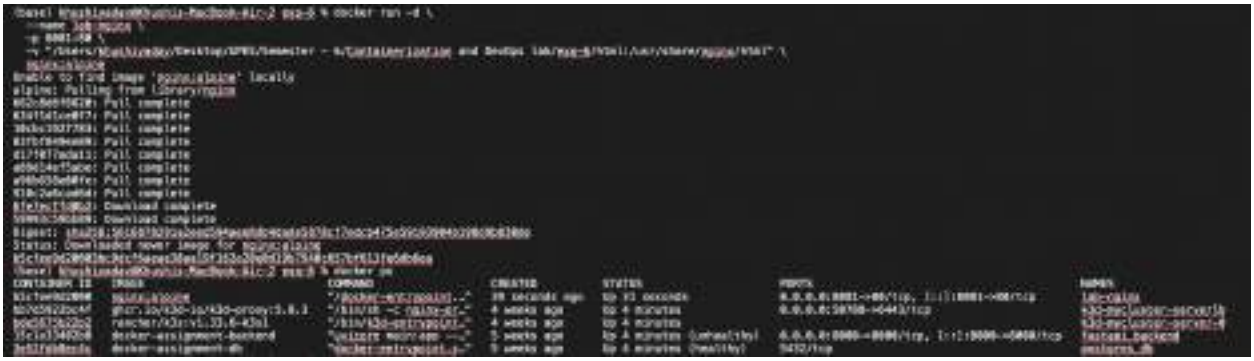
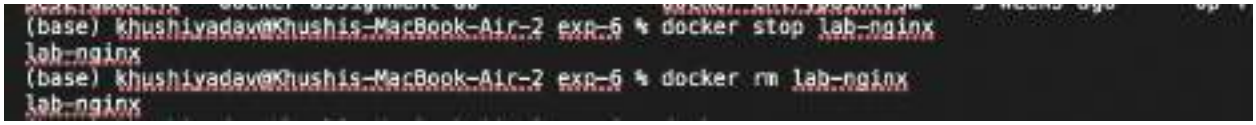


Figure 24: screenshot 1

Open in browser: http://localhost:8081

Stop and remove the container:

```
docker stop lab-nginx
docker rm lab-nginx
```



24.2

24.3 Step 2: Run nginx with docker-compose

Create or use the Compose file:

```
version: '3.8'
```

```
services:
  nginx:
    image: nginx:alpine
    container_name: lab-nginx
    ports:
      - "8081:80"
    volumes:
      - ./html:/usr/share/nginx/html
```

Run the Compose app:

```
docker compose up -d
```

Verify service status:

```
docker compose ps
```

Stop the service:

```
docker compose down
```

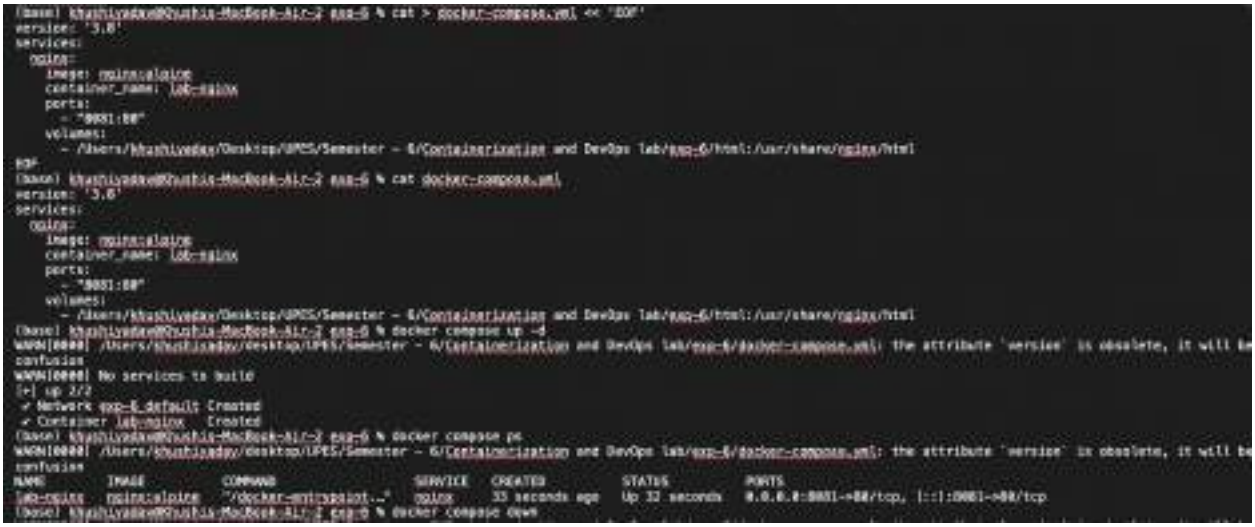


Figure 25: screenshot 1

Part 2 – Multi-Container Application (WordPress + MySQL)

This section demonstrates WordPress with MySQL using both docker run and docker-compose.
Task 2A: Run with docker run
Create a custom network: Run WordPress:
“‘bash
docker run -d –name wordpress –network wp-net -p 8082:80 -e WORDPRESS_DB_HOST=mysql -e WORDPRESS_DB_PASSWORD=secret wordpress:latest
“‘

Open in browser:

Task 2B: Run with docker-compose

Use the Compose file in Part B/docker-compose.yml:

```bash
version: '3.8'
services:
  mysql:
    image: mysql:5.7
    environment:
      MYSQL_ROOT_PASSWORD: secret
      MYSQL_DATABASE: wordpress
    volumes:
      - mysql_data:/var/lib/mysql
```

---

wordpress:
im-
age:
word-
press:latest
ports:
-
“8082:80”
envi-
ron-
ment:
-
WORD-
PRESS_DB_HOST=mysql
-
WORD-
PRESS_DB_PASSWORD=secret
de-
pends_on:
-
mysql
volumes:
mysql_data:
“‘
Start
the
stack:
`bash`
`docker`
`compose`
`up`
`-d`
Stop
and
re-
move
vol-
umes:
##
Part
3 —
Cus-
tom
Node.js
Build
(Part
C/Task
4)

This part builds a simple Express app into a Docker image.

Files involved

- Part C/Task 4/Dockerfile
- Part C/Task 4/server.js
- Part C/Task 4/package.json

Dockerfile

```bash
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 5000
CMD ["node", "server.js"]
```

###
server.js
“‘bash
const
ex-
press
= re-
quire(‘express’);
const
app
= ex-
press();
const
port
=
5000;
app.get(‘/’,
(req,
res)
=> {
res.send(‘Hello
from
your
Cus-
tom
Docker
Build!’);
});
app.listen(port,
() =>
{ con-
sole.log(`Server
running
at
http://localhost:${port}`);
});
###
Build
and
runbash
docker
build
-t lab-
node-
app
“Part
C/Task
4”

---

docker run -d -p 5000:5000 –name lab-node-app lab-node-app

```

Verify by opening: docker stop lab-node-app docker rm lab-node-app docker image rm lab-node-app

```

Part 4 — Persistent MySQL Volume (Part D/Task 5)

This task shows a Compose service with a named volume for MySQL data persistence.

Files involved

- Part D/Task 5/docker-compose.yml
- Part D/Task 5/Dockerfile
- Part D/Task 5/app.js

docker-compose.yml

```bash
version: '3.8'
```

services:
db:
im-
age:
mysql:5.7
con-
tainer_name:
mysql-
persistent
envi-
ron-
ment:
MYSQL_ROOT_PASSWORD:
lab-
pass-
word
MYSQL_DATABASE:
inven-
tory
vol-
umes:
-
db_data:/var/lib/mysql
restart:
al-
ways
volumes:
db_data:
‘‘‘
###
app.js
“‘bash
const
http
= re-
quire(‘http’);
http.createServer((req,
res)
=> {
res.end(‘Docker
Com-
pose
Build
Lab’);
}).lis-
ten(3000);
###
`Dockerfile`bash
FROM
node:18-
alpine
WORKDIR
/app

```
COPY app.js .
EXPOSE 3000
CMD ["node", "app.js"]
```

Image Gallery

The following images document the lab execution and results:

Notes

- The root docker-compose.yml in lab/exp6 is configured to build a Node app from a local Dockerfile.
- If that Dockerfile is not present in the root folder, use the Part C/Task 4 or Part D/Task 5 examples as working references for custom builds.
- Use docker compose ps, docker ps, and docker logs for trou

Lab 7: CI/CD Pipeline with Jenkins, GitHub, and Docker

ALL THE SCREENSHOTS ARE AT THE END OF THE EXPERIMENT

Objective

Build a complete CI/CD pipeline that automatically builds Docker images from GitHub code and pushes them to Docker Hub whenever code is committed.

24.4 Architecture

```
GitHub Repository
       ↓ (webhook on push)
  Jenkins Server
       ↓ (checkout & build)
Build Docker Image
       ↓ (push)
Docker Hub Registry
```

24.5 Process Done

24.5.1 Step 1: Jenkins Setup

Installed Jenkins using Docker with Docker socket access:

```
docker run -d --name jenkins -p 8080:8080 -p 50000:50000 \
  -v /var/run/docker.sock:/var/run/docker.sock \
  -v /usr/bin/docker:/usr/bin/docker \
  jenkins/jenkins:lts
```

Jenkins running at `http://localhost:8080`

24.5.2 Step 2: Installed Jenkins Plugins

Required plugins installed: - GitHub Plugin (webhook triggers) - Docker Pipeline (build Docker images) - Credentials Binding Plugin (secure credentials) - Pipeline Plugin (declarative pipelines)

All plugins active

24.5.3 Step 3: Added Docker Hub Credentials

Stored credentials securely in Jenkins: - **Manage Jenkins → Manage Credentials → Global credentials** - Created credential with ID: `docker-hub-creds` - Used Personal Access Token (not password for security)

Credentials stored

24.5.4 Step 4: Created GitHub Repository

Repository structure:

```
my-app/
  Dockerfile (multi-stage build)
  Jenkinsfile (pipeline configuration)
  src/ (application code)
  package.json (dependencies)
```

Repository ready

24.5.5 Step 5: Created Pipeline Job

Configured Jenkins pipeline: - Job name: `my-app-pipeline` - Type: Pipeline (Declarative) - SCM: Git - Script path: `Jenkinsfile`

Job created and configured

24.5.6 Step 6: Configured GitHub Webhook

Set up automatic trigger: - Webhook URL: `http://YOUR_JENKINS_IP:8080/github-webhook/` - Content type: JSON - Events: Push and Pull Request

Webhook verified (green checkmark)

24.6 Jenkinsfile Pipeline

```
pipeline {
    agent docker

    stages {
        stage('Checkout') {
            steps { checkout scm }
        }
```

```
        stage('Build & Test') {
            steps { sh 'npm install && npm test' }
        }

        stage('Build Docker Image') {
            steps { sh 'docker build -t myapp:${BUILD_NUMBER} .' }
        }

        stage('Push to Docker Hub') {
            steps {
                withCredentials([usernamePassword(credentialsId: 'docker-hub-creds', usernameVariable:
                    sh '''
                        echo $PASS | docker login -u $USER --password-stdin
                        docker tag myapp:${BUILD_NUMBER} $USER/myapp:latest
                        docker push $USER/myapp:latest
                    '''
                }
            }
        }
    }
}
```

24.7 How It Works

When code is pushed to GitHub:

1. **GitHub webhook** sends notification to Jenkins
2. **Jenkins** automatically checks out code
3. **Build stage** - compiles and runs tests
4. **Docker stage** - builds Docker image
5. **Push stage** - logs into Docker Hub and pushes image
6. **Cleanup stage** - removes old images

All **automatic** - no manual steps needed!

24.8 Testing

24.8.1 Manual Build

`Jenkins Dashboard → my-app-pipeline → Build Now`

Build succeeds → Image pushed to Docker Hub

24.8.2 Automated Trigger

```
git add .
git commit -m "Test trigger"
git push origin main
```

Webhook triggers → Build starts → Image pushed automatically

24.9 Quick Commands

```
# Check Jenkins
curl http://localhost:8080

# View logs
docker logs -f jenkins

# Manually build
curl -X POST http://localhost:8080/job/my-app-pipeline/build

# View image
docker pull YOUR_USERNAME/myapp:latest
```

24.10 Key Features

Automated builds on every GitHub push
Secure credentials stored in Jenkins
Docker images built automatically
Images pushed to Docker Hub
Webhook triggers instant builds
Build logs visible for debugging
Scalable for multiple projects

24.11 Conclusion

Complete CI/CD pipeline is now functional. Any code pushed to GitHub automatically: 1. Triggers Jenkins build 2. Builds Docker image 3. Pushes to Docker Hub

No manual intervention required!

Screenshots

Lab 9: Docker Container with SSH Configuration and Ansible Deployment

Objective

To create a Docker container with SSH server capabilities and configure it using Ansible for infrastructure automation and server management.

Tools and Technologies Used - **Docker**: For containerization - **Ansible**: For configuration management and automation - **Ubuntu**: Base operating system - **SSH**: Secure Shell for remote access - **Python**: Scripting language

24.12 Experiment Steps

24.12.1 Step 1: Build the Docker Image from Dockerfile

Description: Create a Docker image that includes SSH server, Python 3, and SSH key authentication.

Command:

```
docker build -t ubuntu-server .
```

Expected Output: Image successfully built Step 1

24.12.2 Step 2: Run the Docker Container

Description: Launch the Docker container with SSH port mapping and resource constraints.

Command:

```
docker run -d --rm -p 2222:22 -p 8221:8221 --name ssh-test-server ubuntu-server
```

Parameters Explained: - `-d`: Run in detached mode (background) - `--rm`: Automatically remove container when it exits - `-p 2222:22`: Map host port 2222 to container SSH port 22 - `-p 8221:8221`: Map additional port for application services - `--name ssh-test-server`: Container name for easy reference - `ubuntu-server`: Image name to run

Expected Output: Container ID returned Step 2

24.12.3 Step 3: Verify Container Status

Description: Check if the container is running and ports are properly mapped.

Command:

```
docker ps
```

Expected Output: Running container with port mappings visible Step 3

24.12.4 Step 4: Test SSH Connection to Container

Description: Establish SSH connection to the running container using port 2222.

Command:

```
ssh -p 2222 root@localhost
```

Expected Output: SSH connection prompt (you may need to accept key fingerprint)

```
The authenticity of host '[localhost]:2222 ([127.0.0.1]:2222)' can't be established.
ED25519 key fingerprint is SHA256:WWmJqPAsxsn8ARhMA0J1M1e+wIqqVh262gnQ6B/Do5s.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
```

Step 4

Figure 26: Step 4

24.12.5 Step 5: Configure Ansible Inventory

Description: Set up the inventory file with container host information for Ansible to connect to.

Inventory File Content (`inventory.ini`):

```
[servers]
localhost:2222
```

```
[servers:vars]
ansible_user=root
ansible_ssh_private_key_file=~/.ssh/id_rsa
ansible_python_interpreter=/usr/bin/python3
```

Key Configuration: - Define server group for organization - Specify SSH user and private key path - Set Python interpreter path

Step 5

Figure 27: Step 5

24.12.6 Step 6: Run Ansible Playbook on Container

Description: Execute Ansible playbook to perform automated configuration tasks on the container.

Command:

```
ansible-playbook -i inventory.ini playbook1.yml
```

Playbook Tasks (from `playbook1.yml`): 1. Update apt package index 2. Install Python 3 (latest available) 3. Create test configuration file with server metadata 4. Execute system information commands (uname, df) 5. Display results for verification

Expected Output: Playbook execution report with task results Step 6

24.12.7 Step 7: Verify Automation Results

Description: Check if all Ansible tasks completed successfully and verify the configuration applied.

Verification Commands:

```
# Check test file was created
ssh -p 2222 root@localhost "cat /root/test_file.txt"

# Verify Python installation
ssh -p 2222 root@localhost "python3 --version"

# Check system packages updated
ssh -p 2222 root@localhost "apt list --upgradable"
```

Expected Output: Confirmation that all tasks executed successfully Step 7

24.13 Cleanup Commands

Stop and remove the container:

```
docker stop ssh-test-server
docker rm ssh-test-server
```

Remove the Docker image:

```
docker rmi ubuntu-server
```

24.14 Key Learnings

1. **Docker SSH Configuration**: Setting up SSH in Docker requires proper key permissions (600 for private keys, 644 for public keys).
2. **Port Mapping**: Docker allows flexible port mapping for accessing services running in containers.
3. **Ansible Automation**: Ansible simplifies infrastructure management by allowing declarative configuration of multiple servers.
4. **Infrastructure as Code**: Using Ansible playbooks provides reproducible, version-controlled infrastructure setup.
5. **Security Considerations**: Using SSH key-based authentication is more secure than password-based authentication.

24.15 Troubleshooting

24.15.1 Issue: Permission Denied (publickey)

Solution: Ensure SSH public key is in `/root/.ssh/authorized_keys` with correct permissions (644).

24.15.2 Issue: Connection Refused

Solution: Verify container is running with `docker ps` and SSH service is active inside container.

24.15.3 Issue: Ansible Connection Failed

Solution: Check inventory.ini file paths and ensure private key file exists at specified location.

24.16 Conclusion

This lab successfully demonstrates the integration of Docker containers with Ansible for automated infrastructure management. The combination allows for scalable, repeatable deployments across multiple containerized environments, making it a powerful approach for DevOps workflows.

25 Lab 10: CI/CD Pipeline with Jenkins and SonarQube

25.1 Introduction

This lab demonstrates how to automate the build, test, and code quality analysis of a Java application using Jenkins and SonarQube. By the end, you will have a working pipeline that checks your code for errors and quality issues every time you push changes.

25.2 Prerequisites

- Docker and Docker Compose installed
- Basic knowledge of Java and Maven
- Familiarity with Jenkins and SonarQube concepts

25.3 Project Structure

```
Lab10/
  docker-compose.yml
  jenkinsfile
  readme.md
  sample-java-app/
```

```
    pom.xml
    src/
        main/
            java/
...
```

25.4 Step-by-Step Instructions

1. **Clone the repository**

   ```
   git clone <repo-url>
   cd Lab10
   ```

2. **Start Jenkins and SonarQube using Docker Compose**

   ```
   docker-compose up -d
   ```

3. **Access Jenkins**

   - Open your browser and go to `http://localhost:8080`
   - Complete the initial setup and install recommended plugins

4. **Access SonarQube**

   - Go to `http://localhost:9000`
   - Log in with default credentials (admin/admin)

5. **Configure Jenkins Pipeline**

   - Create a new pipeline job
   - Use the provided `jenkinsfile` for pipeline configuration

6. **Run the Pipeline**

   - Trigger a build and observe the stages: build, test, and code analysis

25.5 Jenkins Pipeline Overview

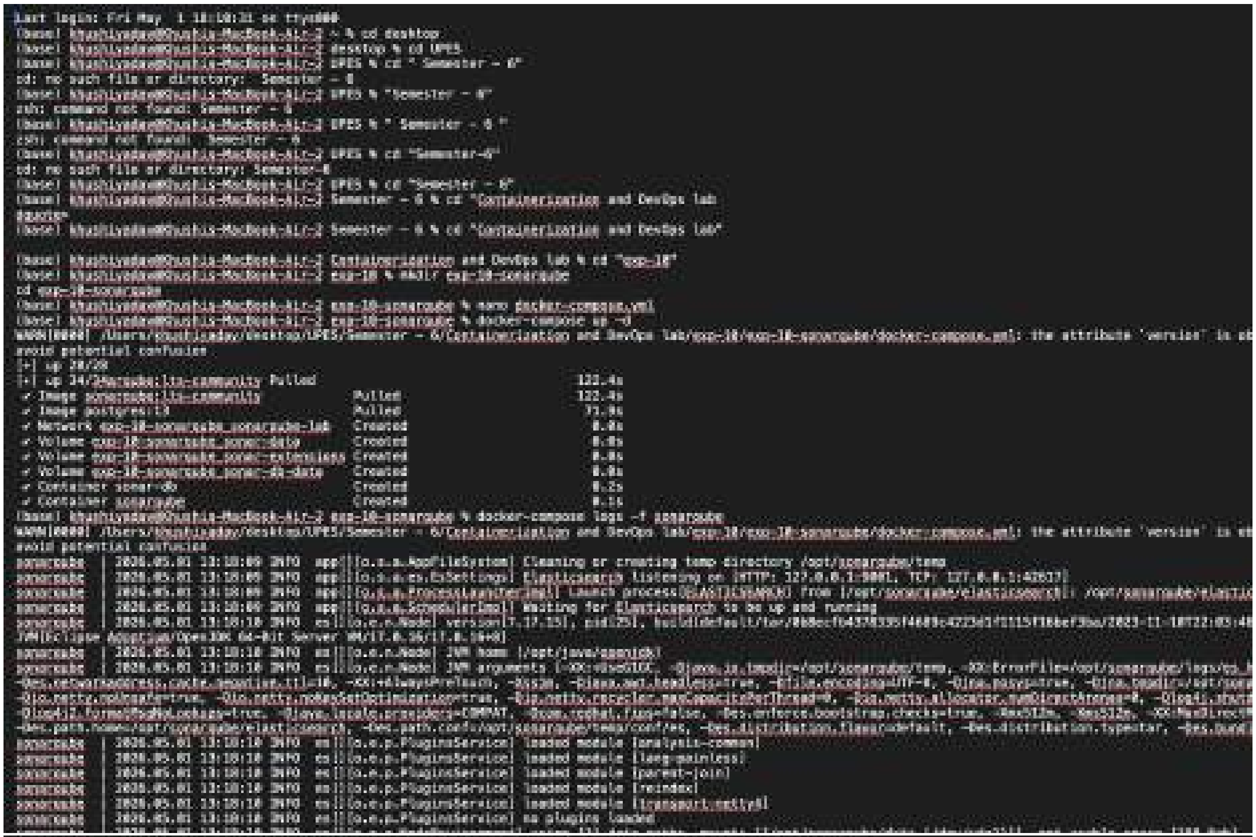
The Jenkins pipeline defined in `jenkinsfile` automates the following tasks: - Build the Java application using Maven - Run unit tests - Analyze code quality with SonarQube - Archive build artifacts

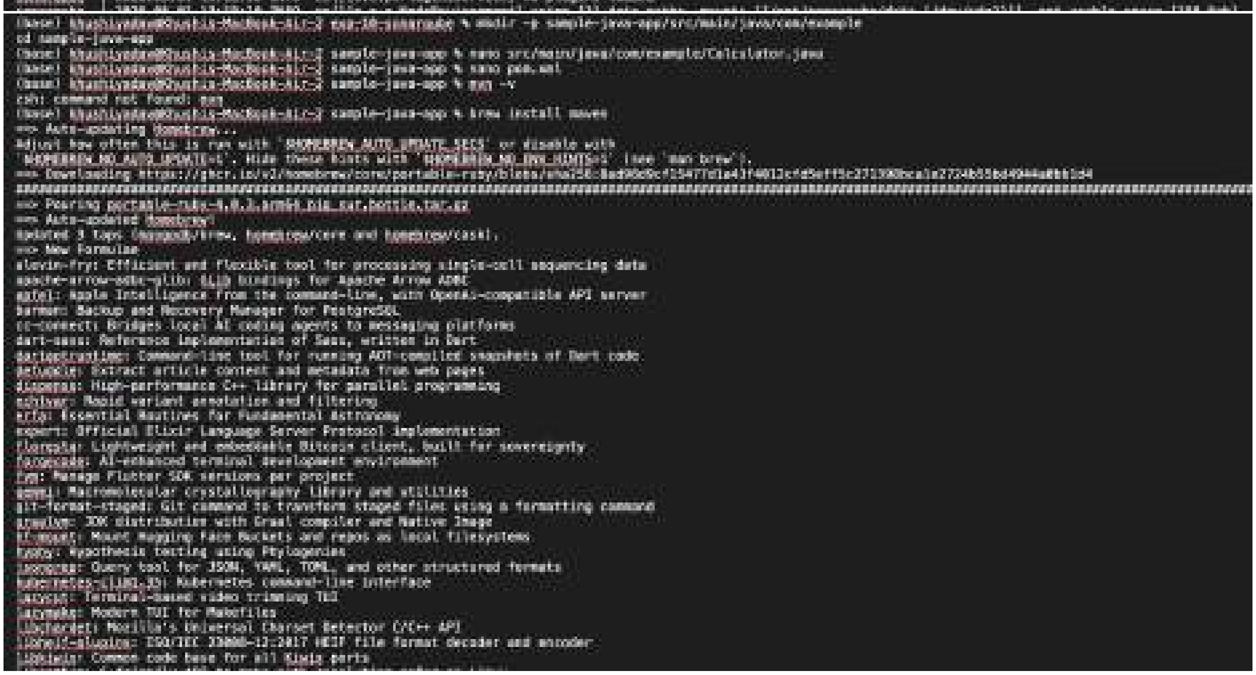
25.6 SonarQube Integration

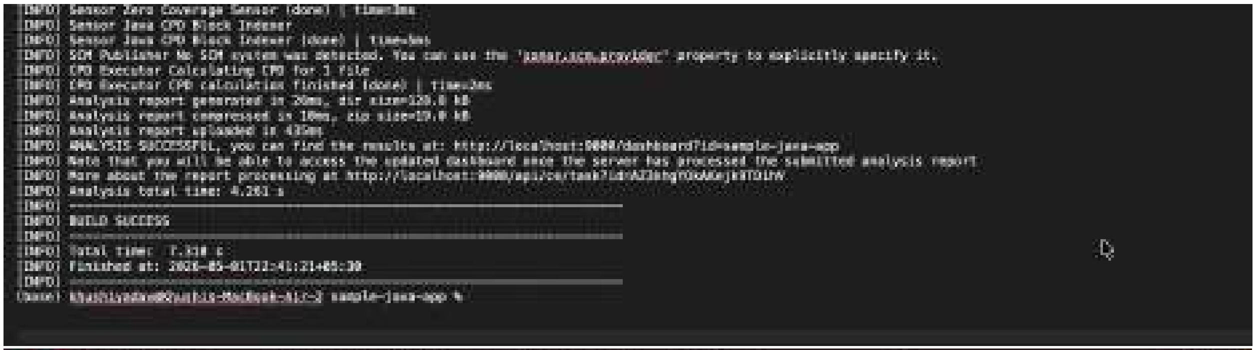
SonarQube is used to analyze the code for bugs, vulnerabilities, and code smells. The pipeline sends analysis results to SonarQube, which you can review in the SonarQube dashboard.

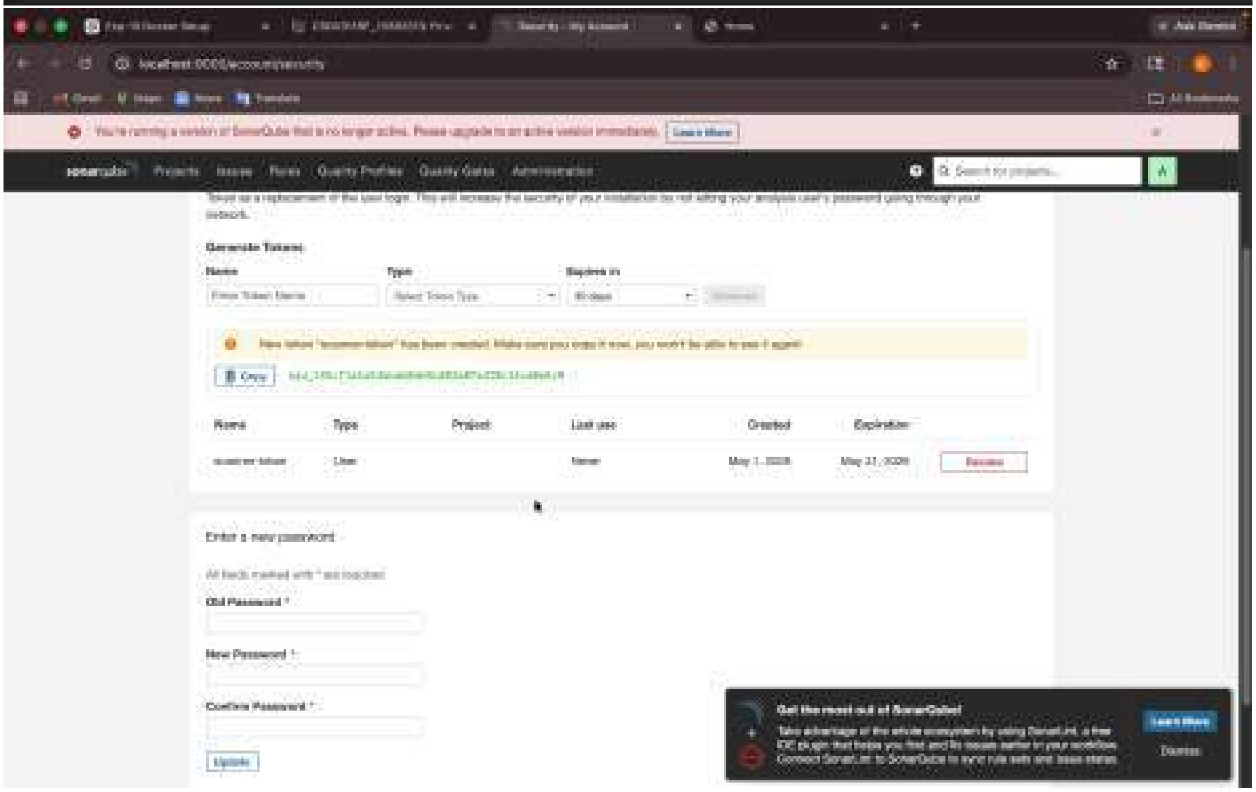
25.7 Screenshots

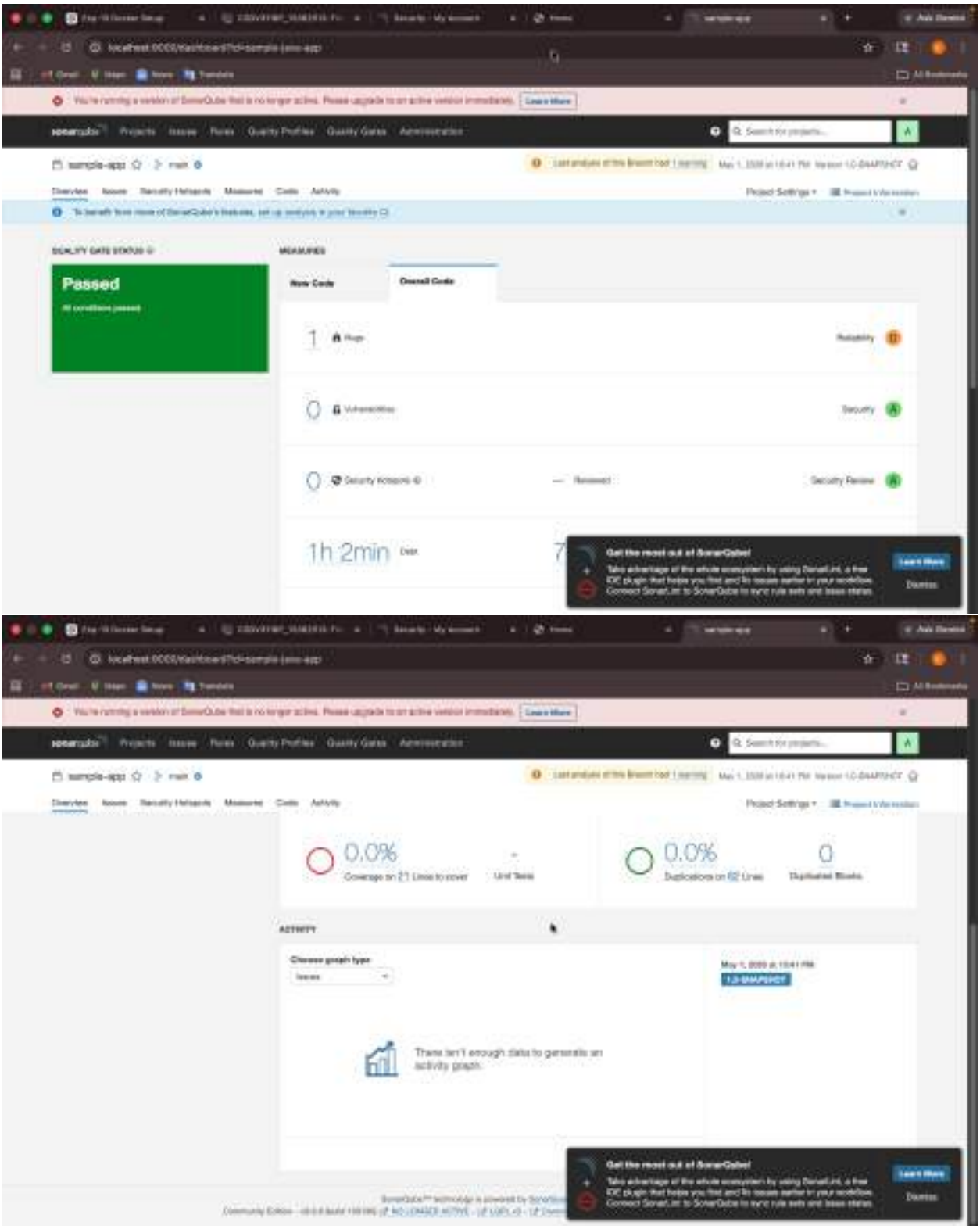
Below are screenshots showing the setup and results at each stage:


cd: no such file or directory: Semester - 6
zsh: command not found: Semester - 6
avoid potential confusion
✓ Image postgres:15
docker-compose up -d


cd sample-java-app
sample-java-app % nano pom.xml
sample-java-app % brew install maven
==> New Formulae
alevin-fry: Efficient and flexible tool for processing single-cell sequencing data
apache-arrow-adbc-glib: GLib bindings for Apache Arrow ADBC
Barman: Backup and Recovery Manager for PostgreSQL
cc-connect: Bridges local AI coding agents to messaging platforms
dart-sass: Reference implementation of Sass, written in Dart


[INFO] Sensor Java CPD Block Indexer
[INFO] CPD Executor Calculating CPD for 1 file
[INFO] Analysis total time: 4.261 s
[INFO] BUILD SUCCESS
sample-java-app %


Projects
Issues
Rules
Quality Profiles
Quality Gates
Administration
Generate Tokens
Name
Type
Expires in
Copy
Name
Type
Project
Last use
Created
Expiration
Revoke
Enter a new password
Old Password *
New Password *
Confirm Password *
Update
Learn More
Dismiss



25.8 Troubleshooting

- If Jenkins or SonarQube do not start, check Docker logs for errors.
- Ensure ports 8080 (Jenkins) and 9000 (SonarQube) are not in use by other applications.

- If SonarQube analysis fails, verify the SonarQube server URL and authentication token in Jenkins.

25.9 Conclusion

You have now set up a basic CI/CD pipeline with Jenkins and SonarQube for a Java application. This setup can be extended to include more advanced features like deployment, notifications, and additional quality gates.

Feel free to experiment and modify the pipeline to suit your project needs. Happy coding!

26 Lab 11: Deploying WordPress with Docker Swarm

26.1 Introduction

This experiment demonstrates how to deploy a WordPress website using Docker Swarm for container orchestration. You will set up a multi-container application with WordPress and MySQL, leveraging Docker Swarm's clustering and scaling features.

26.2 Objectives

- Initialize a Docker Swarm cluster
- Deploy WordPress and MySQL as services
- Use persistent storage with Docker volumes
- Access the WordPress site from your browser

26.3 Prerequisites

- Docker and Docker Compose installed
- Basic knowledge of Docker concepts
- Access to a terminal with administrative privileges

26.4 Steps

26.4.1 1. Initialize Docker Swarm

If your system has multiple network interfaces, specify the IP address to advertise:

```
docker swarm init --advertise-addr <YOUR_IP_ADDRESS>
```

Replace `<YOUR_IP_ADDRESS>` with your main network IP (e.g., 172.30.10.156).

26.4.2 2. Deploy the Stack

Use the provided `docker-compose.yml` to deploy the stack as a Swarm stack:

```
docker stack deploy -c docker-compose.yml wordpress_stack
```

26.4.3 3. Check Service Status

List the running services:

```
docker service ls
```

26.4.4 4. Access WordPress

- Open your browser and go to `http://<YOUR_IP_ADDRESS>:8080`
- Complete the WordPress setup wizard

26.4.5 5. Scaling Services (Optional)

To scale the WordPress service:

```
docker service scale wordpress_stack_wordpress=2
```

26.5 docker-compose.yml Overview

- **db**: Runs MySQL with persistent storage
- **wordpress**: Runs the WordPress application, depends on the database
- **Volumes**: `db_data` and `wp_data` ensure data persists across restarts

---

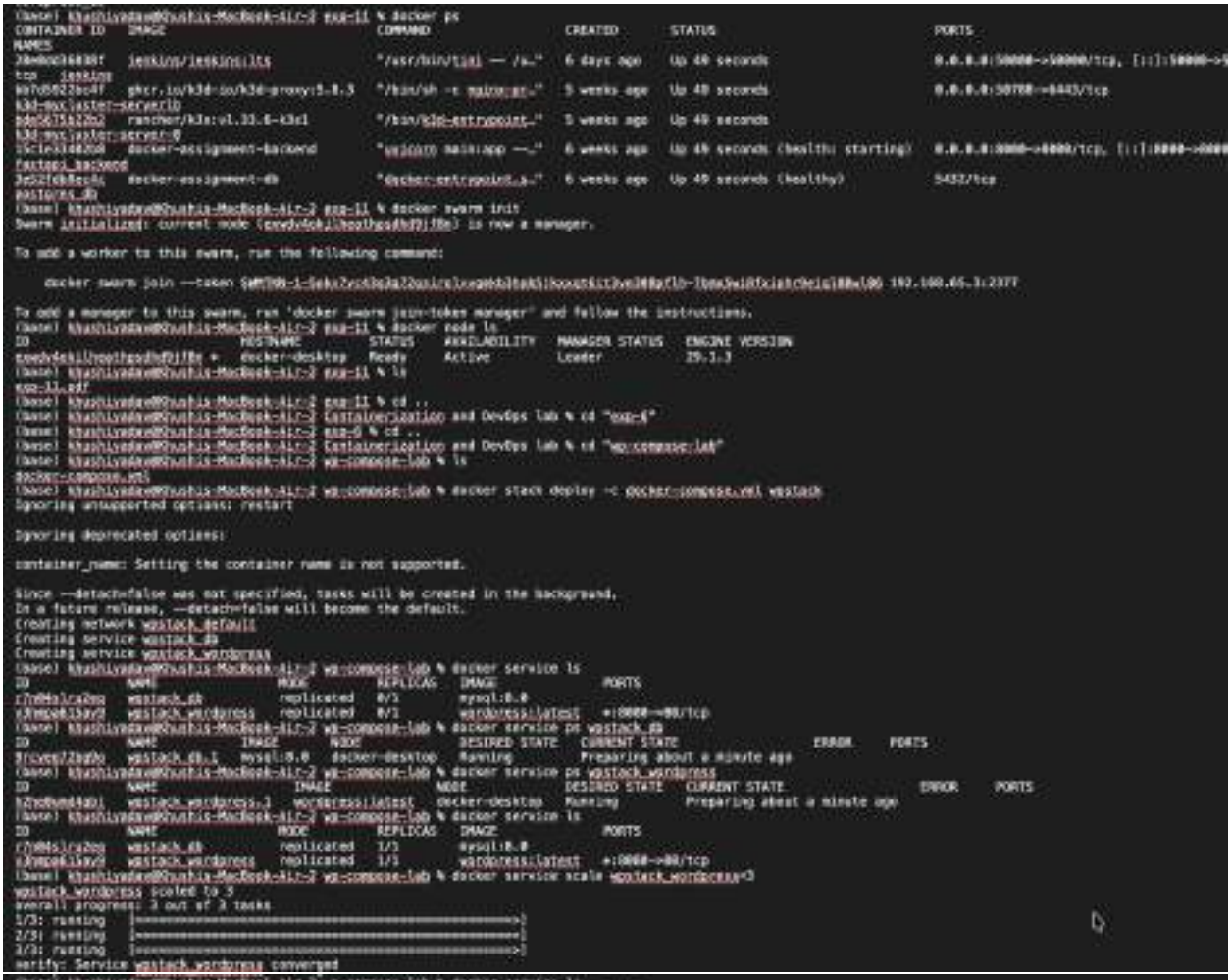
26.6 Troubleshooting

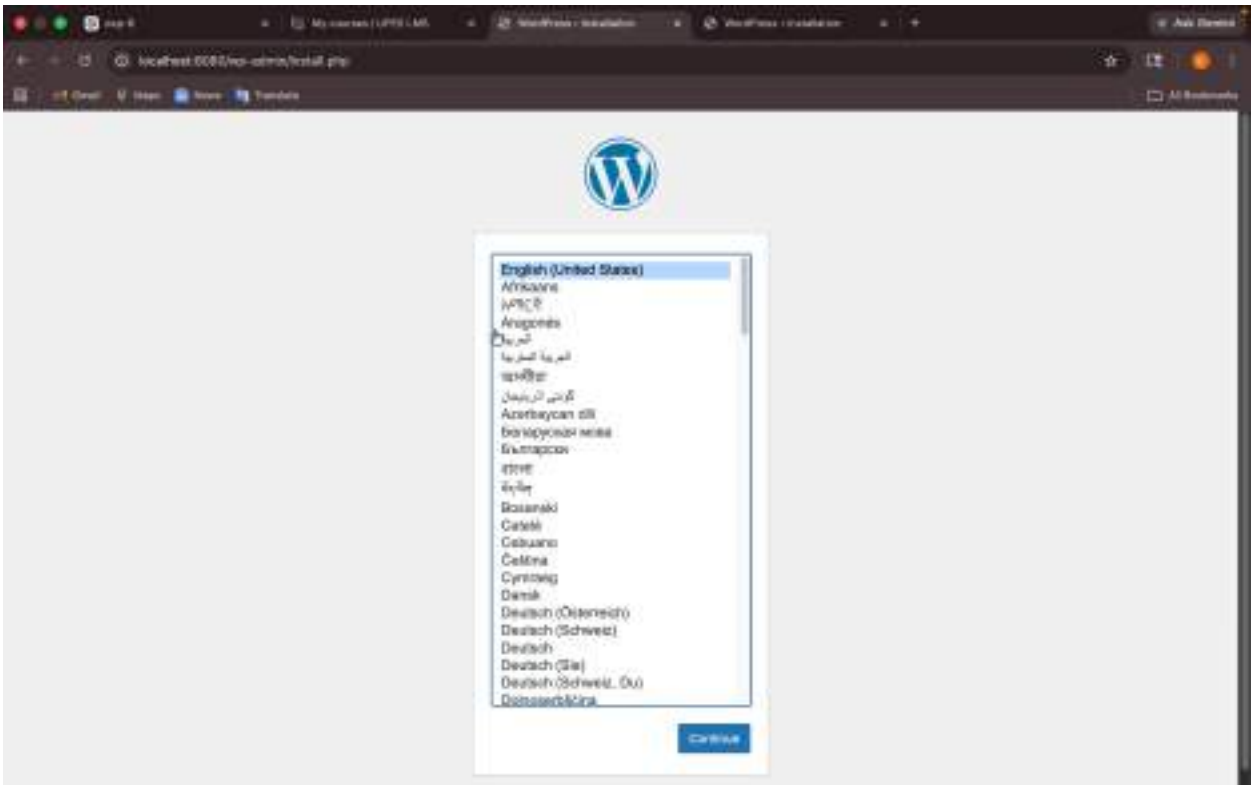
- If you see an error about multiple IP addresses, use `--advertise-addr` as shown above.
- Ensure ports 8080 (WordPress) and 3306 (MySQL) are not blocked or in use.
- Use `docker service ps <SERVICE_NAME>` to check for errors in service startup.

26.7 Conclusion

You have successfully deployed a WordPress site using Docker Swarm. This setup can be extended to include more services, enable scaling, and provide high availability.

26.8 Screenshots





Feel free to experiment with scaling and updating services to explore more Docker Swarm features!

27 Kubernetes Lab Experiment – Deploying WordPress using kubeadm

27.1 Overview

This experiment demonstrates the fundamentals of Kubernetes by deploying a real-world application (WordPress) using core Kubernetes objects like Deployments and Services. It also explores scaling and self-healing capabilities of Kubernetes in a cluster created using `kubeadm`.

27.2 Objectives

- Understand basic Kubernetes architecture and components
- Set up a Kubernetes cluster using `kubeadm`
- Deploy a WordPress application using a Deployment
- Expose the application using a Service
- Perform scaling operations
- Observe self-healing behavior of Kubernetes

27.3 Prerequisites

- Linux environment (Ubuntu / WSL / Virtual Machine)
- Container runtime (Docker / containerd)

- Installed tools:
  - `kubectl`
  - `kubeadm`
  - `kubelet`
- Minimum 2 GB RAM recommended
- Internet connectivity

27.4 Cluster Setup using kubeadm

27.4.1 Step 1: Initialize the Kubernetes Cluster

```
sudo kubeadm init
```

27.4.2 Step 2: Configure kubectl for the user

```
mkdir -p $HOME/.kube
sudo cp /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

27.4.3 Step 3: Install Network Plugin (Calico)

```
kubectl apply -f https://docs.projectcalico.org/manifests/calico.yaml
```

27.5 Application Deployment (WordPress)

27.5.1 Step 4: Deploy WordPress

```
kubectl apply -f wordpress-deployment.yaml
```

27.5.2 Step 5: Expose WordPress Service

```
kubectl apply -f wordpress-service.yaml
```

27.6 Verification Commands

27.6.1 Check Nodes

```
kubectl get nodes
```

27.6.2 Check Pods

```
kubectl get pods
```

27.6.3 Check Services

```
kubectl get svc
```

27.7 Scaling the Application

Increase the number of replicas:

```
kubectl scale deployment wordpress --replicas=3
```

Verify scaling:

```
kubectl get pods
```

27.8 Self-Healing Test

Delete a running pod:

```
kubectl delete pod <pod-name>
```

Kubernetes will automatically recreate the pod to maintain the desired state.

27.9 Observations

- Deployments manage pod lifecycle automatically
- Services provide stable network access to applications
- Kubernetes maintains the desired state continuously
- Pods are recreated automatically when they fail (self-healing)
- Scaling can be done dynamically without downtime

27.10 Issues Faced During Experiment

- Kubernetes cluster initialization failure using `kubeadm`
- API Server connection refused errors (`connection refused 6443`)
- Problems due to WSL environment limitations
- Container runtime and system configuration issues

27.11 Conclusion

In this experiment, we successfully:

- Understood Kubernetes architecture and working
- Deployed a real-world application (WordPress)
- Performed scaling operations
- Tested self-healing capability
- Explored cluster setup using `kubeadm` ## Screenshots screenshot screenshot screenshot screenshot